webMobaLedLib – Installation und Betrieb

Datei-Version: 3.70 (2026-11-12), gilt für webMobaLedLib 3.23

Windows · macOS · Linux-Desktop · Raspberry Pi

Der Programmgenerator läuft als Web-Oberfläche: Das Programm startet auf dem Rechner, an dem der Arduino per USB hängt, und du bedienst es im Browser (Edge, Chrome, Firefox, Safari). Es ersetzt die bisherige Tk-Version und enthält deren Programmmodule. Diese Anleitung gilt für alle vier Systeme. **Eilige lesen nur den Schnellstart.** Wer es genauer wissen will, liest Kapitel 1 und 2 und danach das Kapitel für sein System. Betatester finden eine Checkliste in `install/BETATEST_Checkliste.md`.

Inhalt

Schnellstart für Eilige

1. Welches System, welches Skript?

2. Gemeinsame Grundlagen

- 2.1 Dateien und Ordnerstruktur
- 2.2 Die ARDUINO IDE: Version 1.8.19, nicht 2.x
- 2.3 Bibliotheken, Boards und die Tabelle „Libraries“
- 2.4 Starten, Beenden, Speichern
- 2.5 Das Programmsymbol
- 2.6 Wo liegen meine Daten? (Datenordner)
- 2.7 Protokoll (Log)
- 2.8 Kleine Bildschirme: kompaktes Layout
- 2.9 Bedienung der Tabelle: Makroauswahl, Bearbeiten, Kontextmenü
- 2.9a Blattreiter: Farbe, Umbenennen, Duplizieren, Verschieben (2.63)
- 2.9b Mehrere Zeilen markieren, kopieren und ausblenden (2.67)
- 2.9c Die Knöpfe der Werkzeugleiste (2.68)
- 2.9d Programme für den rechten Arduino (DCC, Selectrix, LNet) mit PlatformIO (2.72)
- 2.9e Schneller Bootloader mit PlatformIO (2.74)
- 2.9f Servo Animation (Makros ServoAnim und ServoAnim2B, 2.78)
- 2.9g LED Farb-Animation (Makro LEDColorAnim, 2.88)
- 2.10 Der Patterngenerator
- 2.11 Vollbild und Adressen der Reiter
- 2.12 Servo Einstellungen
- 2.13 DM Servo Einstellungen, Sound Test und DCC Keyboard
- 2.14 Z21 Simulator
- 2.15 Sprache der Oberfläche
- 2.16 Entwicklungsumgebung: ARDUINO IDE oder PlatformIO
- 2.17 Anzahl der Eingänge (Excel-Programm 3.5.0-K-long)
- 2.17 Grafische Ansicht im Programm Generator (neu)

3. Windows

4. macOS

5. Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch …)

6. Raspberry Pi (Dauerdienst)

- 6.1 Dateien auf den Pi bringen
- 6.2 Installation (ein Befehl)
- 6.3 Das Desktop-Symbol
- 6.4 ARDUINO IDE und Bibliotheken auf dem Pi
- 6.5 Alltag auf dem Pi
- 6.6 Raspberry Pi herunterfahren
- 6.7 Den Raspberry Pi im Netz finden

7. Aktualisieren und Deinstallieren

- 7.1 Update über das Programm (Hilfe → Update webMobaLedLib …)
- 7.1a Testversion (Hilfe → Update Testversion …)

8. Zugriff aus dem Netz und Passwort

9. Fehlersuche

- 9.1 Alle Systeme
- 9.2 Windows
- 9.3 macOS
- 9.4 Linux-Desktop
- 9.5 Raspberry Pi
- 9.6 Wenn nichts mehr geht (Installation beschädigt oder Update unterbrochen)

10. Was nicht oder noch nicht geht

Schnellstart für Eilige

Vorbereitung (für alle Systeme)

1. Die Datei **`webmobaledlib.zip`** aus dem **neuesten Release** herunterladen: https://github.com/haroldlinke/WebMLL/releases (dort beim obersten Eintrag unter „Assets“ auf `webmobaledlib.zip` klicken). Direkter Download der neuesten Version: https://github.com/haroldlinke/WebMLL/releases/latest/download/webmobaledlib.zip
2. Einen **neuen Ordner** **`mobaledlib`** anlegen (zum Beispiel im eigenen Benutzerordner), die ZIP-Datei dort hineinkopieren und **entpacken**: Windows Rechtsklick → *Alle extrahieren* (im vorgeschlagenen Zielpfad ein angehängtes `\webmobaledlib` löschen), Linux und Mac „Hier entpacken“. Danach liegt der Ordner `python` direkt in `mobaledlib`.
3. Die **Komplettinstallation** mit einem Befehl starten (Tabelle unten). Sie richtet alles ein: die Python-Umgebung, **PlatformIO** (übersetzt und lädt das Programm auf den Arduino; seit 2.92 der Standard) und ein Symbol zum Starten. Die **ARDUINO IDE 1.8.19** (nicht 2.x) ist nur noch optional (Option `/ide` bzw. `--with-arduino-ide`, siehe 2.2). Das Fenster zeigt den Fortschritt; es dauert einige Minuten.
4. **Starten:** Symbol **„webMobaLedLib“** (Windows: „webMobaLedLib“) doppelklicken. Nach 20 bis 60 Sekunden öffnet sich der Browser mit dem Programm.

System	Komplettinstallation
Windows 10/11	Ordner `mobaledlib\python\install` öffnen, oben in die Adressleiste des Explorers `cmd` tippen und Enter drücken. Dort eingeben: `install_windows.bat`. Windows fragt nach der Erlaubnis. Wird dabei Python neu installiert, **den PC neu starten** und den Befehl noch einmal eingeben.
Linux (Mint, Debian, Ubuntu)	Terminal: `cd ~/mobaledlib/python/install` und dann `bash install_linux.sh`. Danach einmal ab- und wieder anmelden.
macOS (ab 10.13, wenig getestet, siehe Kapitel 4)	Terminal: `cd ~/mobaledlib/python/install` und dann `bash install_mac.command`.
Raspberry Pi	siehe unten

Die Befehle zum Kopieren (jeweils eine Zeile, im Terminal bzw. in der Eingabeaufforderung im Ordner `install`):

```
# Windows (Eingabeaufforderung im Ordner mobaledlib\python\install)
install_windows.bat

# Linux (Mint, Debian, Ubuntu)
cd ~/mobaledlib/python/install
bash install_linux.sh

# macOS
cd ~/mobaledlib/python/install
bash install_mac.command
```

Tipp zum Eingeben: Die Dateinamen enthalten einen Unterstrich (`_`). Beim Kopieren aus einem PDF kann es passieren, dass der Befehl im Terminal auf **zwei Zeilen** erscheint oder der Unterstrich fehlt, dann meldet das Terminal nur einen Fehler. In dem Fall den Namen nicht kopieren, sondern **vorne eintippen und mit der Tab-Taste ergänzen lassen**: `bash install_m` tippen (Linux: `bash install_l`, Pi: `sudo bash install_p`), dann die **Tab-Taste** drücken. Das Terminal schreibt den Dateinamen zu Ende, danach Enter drücken.

Raspberry Pi: Besonderheiten und einfachster Weg

Der Pi läuft als **Dienst im Dauerbetrieb**: Das Programm startet beim Hochfahren und ist von jedem Rechner im Netz bedienbar (auch ohne Bildschirm am Pi).

1. Mit dem *Raspberry Pi Imager* das neue Betriebssystem **Raspberry Pi OS „Trixie“, 64-Bit, mit Desktop** auf die SD-Karte schreiben. Im Imager Benutzername, Passwort, WLAN und „SSH aktivieren“ einstellen. Pi starten.

2. Auf dem Pi im Browser die Datei `webmobaledlib.zip` herunterladen (derselbe Link wie oben), im Dateimanager einen **neuen Ordner** **`mobaledlib`** anlegen, die ZIP-Datei hineinkopieren und **„Hier entpacken“**.

3. Terminal öffnen und die Komplettinstallation starten:

   ```
   cd ~/mobaledlib/python/pi
   sudo bash install_pi.sh
   ```

4. Danach: Desktop-Symbol **„webMobaLedLib“** (öffnet das Programm auf dem Pi). Von einem anderen Rechner im Netz: `http://raspberrypi.local:8080` (statt `raspberrypi` der im Imager gewählte Rechnername), Benutzer `mobaledlib`, Passwort `mobaledlib` (im Programm unter „Programm Einstellungen“ änderbar). Der Pi wird über das

Menü **Datei → Raspberry Pi herunterfahren** ausgeschaltet, nicht durch Ziehen des Steckers.

Mehr dazu, Optionen und die Fehlersuche stehen in den folgenden Kapiteln.

1. Welches System, welches Skript?

System	Kapitel	Installationsskript (im Projektordner)	Läuft als	Starten
Windows 10 / 11	3	`install\install_windows.bat`	Programm im Konsolenfenster	Verknüpfung „webMobaLedLib“ auf dem Desktop
macOS 10.13 oder neuer	4	`install/install_mac.command`	App `webMobaLedLib.app`	Programm auf dem Schreibtisch
Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch …)	5	`install/install_linux.sh`	Programm im Benutzerkonto	Symbol im Anwendungsmenü und auf dem Schreibtisch
Raspberry Pi (Dauerbetrieb, auch ohne Bildschirm)	6	`pi/install_pi.sh`	Systemdienst, startet beim Hochfahren	Desktop-Symbol, oder Browser von einem anderen Rechner

Gemeinsam ist allen: Der Ordner **`install`** (und für Linux und Pi der Ordner **`pi`**) liegt **im Projektordner, neben `webmobaledlib.py`**. Die Skripte bestimmen den Projektordner selbst als den Ordner über ihrem eigenen. Es muss kein Pfad eingetragen werden.

2. Gemeinsame Grundlagen

2.1 Dateien und Ordnerstruktur

Der **gesamte Projektordner `python`** (mit `pyMobaLedLib.py`, `webmobaledlib.py` und allen anderen Programmdateien) liegt auf dem Rechner, auf dem das Programm läuft. Er enthält die Unterordner `install` (Skripte für Windows, macOS und Linux) und `pi` (Skripte für den Raspberry Pi; das Skript für die Bibliotheken wird auch unter Linux benutzt). Nicht nötig: `__pycache__`. (Konfiguration, Sicherungen und Protokolle liegen nicht im Projektordner, siehe 2.6.)

Das Programm wird mit **`webmobaledlib.py`** gestartet (das erledigen die Starter-Symbole). Im Projektordner liegen außerdem die Programmteile der Web-Oberfläche in den Ordnern **`headless`** und **`_web`**, die angepassten Originaldateien in `ExcelAPI` und `mlpyproggen` sowie die Skripte in `install` und `pi`.

Beispiele für die Lage des Projektordners: `D:\webMobaLedLib\python` (Windows), `~/mobaledlib/python` (Linux, Pi). Die Skripte legen im Projektordner die Python-Umgebung `venv_web` bzw. `venv` an.

2.2 Die ARDUINO IDE: Version 1.8.19, nicht 2.x

Das Programm benutzt die **ARDUINO IDE 1.8.19** zum Kompilieren und Hochladen. Die neue **IDE 2.x wird nicht unterstützt.** Sie steht auf der Startseite von arduino.cc und wird deshalb oft aus Versehen installiert. Eine vorhandene IDE 2.x stört nicht, es wird zusätzlich die Version 1.8.19 gebraucht. Statt der ARDUINO IDE kann auch **PlatformIO** benutzt werden (Abschnitt 2.16).

Am einfachsten installiert sie das Installationsskript mit der passenden Option:

System	Option	Ziel
Windows	`install_windows.bat /ide`	`C:\Program Files (x86)\Arduino` (Installer von arduino.cc, Rückfrage der Benutzerkontensteuerung)
macOS	`bash install_mac.command --with-arduino-ide`	`/Applications/Arduino.app` (ohne Schreibrecht `~/Applications`)
Linux-Desktop	`bash install_linux.sh --with-arduino-ide`	`~/arduino-1.8.19` (ohne root)
Raspberry Pi	`sudo bash install_pi.sh --with-arduino-ide`	`/opt/arduino-1.8.19` (Version für 64-Bit-ARM)

Die Option prüft zuerst, ob die IDE 1.8 schon vorhanden ist, und lädt dann nichts. Danach werden auch die Bibliotheken installiert (Abschnitt 2.3).

Von Hand: Auf arduino.cc unter *Software* im Abschnitt **„Legacy IDE (1.8.x)“** die Version für das System laden (Windows Installer, macOS, Linux 64 bit bzw. ARM 64 bit).

Wie das Programm die IDE findet: Es braucht die **Startdatei der IDE 1.8.x**, unter Linux und macOS die Datei

`arduino`, unter Windows `arduino.exe`. Die IDE 2.x und die Version 1.0.5 aus der Paketverwaltung (`apt install arduino`) funktionieren nicht.

- **Windows:** Das Programm sucht selbst in `C:\Program Files (x86)\Arduino` und `C:\Program Files\Arduino`.
- **macOS:** in `/Applications/Arduino.app` und `~/Applications/Arduino.app`.
- **Linux und Raspberry Pi:** in dieser Reihenfolge: ein gültiger Pfad in „ARDUINO Einstellungen“, die Umgebungsvariable `MLL_ARDUINO_EXE`, `/opt/arduino-1.8*/arduino`, `/opt/arduino*/arduino`, `/usr/local/arduino*/arduino`, `~/arduino-1.8*/arduino`, `~/arduino*/arduino`, **eine Ebene tiefer** in `~/*/arduino-1.8*/arduino` (zum Beispiel `~/Downloads/arduino-1.8.19/arduino`) und zuletzt der Suchpfad (`arduino` im Terminal). Von mehreren Treffern gilt der mit der höchsten Nummer. Die Treffer stehen im Protokoll. Wird nichts gefunden, steht dort „ARDUINO IDE nicht gefunden“ mit allen geprüften Orten. **Ist PlatformIO installiert (Standard seit 2.92), gibt es diese Warnung nicht** (nur eine Info im Protokoll), ebenso keine Meldung „Keine ARDUINO IDE gefunden“ am Ende der Installationsskripte und im Test `pi_check.py` (der meldet dann die Entwicklungsumgebung PlatformIO). Ein Misserfolg wird nur 10 Sekunden gemerkt: Eine danach installierte IDE wird ohne Neustart gefunden.

Am sichersten: Die IDE 1.8.19 nach `~/arduino-1.8.19` entpacken (das macht `install_linux.sh --with-arduino-ide`). Dann findet das Programm sie ohne Einstellung.

Pfad selbst eintragen („ARDUINO Einstellungen“): Die IDE kann irgendwo liegen, dann trägt man den **vollständigen Pfad der Startdatei** ein, zum Beispiel `/home/name/Programme/arduino-1.8.19/arduino` (nicht den Ordner). Ein Dateiauswahlfenster gibt es dafür **nicht**: Die Oberfläche läuft im Browser, und der Browser sieht nur die Dateien seines eigenen Rechners, der Pfad muss aber auf dem Rechner stehen, auf dem das **Programm** läuft (beim Raspberry Pi also auf dem Pi, nicht auf dem PC). Dafür helfen drei Dinge:

- **Prüfung:** Unter dem Feld steht, ob die Datei vorhanden ist, welche Version sie hat und ob der Eintrag falsch ist. Bleibt das Feld leer, steht dort, was die automatische Suche gefunden hat.
- **Schaltfläche „IDE suchen“:** Sie durchsucht bis zu 12 Sekunden lang die üblichen Orte und den Home-Ordner (bis vier Ebenen tief, ohne versteckte Ordner), außerdem `/opt`, `/usr/local` und `/usr/share`. Gefunden wird, was wie eine IDE 1.x aussieht (neben der Datei `arduino` liegen die Ordner `hardware` und `lib`). Mit einem Klick auf einen Treffer wird der Pfad eingetragen, danach die Einstellungen speichern. Die IDE 2.x wird nicht gefunden (sie wird nicht unterstützt).
- **Von Hand im Terminal:** `which arduino` oder `find ~ -maxdepth 5 -type f -name arduino -perm -u+x` zeigt den Pfad. Er lässt sich markieren und einfügen. Im Dateimanager hilft auf vielen Systemen Rechtsklick, **Eigenschaften** oder **Pfad kopieren**.

Der Haken „Individuellen Pfad … verwenden“ ist nur unter Windows nötig. Auf Linux und macOS gilt der eingetragene Pfad immer, wenn die Datei existiert.

2.3 Bibliotheken, Boards und die Tabelle „Libraries“

Zum Kompilieren braucht das Programm die Bibliothek **MobaLedLib** im Sketchbook-Ordner der IDE (Windows `Dokumente\Arduino\libraries`, macOS `~/Documents/Arduino/libraries`, Linux und Pi `~/Arduino/libraries`) und weitere Bibliotheken in bestimmten Versionen, zum Beispiel FastLED. Das Programm prüft die Bibliotheken und Boards beim Start („Überprüfe Bibliotheken und Boards“) und installiert fehlende in der geforderten Version.

- **MobaLedLib** installiert das Installationsskript mit den Optionen aus 2.2, nachträglich mit `/libs` (Windows), `--with-arduino-libs` (macOS und Linux) bzw. `bash pi/install_arduino_libs.sh` (Pi). Oder in der IDE: *Sketch → Bibliothek einbinden → Bibliotheken verwalten …*, „MobaLedLib“ suchen, *Installieren*.
- **Warum zuerst FastLED:** Die IDE zieht fehlende Abhängigkeiten von MobaLedLib immer in der **neuesten** Version nach, das Programm verlangt aber bestimmte Versionen (zum Beispiel FastLED 3.9.12). Die Skripte installieren deshalb zuerst FastLED in der geforderten Version und danach MobaLedLib. Die Voreinstellung ist 3.9.12, änderbar mit `MLL_FASTLED_VERSION=x.y.z` (Pi und Linux: `MLL_FASTLED_VERSION=x.y.z bash install_arduino_libs.sh`).
- **Kontrolle in der Tabelle „Libraries“:** Sie zeigt die installierten Bibliotheken mit ihrer Version und die vom Programm geforderten Versionen. So öffnest du sie: Button **„Optionen“** in der Symbolleiste → Reiter **„Update“** → Button **„Status der Bibliotheken“**. Die Tabelle „Libraries“ wird dann angezeigt und erscheint wie jede andere Tabelle als Tabellenreiter am unteren Rand. (Die Einstellung „Programmgenerator versteckte Tabellen anzeigen“ in „Programm Einstellungen“ blendet alle ausgeblendeten Tabellen dauerhaft ein; für die Bibliotheken ist sie nicht nötig.) Über der Tabelle stehen die Knöpfe:
  1. **Check actual versions** liest die installierten Versionen neu ein und setzt in der Spalte „Select“ einen **Haken (✓)** bei jeder fehlenden oder falschen Bibliothek (zum Beispiel FastLED 3.10.x statt 3.9.12).
  2. **Install selected** installiert alle Zeilen mit Haken in der geforderten Version neu. Das Programm fragt vorher nach und zeigt den Verlauf im Fenster „Installation der Boards und Bibliotheken“.
  3. **Delete selected** löscht die Bibliotheken bzw. Boards der Zeilen mit Haken.

  Den Haken einer Zeile setzt oder entfernst du mit einem **Klick in die Zelle der Spalte „Select“**.
- **Boards** (zum Beispiel Raspberry Pi Pico) installiert das Programm über dieselbe Tabelle „Libraries“. Boards sind groß (mehrere hundert MB) und brauchen Zeit.
- **ESP32:** Das Programm verlangt das ESP32-Paket in der Version 1.0.4. Auf Windows (Intel/AMD) und Intel-Macs funktioniert das. Auf dem Raspberry Pi und anderen 64-Bit-ARM-Rechnern (auch Macs mit Apple-Chip, nicht geprüft)

fehlen für diese Version die Werkzeuge, siehe Kapitel 10. Das Programm wählt das ESP32-Paket auf Linux mit 64-Bit-ARM deshalb **nicht mehr automatisch** aus (ab Version 1.90).

- **Fehlende Bibliotheken beim Start:** Das Programm prüft beim Start und **fragt im Browser** (sobald er verbunden ist), ob fehlende Bibliotheken und Boards installiert werden sollen. Meldungen, die beim Start noch niemand sehen konnte, erscheinen in einem Dialog „Meldungen beim Start des Programms“. Mit „Nein“ geht es ohne Fehlermeldung weiter. Gelingt etwas nicht, wiederholt das Programm einmal nur, **was noch fehlt** (nicht noch einmal alles). Bringt die Wiederholung nichts Neues, endet das Update mit einer Meldung und dem Hinweis auf das Protokoll, ohne Endlosschleife (Hilfe → Logfile anzeigen). Die Platinen sind groß: Internet und Platz auf dem Datenträger prüfen.
- **ESP32 und der Befehl `python`:** Das ESP32-Paket 1.0.4 ruft `python` auf. Neuere macOS-Versionen (ab 12.3) und Debian-Systeme (auch das Raspberry Pi OS) haben nur `python3`. Das Programm legt deshalb beim Start einen Hilfsbefehl `python` im Datenordner an (`bin/python`, ruft den Python des Programms auf), wenn es keinen gibt (ab Version 1.89). Ohne ihn endet das Prüfen und Hochladen mit `exec: "python": executable file not found in $PATH`. Nach dem Einspielen das Programm neu starten.

2.4 Starten, Beenden, Speichern

- Die Oberfläche öffnet sich im Browser unter `http://localhost:8080`. Der Start dauert 20 bis 60 Sekunden. Ist sie schon gestartet, öffnet ein weiterer Doppelklick auf das Symbol nur den Browser.
- **Beenden:** Knopf **„Beenden“** in der Oberfläche (oben rechts). Unter Windows schließt sich dabei das Konsolenfenster. Auf dem Pi startet der Dienst danach von selbst neu.
- Es sollte immer nur **ein Browser** gleichzeitig mit der Oberfläche arbeiten.
- **Kein Tk nötig:** Ab Version 1.33 benutzt das Programm auf **allen** Systemen einen Ersatz für Tk (`no_tk`). Es braucht weder `tkinter` (`python3-tk`) noch ein Fenster auf dem Bildschirm des Servers. Zum Vergleich bei Fehlern startet `MLL_TK_ROOT=1` mit einem unsichtbaren echten Tk-Fenster (nur wenn `tkinter` und ein Bildschirm vorhanden sind, sonst gilt wieder der Ersatz). Das Protokoll nennt die benutzte Art (`Tk: Ersatz no_tk …`).
- **Speichern:** Das Programm speichert beim Beenden („Speichern und beenden“) und zwischendurch automatisch in die Datei `.autosave.json` im Datenordner (2.6). Dateien aus dem Menü „Datei“ werden im Browser heruntergeladen bzw. hochgeladen. Eine PGF-Datei kannst du auch einfach auf die Tabelle ziehen.
- **Meldungen** stehen in der Datei `webmobaledlib.log` im Datenordner (2.6, Einstellungen siehe 2.7), außerdem `webmobaledlib-console.log` bei macOS und Linux und `journalctl` auf dem Pi. Welche Programmversion läuft, steht unter dem Titel der Oberfläche (zum Beispiel „webMobaLedLib 1.16“) und in den Köpfen der Dateien („Datei-Version“).

Welcher Browser? Das Programm ist **nicht** auf einen Browser festgelegt. Es öffnet die Adresse im **Standardbrowser des Systems** und prüft, ob das geklappt hat:

System	Reihenfolge
alle	zuerst die Umgebungsvariable `MLL_BROWSER` (Programm, auch mit Argumenten, zum Beispiel `MLL_BROWSER="brave-browser --new-window"`), dann `BROWSER`
Windows	die Standard-App für Webadressen
macOS	`open`
Linux	`xdg-open` (Standardbrowser), `gio open`, `sensible-browser`, `x-www-browser`, danach bekannte Browser: Google Chrome, Chromium, Brave, Edge, Vivaldi, Opera, Firefox, Epiphany, Falkon, zuletzt das Modul `webbrowser` von Python

Endet ein Weg mit einem Fehler (zum Beispiel `xdg-open`, weil kein Standardbrowser eingetragen ist), geht es mit dem nächsten weiter. Der benutzte Weg steht im Protokoll (`Browser geöffnet (xdg-open): http://localhost:8080`). Findet sich gar kein Browser, nennt das Programm die Adresse auf dem Bildschirm und im Protokoll, dann von Hand eingeben. Unter Linux zeigt `xdg-settings get default-web-browser`, welcher Browser als Standard eingetragen ist, und `xdg-settings set default-web-browser google-chrome.desktop` (oder `brave-browser.desktop`) stellt ihn ein.

2.5 Das Programmsymbol

Das Symbol ist das des originalen ProgrammGenerators (Ring aus bunten LEDs, Tabelle, Stift) mit einer kleinen blauen Weltkugel unten rechts. Dieselbe Datei steht als Symbol im Browserreiter. `install/make_icons.py` erzeugt daraus `mobaledlib.png` (Verknüpfungen, Raspberry Pi), `favicon.png`, `mobaledlib.ico` (Windows) und `mobaledlib.icns` (macOS). Ohne Weltkugel: `python make_icons.py --plain`. Die fertigen Dateien liegen bei; die Installer erzeugen sie nur neu, wenn sie fehlen. `make_icons.py` braucht Pillow (`pip install Pillow`), die Web-Oberfläche selbst nicht. Die Vorlage `mobaledlib_source.png` ist nur 128 × 128 Pixel groß und wirkt vergrößert etwas weich; wer sie höher aufgelöst hat, ersetzt die Datei und startet `make_icons.py` noch einmal.

2.6 Wo liegen meine Daten? (Datenordner)

Alles, was das Programm **selbst erzeugt**, liegt **außerhalb des Projektordners** in einem Datenordner des Benutzers. Dadurch bleiben deine Daten erhalten, wenn bei einem Update der Ordner `python` ersetzt oder überschrieben wird.

System	Datenordner
Windows	`%APPDATA%\webMobaLedLib` (das ist `C:\Users\<Name>\AppData\Roaming\webMobaLedLib`)
macOS	`~/Library/Application Support/webMobaLedLib`
Linux, Raspberry Pi	`~/.local/share/webMobaLedLib` (beim Pi für den Benutzer, unter dem der Dienst läuft)

Bis Version 1.30 hieß der Ordner `pyMobaLedLib`. Das Programm benennt ihn beim ersten Start mit der neuen Version selbst um (Konfiguration, Sicherungen, Passwort und Protokolle bleiben erhalten; ist das Umbenennen nicht möglich, wird er kopiert und der alte Ordner darf später gelöscht werden).

Einen anderen Ort legt die Umgebungsvariable **`MLL_DATA_DIR`** fest (zum Beispiel `D:\MobaLedDaten`). Der Ordner wird bei Bedarf angelegt. Wo er ist, zeigen das Fenster beim Start, das Protokoll, der Menüpunkt **Hilfe → Über …** und der Befehl `python userdata.py` im Projektordner.

Was dort liegt: die Konfigurationsdateien (Einstellungen, Farbtest, Palette), die Sicherung der Tabellen (wird beim Beenden mit „Speichern“ geschrieben und beim Start geladen), die Zwischensicherung `.autosave.json`, die Passwort-Datei `web_password.json` (nur als Hash) und die Protokolle `webmobaledlib.log` und `webmobaledlib-console.log`, das Absturzprotokoll `webmobaledlib-crash.log` sowie der Unterordner `nicegui` (interner Speicher der Oberfläche).

Übernahme alter Dateien: Frühe Versionen der Web-Oberfläche (bis 1.22) haben Konfiguration, Tabellensicherung und Passwort-Datei außerhalb des Datenordners abgelegt. Das Programm kopiert sie beim ersten Start einmalig in den Datenordner (die alten Dateien dürfen danach gelöscht werden). **Dateien der Tk-Version werden nicht übernommen:** Die Web-Version teilt keine Dateien mit ihr. Tabellen aus der Tk-Version lassen sich über eine gespeicherte PGF-Datei laden (auf die Tabelle ziehen).

Was im Projektordner bleibt: das Programm, die Vorlagen und Definitionen und die Python-Umgebung (`venv_web` bzw. `venv`). Wird bei einem Update der ganze Ordner gelöscht und neu entpackt, ist auch die Python-Umgebung weg: dann das Installationsskript noch einmal ausführen.

Sichern und Umziehen: den Datenordner kopieren. **Prüfen, dass nichts im Projektordner landet:** `python install/check_project_writes.py snapshot` vor dem Start, nach einer Sitzung `python install/check_project_writes.py compare` (zeigt neue, geänderte und gelöschte Dateien im Projektordner, mit `--parent` auch eine Ebene darüber).

2.7 Protokoll (Log)

Das Programm schreibt ein Protokoll mit dem Standard-Logger von Python. Es enthält auch die Meldungen der Programmmodule. Die Datei **`webmobaledlib.log`** liegt im Datenordner (2.6). Ohne Angabe wird auf der Stufe **DEBUG** protokolliert, wie im Original. Die Datei dreht sich bei 2 MB (die älteren heißen `webmobaledlib.log.1` bis `.5`), und auf der Konsole (beim Pi im Journal des Dienstes) erscheinen die Meldungen ab INFO. Jede Sitzung beginnt mit einem Kopf aus Python-Version, System, Parametern, Ordnern, Stufe, Dateiname, den Versionen der wichtigsten Pakete und der Umgebung (Sitzungsart, Display, Tk). Außerdem steht darin, wann sich ein Browser verbindet und trennt (`Browser verbunden` / `Browser getrennt`), wie lange der Aufbau der Seite gedauert hat, und eine Warnung, wenn der Server ungewöhnlich lange blockiert war. Bricht der Python-Prozess ab (zum Beispiel durch einen Fehler in einer Systembibliothek), steht der Grund in `webmobaledlib-crash.log`. Auch die Ausgabe der Fenster **„Kompilieren und Hochladen“** und **„Installation der Boards und Bibliotheken“** steht im Protokoll (Logger `webmobaledlib.arduino`, ab Stufe INFO; Zeilen mit Fehlermarkierung als WARNING, jeder Lauf mit einer Kopfzeile). So lässt sich ein Lauf später nachlesen. In der Oberfläche zeigt **Hilfe → Logfile anzeigen** die letzten Zeilen (am Ende steht das Neueste) mit Pfad, Größe und Stufe der Datei. Mit **Aktualisieren** liest es neu, mit **Ganze Datei herunterladen** bekommst du die komplette Datei, zum Beispiel für eine Fehlermeldung.

Kommandozeile	Umgebungsvariable	Wirkung
`--loglevel DEBUG`, `INFO`, `WARNING`, `ERROR` oder `CRITICAL`	`MLL_LOGLEVEL`	Stufe der Protokolldatei (Standard `DEBUG`)
`--logfile NAME`	`MLL_LOGFILE`	Dateiname. Ohne Pfad liegt die Datei im Datenordner, mit absolutem Pfad dort, `stdout` schreibt keine Datei
–	`MLL_DEBUG_TIMING=1`	schreibt die Zeit für Aufbau und Aktualisierung der Tabelle und der Testtasten ins Protokoll (Zeilen mit `TIMING:`), hilft bei der Suche nach langsamen Stellen
–	`MLL_DEBUG_SHAPES=1`	schreibt zu jeder Aktualisierung der Tabelle die Zahl der Symbole und Testtasten und welche Zeilen sie tragen ins Protokoll (Zeilen mit `SHAPES:`), für die Fehlersuche bei fehlenden Symbolen oder Testtasten
–	`MLL_GRID_UPDATE=full`	Die Tabelle wird bei jeder Änderung

		komplett neu gesendet (wie bis Version 1.48). Zum Prüfen, ob ein Anzeigefehler an der gezielten Aktualisierung liegt; langsamer

Beispiele: Windows `install\start_windows.bat --loglevel INFO`, Linux und macOS `install/start_linux.sh --loglevel INFO`, Raspberry Pi die Zeile `MLL_LOGLEVEL=INFO` in `/etc/mobaledlib-web.env` und danach `sudo systemctl restart mobaledlib-web`. Die sehr gesprächigen Bibliotheken (Webserver, Websocket) bleiben auch bei DEBUG gedämpft. Wer sie sehen will, setzt `MLL_LOG_LIBS=1`. Bei einer Fehlermeldung bitte die Datei `webmobaledlib.log` schicken (und `webmobaledlib.log.1`, falls vorhanden).

2.8 Kleine Bildschirme: kompaktes Layout

Auf kleinen Bildschirmen (zum Beispiel einem Laptop mit 1366 × 768 Punkten, auf dem der Browser nur rund 600 Punkte Höhe hat) bleibt für die Tabelle wenig Platz, weil Kopfzeile, Menü, Reiter und die großen Symbole der Funktionsknöpfe mehr als die Hälfte belegen. Das **kompakte Layout** schafft Platz:

- Menü (Datei, Tabelle, ARDUINO, Hilfe), Titel, Statusmeldung und der Knopf **Beenden** stehen in **einer** Zeile. Der Titel ist kleiner, der Knopf Beenden ohne Symbol, die Versionsangabe und die Zeit der Zwischensicherung sind ausgeblendet (die Version steht unter Hilfe → Über …).
- Die Reiter sind schmaler.
- Die **Symbole der Funktionsknöpfe werden durch ihren Text ersetzt:** eine schmale Reihe kleiner Textknöpfe statt großer Symbole.
- Leerzeilen, Ränder und Abstände entfallen.
- In den **Makrodialogen** (Makroauswahl, Parameterformulare, Haus-Dialog) sind die **Beschreibungstexte auf eine Zeile eingeklappt**. Ein Klick auf den Text (Pfeil ▸) klappt ihn auf, ein weiterer Klick wieder zu.
- Die Zeile „Zeile x letzte benutzte Zeile von y Zeilen“ steht unten rechts neben den Blattreitern statt in einer eigenen Zeile.

Einschalten: Reiter **Programm Einstellungen**, Karte **Darstellung auf kleinen Bildschirmen**, Feld **Kompaktes Layout**:

Stufe	Wirkung
Aus (Standard)	normales Layout
Ein	immer kompakt
Automatisch	kompakt, wenn das Browserfenster niedriger als 800 Pixel ist, und wieder normal, wenn es höher wird

Die Auswahl wirkt sofort, gesichert wird sie mit **Speichern**. Die Einstellung gilt für das Programm auf diesem Rechner (sie steht in der Konfiguration im Datenordner). Ohne gespeicherten Wert gilt die Umgebungsvariable `MLL_LAYOUT` (`normal`, `compact` oder `auto`). Zusätzlich schaffen **Vollbild (F11)** und der **Browser-Zoom** (Strg und Minus) Platz.

2.9 Bedienung der Tabelle: Makroauswahl, Bearbeiten, Kontextmenü

- **Ein Klick auf das Symbol** (Spalte „Icon“) öffnet die Makroauswahl.
- **Zellen von Hand bearbeiten: F2** oder **Enter** auf der markierten Zelle, ein **Doppelklick** (siehe unten) oder im Kontextmenü (Rechtsklick auf die Zelle) **Zelle bearbeiten**. Das ist zum Beispiel in der Spalte „Beleuchtung, Sound, oder andere Effekte“ oft schneller als der Dialog, um einen Parameter zu ändern.
- **Doppelklick auf Name und Beleuchtung:** Wie in der Excel-Version kann ein Doppelklick auf diese beiden Spalten die **Makroauswahl** öffnen. Das ist **standardmäßig aus** (ein Doppelklick bearbeitet die Zelle). Einschalten: Reiter **Programm Einstellungen**, Karte **Bedienung der Tabelle**, Haken **„Doppelklick auf Name und Beleuchtung öffnet die Makroauswahl“**, dann **Speichern**. Er gilt nur in Datenzeilen der Datenblätter, die Kopfzeilen bleiben unverändert.
- **Kontextmenü** (Rechtsklick auf eine Zelle): Zelle bearbeiten (F2), Kopieren (Strg+C), Ausschneiden (Strg+X), Einfügen (Strg+V), Als Zeilen einfügen, Löschen (Entf), Farbe setzen, Farbe entfernen. Sind mehrere Zeilen markiert und man klickt in eine davon, wirken Kopieren, Ausschneiden und Löschen auf die markierten Zeilen.
- **Spalten hinter Q:** Datenblätter zeigen nur die Spalten bis Q. Der Knopf **„Alle Spalten“** (Werkzeugleiste, hinter „Zeilen färben“; **nur zum Testen** und deshalb **nur sichtbar, wenn in den Programm Einstellungen „Programmgenerator versteckte Tabellen anzeigen“ eingeschaltet ist**; Ausschalten blendet den Knopf wieder aus und zeigt wieder nur die Spalten bis Q) blendet auch die Spalten dahinter ein, zum Beispiel die LED-Zahlen der Kanäle 1 bis 9 in Zeile 1. **„Spalten bis Q“** schaltet zurück. Die Einstellung gilt bis zum Beenden des Programms.
- **Menü Tabelle → Tabelle neu berechnen:** rechnet **alle** Tabellenblätter neu (LED-Zahlen je Kanal und Start-LED-Nummern, auch die der eingebundenen Blätter) und aktualisiert danach für das aktive Blatt die Symbole und die Testtasten. Nötig ist das zum Beispiel, wenn gespeicherte Blätter aus einer früheren Version stammen und falsche LED-Zahlen in Zeile 1 stehen. Vor dem Erzeugen der Datei `LEDs_AutoProg.h` („Setup übertragen“) werden ohnehin alle Blätter neu gerechnet.
- **Menü Tabelle → Tabelle reparieren (Symbole und Testtasten neu aufbauen) …:** Reparatur, wenn **Symbole**

oder Testtasten fehlen oder an der falschen Zeile stehen (zum Beispiel nach einer fehlerhaft geladenen Tabelle). Nach einer Rückfrage werden Symbole und Testtasten **aller Datenblätter** gelöscht und aus dem Tabelleninhalt neu angelegt (auch die Namen aus dem Makro neu gesetzt); das kann bei großen Tabellen einige Sekunden dauern. „Tabelle neu berechnen“ legt fehlende Symbole und Testtasten nicht an. (Die früheren Einträge „Symbole aktualisieren“ und „Testtasten neu aufbauen“ gibt es nicht mehr: Beides macht „Tabelle neu berechnen“ für das aktive Blatt.)

2.9a Blattreiter: Farbe, Umbenennen, Duplizieren, Verschieben (2.63)

Die Blattreiter am unteren Rand der Tabelle haben ein **Kontextmenü** (rechte Maustaste auf den Reiter; auf dem Tablet langes Drücken):

- **Reiterfarbe:** Untermenü mit **20 Farben** (die zehn Standardfarben von Excel und zehn weitere), **Keine Farbe** und **Weitere Farben …** (beliebige Farbe). Die **Schrift** wird automatisch **weiß oder schwarz**, je nachdem, was auf der gewählten Farbe besser lesbar ist (mindestens 3 : 1 Kontrast: weiß auf Rot, Blau und Violett, schwarz auf Gelb, Grün und Hellblau). Der **gewählte** Reiter wird heller und hat einen **Verlauf nach Weiß** (wie in Excel). Beim Start haben die Reiter **keine Farbe**; die Farbe wird **je Blatt mit der Arbeitsmappe gesichert** (automatische Sicherung und Datei).
- **Umbenennen …:** fragt nach dem neuen Namen. Das geht nur bei **eigenen Blättern (Kopien)**: Die Blätter des Programms (**DCC, Selectrix, CAN, LNet** und die Systemblätter wie Languages oder Lib_Macros) lassen sich **nicht umbenennen**, weil das Programm sie am Namen sucht; der Eintrag ist dort ausgegraut. Der Name darf höchstens **31 Zeichen** haben und keines der Zeichen `[ ] : * ? / \` enthalten, nicht mit einem Apostroph beginnen oder enden, nicht schon vorkommen (Groß- und Kleinschreibung zählt nicht) und nicht der Name eines Blatts des Programms sein; sonst zeigt das Fenster den Grund.
- **Duplizieren …:** legt eine **Kopie eines Datenblatts** an (DCC, Selectrix, CAN, LNet oder eine Kopie davon; der Typ steht im Blatt selbst in der ersten Zeile). Das Fenster schlägt einen freien Namen vor (zum Beispiel `DCC_1`). Die Kopie hat die **Zellen**, die **Symbole und Testtasten** und die **Reiterfarbe** des Originals, steht **am Ende** und wird sofort angezeigt. Sie wird wie jedes andere Datenblatt **mit der Arbeitsmappe gesichert** und beim **Erzeugen des Programms** wie das Original mit verarbeitet (in der Reihenfolge der Reiter).
- **Nach links verschieben**, **Nach rechts verschieben:** der Reiter wandert um einen sichtbaren Reiter (zum Beispiel auf dem Tablet, wo es keine Alt-Taste gibt).

Reiter ziehen: Mit gedrückter **Alt-Taste** den Reiter mit der **linken Maustaste** anfassen und an eine andere Stelle ziehen; eine blaue Marke zeigt, wo er eingefügt wird (Loslassen = einfügen; **Esc** bricht ab). Ohne Alt-Taste wählt ein Klick den Reiter wie bisher. Die **Reihenfolge** kann beliebig sein und wird **mit der Arbeitsmappe gesichert**; da die Systemblätter normalerweise nicht sichtbar sind, spielt ihre Stelle keine Rolle (sie behalten ihre Stelle zu den übrigen).

2.9b Mehrere Zeilen markieren, kopieren und ausblenden (2.67)

Die Knöpfe der Symbolleiste **Zeilen duplizieren** (früher „Kopiere Zeilen“) und **Aus- oder Einblenden** wirken auf **alle markierten Zeilen**: Zeilen über die Zeilennummern markieren (Klick, mit Umschalt- oder Strg-Taste mehrere) oder einen Bereich von Zellen über mehrere Zeilen aufziehen. **Zeile kopieren** fügt die Kopien **hinter der letzten markierten Zeile** ein, in derselben Reihenfolge wie die Originale (markiert: B und C, danach stehen B, C, B, C hintereinander). Sind die markierten Zeilen nicht zusammenhängend (zum Beispiel B und D), gilt wie in Excel der **Block von der ersten bis zur letzten markierten Zeile** (B, C, D). Zeilen oberhalb des Datenbereichs (Kopfzeilen) werden nicht kopiert. Ab Version 2.67 wirken diese Knöpfe auch bei **einer** markierten Zeile auf **diese** Zeile; vorher wurde beim Kopieren der Inhalt der Zeile **darüber** kopiert und beim Ausblenden die Zeile darüber versteckt.

Farbe wählen: Im Fenster **„Hintergrundfarbe der Zelle“**, im Fenster **„Reiterfarbe“** und bei Farbfeldern in den Formularen geht die Farbwahl (Klick auf die **Pipette**) jetzt **rechts neben dem Knopf** auf und verdeckt die Knöpfe **OK** und **Abbrechen** nicht mehr (vorher musste man erst irgendwo klicken, damit sie verschwand).

Zeilen aktivieren und deaktivieren (2.93): Mit der **Leertaste** schaltet man den **Haken** der Spalte **Aktiv** aller **markierten Zeilen** um (Zeilen über die Zeilennummern markieren oder eine Zelle der Zeile wählen). **Maßgeblich ist die oberste markierte Zeile mit Inhalt:** Hat sie einen Haken, wird er bei allen markierten Zeilen entfernt, sonst bei allen gesetzt (auch wenn die markierten Zeilen vorher verschieden waren). Ganz **leere Zeilen** und **ausgeblendete Zeilen** bleiben unverändert. Dasselbe macht der Eintrag **Zeilen aktivieren/deaktivieren** im **Kontextmenü** (rechte Maustaste), auch auf Tablet und Smartphone ohne Leertaste. Die LED-Nummern werden **einmal** für alle Zeilen neu berechnet. Beim **Bearbeiten einer Zelle**, in Eingabefeldern und bei geöffnetem Dialog bleibt die Leertaste unverändert (Leerzeichen). Auf dem Blatt **Libraries** gibt es die Funktion nicht (dort wirkt der Klick auf die Spalte **Select**). Nützlich, um **ein ganzes Haus** für Wartungsarbeiten oder Erweiterungen vorübergehend abzuschalten (Idee eines Testers).

2.9c Die Knöpfe der Werkzeugleiste (2.68)

Die Knöpfe der Werkzeugleiste des Programm Generators (Dialog Wizard, Setup übertragen, Zeile einfügen, Zeilen löschen, Zeilen verschieben, Zeilen duplizieren, Aus- oder Einblenden, Alle Einblenden, Tabelle löschen, Neue Tabelle, Optionen Werkzeuge, Online Wiki, Zeilen Färben) bestehen aus einem **Symbol ohne Text** und einer **Beschriftung unter dem Symbol in der eingestellten Sprache** (jeder Knopf ist nur so breit wie sein Symbol): Die Texte kommen aus demselben Blatt „Languages“ wie im Excel-Programm (Deutsch, Englisch, Niederländisch, Französisch, Italienisch,

Spanisch, Dänisch); vorher stand der deutsche Text im Bild und blieb in jeder Sprache deutsch. Der Tooltip (Hinweis beim Darüberfahren) ist ebenfalls übersetzt.

Eingedrückt: Beim Klicken erscheint der Knopf **eingedrückt** (innerer Schatten, Symbol und Text einen Pixel tiefer). Er **bleibt eingedrückt, solange der Befehl läuft**, auch wenn der Befehl auf eine Antwort in einem Fenster wartet (zum Beispiel eine Rückfrage oder die Auswahl eines Anschlusses), und kommt danach wieder hoch. Läuft schon ein Vorgang, meldet das Programm das wie bisher. Die Knöpfe haben ein modernes, flaches Aussehen mit Rahmen, Hover (etwas dunkler beim Darüberfahren) und ein dunkles Aussehen im dunklen Modus des Browsers. Die **Größe** folgt der Einstellung „PGIconSize“ in den Programm Einstellungen (Symbol und Schrift). Im **kompakten Layout** (Programm Einstellungen, Darstellung) werden wie bisher nur die Texte in einer Zeile gezeigt.

Symbole in der Spalte Icon (2.77): Die Symbole der Tabelle sind etwas kleiner als die Zeile (18 statt 22 Pixel bei normaler Größe) und senkrecht mittig, damit nichts abgeschnitten wird (vorher fehlte zum Beispiel unten ein Stück der „2“ im Symbol der Servos); die Zeilenhöhe ändert sich nicht und folgt wie bisher der Schriftgröße der Tabelle.

Alle Titel haben zwei Zeilen, damit alle Knöpfe gleich hoch sind (auch in den anderen Sprachen; die Beschriftung ist mindestens zwei Zeilen hoch). Die Titel (ab 2.76): **Dialog Wizard**, **Setup übertragen** (früher „Z. Arduino schicken“), **Zeilen löschen**, **Zeilen verschieben**, **Zeilen duplizieren**, **Tabelle löschen**, **Optionen Werkzeuge**.

Die Symbole liegen im Ordner `images_btn` (Dateinamen wie die alten Bilder `Btn_*.png`, aber ohne Text, etwa 214 × 164 Pixel, durchsichtiger Hintergrund). Fehlt ein Symbol dort, zeigt das Programm das alte Bild aus dem Ordner `images` (mit dem deutschen Text im Bild). Die Knöpfe des Patterngenerators haben noch die alten Bilder.

2.9d Programme für den rechten Arduino (DCC, Selectrix, LNet) mit PlatformIO (2.72)

In den **Optionen** gibt es bei einem Nano die Reiter **DCC ARDUINO**, **SELECTRIX ARDUINO** und **LNET ARDUINO**. Der Knopf **„Prog. Installieren“** schreibt das Programm für den rechten Arduino (die Zentrale, die die Signale von DCC, Selectrix oder LocoNet in Schaltbefehle umsetzt). Mit der **ARDUINO IDE** wird dazu wie bisher die Quelle (`23_A.DCC_Interface.ino`, `23_A.Selectrix_Interface.ino`, `23_A.LNet_Interface.ino` im Ordner `libraries/MobaLedLib/examples`) übersetzt und hochgeladen.

Mit PlatformIO wird nichts übersetzt, sondern die **fertig übersetzte Datei hochgeladen**: Sie heißt wie die Quelldatei mit der Endung **`.hex`** (`23_A.DCC_Interface.hex`, `23_A.Selectrix_Interface.hex`, `23_A.LNet_Interface.hex`) und liegt im Ordner **`libraries/MobaLedLib/firmware`** im Python-Ordner. Das Programm sucht dort zuerst, danach im Ordner `MobaLedLib/firmware` des Python-Ordners; Groß- und Kleinschreibung der Namen spielt keine Rolle. Fehlt die Datei, zeigt das Programm eine Meldung mit dem Namen und den Ordnern, in denen es gesucht hat.

Ablauf: Das Programm prüft wie bisher den Anschluss und erkennt, ob der Nano den alten oder den neuen Bootloader hat (daran hängen Platine und Geschwindigkeit). Dann legt es ein kleines Hilfsprojekt im Ordner für temporäre Dateien des Betriebssystems an (`webmobaledlib_hex_upload`, bleibt stehen) und lässt PlatformIO die Datei hochladen (`pio run -t nobuild -t upload`: es wird **nichts übersetzt**). Ein kleines Vorab-Skript des Hilfsprojekts (`copy_hex.py`) kopiert die Datei dabei in den Build-Ordner von PlatformIO, **nachdem** PlatformIO diesen aufgeräumt hat (PlatformIO leert den Ordner beim ersten Mal und wenn sich das Projekt ändert, zum Beispiel bei einem Wechsel zwischen altem und neuem Bootloader). Der Fortschritt steht wie sonst im Fenster des Arduino-Monitors. Dafür muss PlatformIO die Plattform **atmelavr** haben; sie wird beim ersten Mal aus dem Netz geholt (ein Übersetzen für den Nano hat sie schon geholt).

Nach dem Hochladen merkt sich das Programm, dass der rechte Arduino beschrieben ist („R OK“), und fragt beim nächsten Hochladen des LED-Programms nicht noch einmal danach (vorher wurde es bei jedem LED-Upload erneut angeboten).

2.9e Schneller Bootloader mit PlatformIO (2.74)

Mit dem **schnellen Bootloader** (Optiboot, Version 108.1) wird das Programm bei einer Änderung der Konfiguration deutlich schneller zum **linken** Arduino geschickt. Der Reiter **Bootloader** in den **Optionen** (Platine Nano) hat den Knopf **Schnellen Bootloader**. Er steht ab Version 2.74 auch mit **PlatformIO** zur Verfügung (die Jumper gibt es erst ab der Hardware-Version 1.5 der Hauptplatine).

Ablauf (wie im Excel-Programm): 1. **Hinweis zu den Jumpern** (Text wie im Excel-Programm): Die Jumper unter dem linken Arduino werden gesetzt (der untere Jumper verbindet den linken mit dem mittleren Pin), der linke Arduino wird wieder eingesteckt und der **rechte** Arduino mit dem USB-Anschluss des PC verbunden. **OK** startet, **Abbrechen** beendet. (Das Bild der Jumper aus dem Excel-Programm zeigt die Webversion noch nicht.) 2. Das Programm lädt das Programm **ArduinoISP** auf den **rechten** Arduino. Mit PlatformIO ist das die **fertig übersetzte Datei** **`03.ArduinoISP.hex`** im Ordner `libraries/MobaLedLib/firmware` (wie bei den Programmen für DCC, Selectrix und LNet, Abschnitt 2.9d); es wird nichts übersetzt. 3. Über den ArduinoISP des rechten Arduino wird der **linke** Arduino beschrieben. Dazu benutzt das Programm das **avrdude von PlatformIO** (Paket `tool-avrdude`, es kommt mit der Plattform atmelavr beim ersten Hochladen für einen Nano; hat das Programm es noch nicht, erklärt eine Meldung, was zu tun ist). Zuerst wird die **Signatur** des Mikrocontrollers gelesen (ATmega328P oder ATmega328PB; bei einem anderen Chip oder wenn nichts gelesen werden kann, kommt eine Meldung mit dem Hinweis auf die Jumper). Dann wird der Bootloader aus `libraries/MobaLedLib/extras/ArduinoISP` geschrieben (`optiboot_atmega328_Ver108.1.hex`; Lock-Bits 0x0F, High-Fuse 0xDE). **Bei einem ATmega328PB** fragt das Programm einmal, ob der Bootloader für den ATmega328P geschrieben werden soll

(die Signatur wird die des ATmega328P; das ist die Vorgabe des Excel-Programms; „Nein“ schreibt `optiboot_atmega328pb_16mhz.hex`). Die Ausgabe von avrdude steht im Fenster des Arduino-Monitors. **Jeder Schritt hat dort seine eigene Überschrift** („Schritt 1 von 3: ArduinoISP auf den rechten Arduino laden“, „Schritt 2 von 3: Signatur des linken Arduino lesen“, „Schritt 3 von 3: Bootloader auf den linken Arduino schreiben“); das Lesen der Signatur ist also **kein** zweiter Upload. Beim Hochladen einer fertigen `.hex`-Datei (auch bei DCC, Selectrix und LNet, Abschnitt 2.9d) steht dort „Hochladen gestartet. Die fertig übersetzte Datei wird hochgeladen (es wird nichts übersetzt).“ statt der Meldung zum Kompilieren. 4. Frage **„Noch einen Arduino aktualisieren?“** (Vorgabe Nein): Bei **Ja** den nächsten Arduino in den linken Steckplatz stecken (der rechte darf nicht entfernt werden); Schritt 3 wird wiederholt. Ging das Schreiben schief, fragt das Programm, ob es **noch einmal versucht** werden soll. 5. Am Ende lädt das Programm wieder das **DCC-, LNet- oder Selectrix-Programm** (die `.hex`-Datei) auf den rechten Arduino, und eine Meldung erinnert daran, **die Jumper wieder zu entfernen** (sonst geht nichts mehr) und das USB-Kabel wieder auf den linken Arduino zu stecken.

Makroauswahl (Doppelklick auf eine Zelle mit einem Makro): Die Auswahl öffnet sich mit dem **Makro der Zelle markiert** (gelb) und **scrollt es in den sichtbaren Bereich** der Liste (seit 2.84; vorher sah es bei Makros am Ende der Liste, zum Beispiel unter **PatternConfig** wie ServoAnim, so aus, als sei keines ausgewählt, weil die Markierung unterhalb des sichtbaren Bereichs lag). Das Makro der Zelle wird zuerst über die Spalte „Find Name“ der Makrobibliothek gesucht; **findet das nichts oder ergibt es einen Eintrag, der in der Liste nicht steht, sucht das Programm über den Namen des Makros** (seit 2.85, zum Beispiel `ServoAnim` aus `// #ServoAnim(…)` und schreibt in dem Fall eine Meldung in das Debug-Fenster („Makroauswahl: Suchtext … ergab über ‚Find Name‘ die Zeile …“). **ServoAnim2B** (N_Buttons) steht in der Bibliothek in einer ausgeblendeten Gruppe (`*PatternConfig`) und kann deshalb nicht vorausgewählt werden (wie im Excel-Programm).

2.9f Servo Animation (Makros ServoAnim und ServoAnim2B, 2.78)

Die Makros **ServoAnim** und **ServoAnim2B** (Dialog **Servo Animation**, aus dem Makrobaum oder mit dem Knopf **Dialog Wizard** über die Zeile) öffnen einen Dialog, in dem der Ablauf eines Servos als Kurve bestimmt wird. Er unterstützt den Direkt-Mode-Servo (Servo DM) und die normalen MLL-Servos (Servo1 rot, Servo2 grün, Servo3 blau). Das Ergebnis ist ein Makro `PatternT…` mit einer Kommentarzeile `#ServoAnim(…)` mit den 16 Einstellungen; mit **OK** wird es in die Zeile eingetragen, ein vorhandenes Makro wird beim Öffnen wieder eingelesen und kann geändert werden (bei **Kurventyp Individuell** werden auch die **Punkte** der Kurve aus der Zeile `PatternT…` gelesen, so wie sie in der Tabelle stehen; seit 2.82, vorher war die Kurve beim Wiederöffnen leer).

Einstellungen: Servo-Adresse und Anschluss-Kanal; Servotyp; **Ausführung** (Wiederholen, PingPong, Ein/Aus; bei PingPong entfällt der Ausschaltteil); **EIN-** und **AUS-schalt Parameter** (Startpause, Dauer, Endepause in ms); **Kurventyp** (Linear, Beschleunigung, Individuell) mit Anfangswert (0 bis 255) und Endwert (1 bis 254); „ARDUINO Überblendung AUS“; **Zeitstufen** (Automatisch, Fester Wert mit Stufenabstand, Individuell). Rechts zeigt die **Grafik „Kurvenverlauf“** die berechnete Kurve (waagerecht die Zeit in ms, senkrecht der Wert), die gestrichelte Linie ist das Ende der Einschaltzeit. Jede Änderung zeichnet die Grafik sofort neu; **Update Grafik** rechnet noch einmal und beendet einen laufenden Test.

Fenster (2.80): Das Fenster ist groß (bis 1280 × 860 Pixel, höchstens so groß wie das Browserfenster) und **lässt sich verschieben und in der Größe ändern**: Am **Titel „Servo Animation“** mit der Maus (oder dem Finger) ziehen verschiebt es (ein Stück bleibt immer sichtbar); am **Rand unten rechts** ziehen ändert die Größe (mindestens 460 × 340 Pixel). **Größe und Lage werden im Browser gemerkt** (nach dem Loslassen am Titel oder am Rand) und beim nächsten Öffnen wiederhergestellt; setzt die Seite die Größe zurück (zum Beispiel nach einem Klick auf „Starte Test“), stellt sie das Fenster sofort wieder her. Die **Grafik wächst mit dem Fenster**; in einem schmalen Fenster steht sie **unter** den Einstellungen statt daneben.

Kurventyp Individuell: Die grünen Punkte lassen sich **mit der Maus ziehen** (der Wert zwischen 1 und 254; die Zeit nur bei den Zeitstufen Individuell und nicht beim ersten und letzten Punkt, höchstens bis zu den Nachbarpunkten). Der angeklickte Punkt wird orange; seine Zeit und sein Wert stehen in den Feldern unter der Grafik und lassen sich dort eingeben. **Punkt einfügen** setzt einen neuen Punkt hinter den ausgewählten (in der Mitte zum nächsten), **Punkt löschen** entfernt einen inneren Punkt. Ändert man die Dauer, werden die Zeiten der Punkte mitgestreckt.

Servo ausprobieren: Ist ein ARDUINO verbunden, bewegt sich der Servo, wenn man einen Punkt anklickt oder zieht oder Anfangs- oder Endwert ändert (Meldung `#L …` im Direktmodus). **Starte Test** spielt die Kurve in Schritten von 40 ms immer wieder ab (roter Strich in der Grafik, „Zeit“ und „Wert“ darüber); **Stop Test** hält sie an. Ohne verbundenen ARDUINO meldet das Programm das. **Die Frage, ob der ARDUINO angeschlossen werden soll, kommt nur einmal:** Antwortet man mit „Abbrechen“ (oder ist keiner da), fragt das Programm beim Ziehen und Anklicken der Punkte und beim Ändern von Anfangs- und Endwert **nicht mehr von selbst** (bis zum Neustart des Programms, auch nicht in anderen Servo-Dialogen). Nur der Knopf **Starte Test** fragt wieder, denn dort will man ausdrücklich testen; kommt dabei eine Verbindung zustande, wird wieder normal gesendet. Beim Schließen wird der Direktmodus beendet. **Hilfe** öffnet die Seite im Wiki.

Abweichungen vom Excel-/Tk-Programm: Die Textfelder „Name“ und „Kurzbeschreibung“ des Tk-Dialogs (ohne Wirkung auf das Makro) gibt es nicht; Einfügen und Löschen der Punkte mit Knöpfen statt mit den Tasten Einfg und Entf; der ARDUINO wird nicht schon beim Öffnen des Dialogs verbunden, sondern erst beim Ausprobieren. Bei Abstand 0 oder Dauer 1 ms hängt das Tk-Original; die Webversion rechnet mit einem kleinsten Abstand.

2.9g LED Farb-Animation (Makro LEDColorAnim, 2.88)

Das Makro **LEDColorAnim** (Gruppe PatternConfig, Dialog **LED Farb-Animation**) animiert die **Farbe einer RGB-LED**. Der Dialog arbeitet wie die Servo Animation (Abschnitt 2.9f: Fenster, Grafik, Punkte ziehen, Test), hat aber **drei Kurven**: **Rot, Grün und Blau**, jede mit eigenem **Start- und Endwert** (Abschnitt „Farbe“). Mit der Ausführung **HSV** sind es **Farbton (H), Sättigung (S) und Helligkeit (V)**; die LED bekommt die daraus berechnete Farbe.

Einstellungen: LED-Adresse und Anschlusskanal; **Ausführung** (Wiederholen, PingPong, Ein/Aus, HSV); **EIN-** und **AUS-Zeiten** (Start, Dauer, Ende in ms); **Farbverlauf** (Linear, Beschleunigung, Individuell) mit den Startwerten und Endwerten für Rot, Grün und Blau; „ARDUINO Überblendung AUS“; **Zeitstufen** (Automatisch, Fix, Individuell). Das Ergebnis ist ein Makro `PatternT…` mit den Farbwerten (je Punkt Rot, Grün, Blau) und der Kommentarzeile `#LEDColorAnim(…)` mit den 18 Einstellungen. Ein vorhandenes Makro wird beim Öffnen wieder eingelesen (bei **Individuell** auch alle drei Kurven, so wie sie in der Tabelle stehen); der Haken „Überblendung AUS“ wird daran erkannt, dass die Zeile `PatternT…` ohne vorangestelltes `A` beginnt.

Kurven bearbeiten (Farbverlauf Individuell): Mit **Aktiv** wählt man die Kurve, die sich bearbeiten lässt (Rot, Grün oder Blau); die anderen werden dünn mitgezeichnet. Die **Punkte der aktiven Kurve** lassen sich ziehen (Wert 1 bis 254; die **Zeit** nur bei den Zeitstufen Individuell, **sie gilt für alle drei Kurven**), in den Feldern **Zeit** und **Wert** eingeben, **Punkt einfügen** setzt in **allen drei Kurven** einen Punkt, **Punkt löschen** nimmt ihn aus allen. Beim Anklicken oder Ziehen eines Punktes bekommt die LED **die Farbe aller drei Kurven an dieser Stelle**; die **Farbfläche** neben „Aktiv“ zeigt dieselbe Farbe.

LED ausprobieren: Wie beim Servo: Ist ein ARDUINO verbunden, zeigt die LED die Farbe beim Anklicken und Ziehen eines Punktes und beim Ändern der Start- und Endwerte; **Starte Test** spielt den Farbverlauf in Schritten von 40 ms ab. Die Frage nach dem Anschließen des ARDUINO kommt nur einmal (Abschnitt 2.9f).

Abweichungen vom Excel-/Tk-Programm: Bei geänderter Dauer werden die Zeiten der Kurve Individuell für **alle drei Kurven** gleich gestreckt (das Tk-Programm streckte nur die erste; die Ausgabe benutzte nur deren Zeiten); beim Senden an die LED gilt der Wert 0 für Grün und Blau (das Tk-Programm ersetzte 0 durch den Startwert); Einfügen und Löschen der Punkte mit Knöpfen.

2.10 Der Patterngenerator

Der Reiter **Patterngenerator** erstellt aus einer Tabelle ein Muster für die LEDs (Wechselblinker, Lauflicht usw.). Das Ergebnis ist ein Makro, das im Programmgenerator eingetragen wird. **Stand:** Anzeigen, Bearbeiten, Berechnen sowie **Export zu ProgGen** und **Import von ProgGen** gehen. Dazu kommen **Optionen** mit den Beispielen und die Auswahldialoge für **Spezial Mode**, **RGB Modul Nr** und **Goto Aktivierung**. Die Seiten zeigen auch ihre **Bilder** und die **Goto-Pfeile**, **Bild einfügen** fügt eigene Bilder ein. **Test Pattern** (seit 2.90) spielt das Muster auf dem Bildschirm ab. **Noch nicht** gibt es das Senden an ein Modul (zurückgestellt, bis ein Tester es braucht; damit auch das Blinken der LED bei RGB Modul Nr), die Wirkung der Spezial Modes auf dem Modul und den Multiplexer.

- **Aufbau:** Oben stehen die gelben Eingabefelder (Erste RGB LED, Anzahl der Ausgabe Kanäle, Mode usw.) und darunter das Ergebnis (`PatternT1(…)`), der Makroname und das Makro. Unten liegt die Tabelle der Dauer pro Spalte und der LEDs. Ein **x** schaltet die LED in dieser Spalte ein, die **Balken** über den Zellen zeigen das Muster. Die Spalten A bis D sind gesperrt.
- **Eingeben:** Zelle anklicken und schreiben, **F2** oder Doppelklick bearbeitet eine Zelle, **Entf** leert sie. **Strg+C**, **Strg+X** und **Strg+V** kopieren, schneiden aus und fügen ein. Beim Einfügen von mehreren Zellen (aus einer Tabellenkalkulation) gilt: Zeilen durch Zeilenwechsel, Spalten durch Tabulator, ab der markierten Zelle. Das Kontextmenü (Rechtsklick) bietet dasselbe. Beim ersten Einfügen aus der Zwischenablage fragt der Browser, ob er sie lesen darf.
- **Beschreibung:** Unter der Werkzeugleiste steht (aufklappbar) die **Beschreibung des Blatts**. Beim Laden einer Datei `.MLL_pcf` kommt sie aus dem Textfeld **msoTextBox0** (der deutsche Text; gibt es mehrere, ist es das mit dem längsten Text, eine einzeilige farbige Überschrift wie „Morsecode by Lorenz“ zählt nicht). Das Feld ist **änderbar**: Der Text gehört zum Blatt und wird beim **Speichern** (Aktuelle Seite speichern, Alle Seiten speichern) wieder als **msoTextBox0** in die Datei geschrieben, mit der Lage des bisherigen Textfelds (die Höhe wächst mit der Zahl der Zeilen; hat das Blatt noch keines, entsteht eines wie bei den Beispielen). Ein fetter Anfang oder eine Farbe im Text bleiben erhalten, solange der Anfang des Textes unverändert ist, sonst entfällt das Format. Ein leerer Text entfernt das Textfeld. Die Texte der anderen Sprachen (msoTextBox1 bis 3) und die Überschrift bleiben, wie sie sind. Auf dem Blatt Main steht dort nur der Hilfetext der Seite (nicht änderbar).
- **Bilder:** Die Seiten zeigen ihre Bilder (zum Beispiel das Signal oder die Anlage zum Muster) unter der Tabelle. Die Bilder **bleiben, wo sie sind**: Bilder der Beispiele im Ordner `Pattern_Config_Examples`, eigene Bilder im Datenordner (`pattern_pictures`). Mit der Seite werden nur der Pfad und die Lage gemerkt. Fehlt eine Bilddatei nach einem Neustart, fehlt das Bild.
- **Bild einfügen:** Der Knopf lädt ein PNG- oder JPG-Bild hoch und setzt es in natürlicher Größe unter die Tabelle (Zelle B63). Da das Hochladen nur eine Zwischendatei liefert, wird die Datei in den Datenordner kopiert (gleicher Name und gleicher Inhalt: keine zweite Kopie; gleicher Name und anderer Inhalt: Name mit `_2`). Das Bild lässt sich nicht verschieben. **Optionen → Bilder dieser Seite entfernen** nimmt alle Bilder der Seite weg (die Dateien bleiben). Beim Speichern einer Seite (Optionen) gehören die Bilder zur ZIP-Datei.
- **Aussehen wie in Excel:** Die Kopfzeile der Tabelle (Spalte Nr) ist hellblau, die Spalte LED Nr hellgelb, die Beschriftung der Goto Tabelle hellgrün. Um die Zeile Dauer, die Goto Tabelle und die Tabelle der LEDs (so viele Zeilen, wie LEDs oder Kanäle eingestellt sind) liegen schwarze Rahmen. Die Zeile Ergebnis ist blau, die Zeile Makro

grün. Bei aktivem Goto Mode sind die Zeilen über und unter der Goto Tabelle viel höher als die anderen, damit die Pfeile und Bögen Platz haben, wie in Excel.

- **Goto-Pfeile:** Bei aktivem Goto Mode zeigt die Tabelle, wie das Muster startet und springt: grüne Pfeile mit der Nummer der Startspalte, rote Pfeile mit **E** am Ende des Musters und blaue gebogene Pfeile für die Sprünge zwischen den Spalten.
- **Doppelklick auf besondere Zellen:** Ein Doppelklick auf **Spezial Mode** öffnet eine Liste der Spezial Modes. Unter der Liste stehen die ausführliche Beschreibung des gewählten Eintrags; ein Doppelklick auf einen Eintrag oder **Auswahl** trägt ihn in die Zelle ein. Ein Doppelklick auf **RGB Modul Nr** öffnet eine Zahleneingabe mit − und +. Ohne ein Modul am USB-Anschluss blinkt dabei keine LED, die Nummer wird nur eingetragen. Tippen und **F2** bearbeiten die Zelle weiterhin direkt.
- **Goto Aktivierung:** Wenn der Goto Mode benutzt wird, kann das Muster an mehreren Stellen starten (Spalten mit nummerierten Pfeilen). Die **Goto Aktivierung** legt fest, wie die Startspalte im Betrieb gewählt wird (Taster, Zähler, Zufall ...). Ein Doppelklick auf die Zelle öffnet die Liste mit Kurz- und ausführlicher Beschreibung, **Löschen** leert die Zelle. Bei aktivem Goto Mode öffnet sich die Liste auch schon, wenn man die **leere** Zelle anklickt. Die Einträge `Counter`, `RandButton`, `RandomTime`, `RandomCount` und `RandomPingPong` fragen danach noch ihre Parameter ab: Zeiten werden als Zahl in Millisekunden oder mit Einheit angegeben (`30 Sek`, `5 Min`, `500 ms`) und auf ihren Bereich geprüft. Beim **Export zu ProgGen** fragt das Programm, wenn der Goto Mode mehrere Startspalten hat und keine Aktivierung eingetragen ist, ob die Auswahl geöffnet werden soll. Dann erscheinen nur Einträge mit genug Ausgängen.
- **Werkzeugleiste:**
  - **Test Pattern** (2.90): öffnet ein Fenster, in dem das Muster des Blatts **auf dem Bildschirm abläuft**, wie im Excel-Programm: Für jede LED gibt es einen **Kreis** mit dem Namen der LED (bei **RGB LED** je drei Kanäle ein Kreis), der mit den **Zeiten des Musters** (Zeile Dauer, die Zeiten werden der Reihe nach wiederholt) die Farbe wechselt. Die Farbe ist die **Farbe der LED** in der LED-Tabelle (Spalte E) mal der **Helligkeit** des Musters (bei RGB aus den drei Kanälen). Darüber steht „Muster n von N“. **Wiederholen** läuft endlos; mit dem Modus **PM_PINGPONG** vor und zurück (1, 2, 3, 2, 1, 2, ...); bei **Goto-Mustern** (Marken S, P, G, E in der Goto-Zeile) beginnt der Ablauf am Anfang, folgt den Sprüngen G und **endet an E**; mit den Knöpfen **Anfang** und **Start 1, 2 …** (je Startmarke S) beginnt man neu. **Stop** hält an, **Start** setzt fort, **Schließen** beendet den Test. Es wird **nichts an einen Arduino gesendet**. Wie im Excel-Programm zeigt der Test nur die LED-Kreise; die Wartezeit pro Muster ist die volle Zeit (das Excel-Programm zog 0,5 s Rechenzeit ab) und mindestens 40 ms.
  - **Neues Blatt:** legt eine Kopie des aktuellen Blatts an (der Name wird abgefragt, ein Reiter erscheint unten).
  - **Aktualisieren:** zeichnet die Balken neu. **Neu berechnen:** berechnet alle Formeln neu. **Hinweis:** Die Balken werden nicht mitgespeichert. Nach einem Neustart des Programms zeichnet jede Seite ihre Balken beim ersten Anzeigen (Reiter anklicken) neu. Fehlen sie trotzdem, hilft **Aktualisieren**. Die Tabelle selbst ist vollständig wieder da, und die **Bilder** der Seiten kommen von selbst zurück.
  - **+** und **−:** fügt vor der markierten Zelle eine Spalte in die Tabelle ein bzw. löscht die Spalte. Dazu muss die markierte Zelle **in der Tabelle** liegen (LED-Tabelle, Dauer oder Goto-Zeile), sonst erscheint die Meldung „Cursor outside the table“. Beim ersten Mal erklärt das Programm die Funktion.
  - **RGB LED:** schaltet das Blatt auf RGB-LEDs um.
  - **Optionen:** öffnet einen Dialog mit den Beispielen und der Dateiverwaltung.
    - **Beispiele laden:** Haken bei den gewünschten Sammlungen (zum Beispiel „Ampeln“, „Signale“, „Baustellenlichter“) oder „Alle Beispiele“, dann **Ausgewählte Beispiele laden**. Jede Sammlung bringt mehrere Seiten mit, sie erscheinen als neue Reiter. Seiten, die es schon gibt, werden nicht noch einmal geladen.
    - **Datei laden …:** lädt eine eigene Datei (`.MLL_pcf`) oder eine ZIP-Datei mit der Datei und ihren Bildern. Sie wird über den Browser hochgeladen.
    - **Aktuelle Seite speichern**, **Alle Seiten speichern:** schreibt die Seite(n) in eine Datei im Format des Excel-Programms (`.MLL_pcf`) und bietet sie zum Speichern an (Speichern unter … oder Downloads-Ordner). Gehören Bilder zu den Seiten, entsteht eine ZIP-Datei mit allen Dateien.
    - **Aktuelle Seite löschen**, **Alle Beispiele löschen:** löscht die Seite bzw. alle Seiten außer Main nach einer Rückfrage. Das kann nicht rückgängig gemacht werden.
  - **Export zu ProgGen:** trägt das Makro des Blatts in eine Zeile des Programm Generators ein. Vorher prüft das Programm, ob ein Makroname eingetragen ist, die LED-Tabelle nicht leer ist und die Dauer-Zeile stimmt, und meldet sonst den Fehler. Danach springt der Browser zum Programm Generator, über der Tabelle erscheint ein **gelbes Band**: In die Zeile klicken, in der das Muster stehen soll, und **OK** drücken. Die Haken „gleich zum Arduino schicken“ (erzeugt die Dateien, kompiliert und lädt hoch) und „Anschließend zurück zum Patterngenerator“ gelten für diesen Vorgang. Bereits vorhandene Einträge in der Spalte „Beleuchtung, Sound, oder andere Effekte“ werden überschrieben, Symbol und Name der Zeile kommen dazu. **Abbrechen** bringt zurück zum Patterngenerator.
  - **Import von ProgGen:** liest ein Muster aus einer Zeile des Programm Generators. Im gelben Band eine Zeile mit einem `PatternT…`-Makro anklicken und **OK** drücken. Das Programm fragt, ob die Daten in das aktuelle Blatt kommen oder ein neues Blatt angelegt wird. Danach zeigt der Browser den Patterngenerator. Wie in der Excel-Version steht im Feld „Schalter Nummer“ danach `#InCh`.
- **Beschreibung:** Der Text des Blatts lässt sich über „Beschreibung“ oben aufklappen.
- **Spaltenbreite:** Die Muster-Spalten sind gleich breit. Das Programm passt die benutzten Spalten an die Dauer an, wenn die Zeiten sehr unterschiedlich sind (wie in Excel). In dieser Tabelle lassen sich die Spalten nicht von Hand

verstellen, weil die Balken an den Spaltenbreiten hängen.
- **Daten:** Die Blätter werden beim Beenden im Datenordner gesichert (`temp_workbook.table_PatternWorkbook.json`) und beim nächsten Start geladen.

Dateien aus dem Excel-Programm laden: Dateien im Format `.MLL_pcf` und `.MLL_pgf`, die mit dem Excel-Programm gespeichert wurden, haben den Windows-Zeichensatz (cp1252; als Trennzeichen im Patterngenerator dient das Zeichen `¯`). Das Programm erkennt den Zeichensatz beim Laden selbst: Dateien in **UTF-8** (vom Web-Programm gespeichert, auch mit Kennzeichen am Anfang) und im **Windows-Zeichensatz** lassen sich beide laden, mit Umlauten und Sonderzeichen. Früher kam bei einer Datei aus Excel die Meldung „'utf-8' codec can't decode byte 0xaf …“. Beim **Speichern** schreibt das Programm die Dateien (`.MLL_pcf` und `.MLL_pgf`) ebenfalls im **Windows-Zeichensatz**, damit auch das Excel-Programm sie lesen kann (sonst stimmen Trennzeichen und Umlaute dort nicht). Das lässt sich in den **Programm Einstellungen** mit der Option **Dateien für das Excel-Programm speichern** abschalten (Standard: an). Enthält eine Datei Zeichen, die im Windows-Zeichensatz keinen Platz haben (zum Beispiel ein polnisches „ł“), bleibt sie UTF-8; das Excel-Programm kann sie dann nicht richtig lesen. Das Laden geht immer mit beiden.

2.11 Vollbild und Adressen der Reiter

- **Verbindung zum Arduino:** In der Kopfzeile, links neben **Vollbild**, steht für **alle Reiter** der Knopf **Mit ARDUINO verbinden**. Er ist **rot**, solange keine Verbindung besteht, und **grün** („Mit ARDUINO verbunden“), wenn die Verbindung steht (auch wenn man im Menü ARDUINO → Verbinden oder Trennen wählt). Ein Klick auf den grünen Knopf bewirkt nichts; getrennt wird im Menü ARDUINO → Trennen. Nach dem Verbinden startet ein offener Reiter mit Direktmodus (Servo Einstellungen, DM Servo Einstellungen, Sound Test) den Direktmodus neu.
- **Version der Bibliothek:** Auf der Seite Programm Generator steht rechts in der Zeile der Funktionsknöpfe die Version der MobaLedLib, zum Beispiel **Ver.3.5.0-K** (wie in Excel). Das Feld bleibt leer, solange die Bibliothek nicht installiert ist. Die Version wird beim Aufbau der Seite, nach der Prüfung der Bibliotheken beim Start und nach jedem Befehl (zum Beispiel einer Installation der Bibliotheken) gelesen, nicht laufend.
- **Vollbild:** Der Knopf **Vollbild** in der Kopfzeile (neben **Beenden**) blendet Titel, Menü und Reiterleiste aus. Die Statusmeldungen und die Knöpfe Vollbild und Beenden bleiben, die Tabelle bekommt den Platz. Wenn der Browser es erlaubt, geht er dabei auch in den Vollbildmodus (auf dem iPhone gibt es das nicht, dort wirkt nur das Ausblenden). Derselbe Knopf (dann **Vollbild beenden**) oder die Taste ESC schaltet zurück. Gedacht ist es für kleine Bildschirme, zum Beispiel das Smartphone. Im Vollbild ist die Reiterleiste weg: Zum Wechseln erst das Vollbild beenden.
- **Reiter direkt aufrufen:** Jeder Reiter hat eine eigene Adresse, zum Beispiel `http://localhost:8080/Farbtest`:

Adresse	Reiter
`/ProgGenerator`	Programm Generator
`/Patterngenerator`	Patterngenerator
`/ProgGeneratorEinstellungen`	ProgGenerator Einstellungen
`/Farbtest`	Farb Test
`/ServoTest1`	Servo Einstellungen
`/ServoTest2`	DM Servo Einstellungen (nur mit eingeschalteter Option, siehe 2.13)
`/SoundTest`	Sound Test
`/DCCKeyboard`	DCC Keyboard
`/Z21Simulator`	Z21 Simulator (nur mit eingeschalteter Option, siehe 2.14)
`/SerialMonitor`	Serial Monitor
`/ARDUINOEinstellungen`	ARDUINO Einstellungen
`/ProgrammEinstellungen`	Programm Einstellungen

Groß- und Kleinschreibung ist egal (`/farbtest` wird umgeleitet). Mit dem Zusatz `?vollbild=1` öffnet sich der Reiter gleich im Vollbild (zum Beispiel `/ServoTest1?vollbild=1`); der erste Tipp auf die Seite schaltet dann auch den Browser auf Vollbild. Bei einem unbekannten Namen zeigt das Programm die Liste der gültigen Adressen. Die Startseite `/` bleibt der Programm Generator. Aus dem Netz heißt es `http://<Adresse des Rechners>:8080/Farbtest`; das Passwort (Kapitel 8) gilt auch hier.

2.12 Servo Einstellungen

Der Reiter **Servo Einstellungen** (früher Servo Test 1, Adresse `/ServoTest1`) bedient die Servoplatine der MobaLedLib (Servos über LED-Befehle) und ist wie der **Excel-Dialog „Servo Test und Einstellung“** aufgebaut. Er arbeitet über dieselbe serielle Verbindung wie der Farb Test: zuerst mit dem Arduino **verbinden** (Knopf **Mit ARDUINO verbinden** in der Kopfzeile oder Menü ARDUINO → Verbinden).

- **Servo wählen:** In der Zeile **Servo Modul Auswahl** stehen **LED Nummer**, **Anschluss** (0 bis 2) und **LED Kanal**. Nach jeder Änderung wählt das Programm den Servo und lässt ihn **kurz wackeln**, damit man ihn erkennt. Die Werte werden gemerkt.

- **Position:** Der Regler (10 bis 210) mit den Pfeilen daneben stellt die Position ein, rechts steht der Wert.
- **Servo Konfigurieren** führt in drei Schritten durch **Minimalposition**, **Maximalposition** und **Geschwindigkeit**. Mit den Pfeiltasten **<< < > >>** wird der Wert eingestellt (**gedrückt halten**: schneller). **Weiter** geht zum nächsten Schritt, **Einstellungen speichern** speichert (in jedem Schritt möglich), **nicht speichern** bricht ab.
- **Servo zurücksetzen** stellt nach einer Rückfrage die Grundeinstellungen des Servos wieder her (Limits und Geschwindigkeit).
- **Beschreibung:** Der Text oben ist einzeilig; ein Tipp darauf klappt ihn auf.
- **Verlassen:** Beim Wechseln des Reiters, beim Schließen oder Neuladen der Seite und beim Beenden des Programms bekommt der Servo den **Wert 0**, ein laufender Ablauf wird abgebrochen und der Direktmodus beendet. So bleibt kein Servo in Bewegung.
- Die Seite ist für das **Smartphone** gebaut (eine Spalte, große Tasten); zusammen mit dem Vollbild (2.11) bleibt viel Platz.

2.13 DM Servo Einstellungen, Sound Test und DCC Keyboard

- **DM Servo Einstellungen** (früher Servo Test 2, Adresse `/ServoTest2`) steuert Servos im **Direct Mode** (ein ATtiny statt des WS2811-Chips). Parameter: **Servo Modul Adresse**, **LED Kanal**, **Servo Betriebsart** (1 Normal, 2 und 3 Training, 4 Max Speed, 5 Invers, 6 Servo zurücksetzen, 7 Werkseinstellung, 8 letzte Position zurücksetzen, 9 Oszillator trimmen, 10 Speed Legacy; 0: es wird nichts gesendet) und **Fast Refresh**. Regler 0 bis 255, **Pos 0**, **Pos 255** und die Taste **ENTER** (gedrückt halten; in den Betriebsarten 2 und 3 zweimal drücken, die Beschriftung zeigt „2nd Step“). Nach ENTER in den Betriebsarten 4 bis 8 und 10 steht die Betriebsart auf 0. Die Betriebsart wird nicht gemerkt, die Seite beginnt immer mit 1. Das Programmieren des ATtiny (Hex-Datei) und das Formular für den Programmieradapter gibt es noch nicht. **Der Reiter erscheint nur, wenn in den Programm Einstellungen die Option „DM Servo Einstellungen anzeigen“ eingeschaltet ist** (und nach dem Speichern sofort, ohne Neustart; vorher hieß die Option „Programmgenerator anzeigen“ und hatte im Web keine Wirkung). Ruft man `/ServoTest2` bei ausgeschalteter Option auf, kommt ein Hinweis, und der Programm Generator bleibt offen.
- **Sound Test** (`/SoundTest`) prüft die MLL-Soundplatine (MP3-TF-16): **Soundmodul-Adresse**, **LED Kanal**, **Impulslänge** und die Werte **R, G, B** (Sound-Auswahl) bestimmen den Ton. **Sound an ARDUINO schicken** sendet den Wert und beendet den Impuls nach der Impulslänge; Änderungen an Adresse, Impulslänge, R, G und B senden gleich (nach kurzer Ruhe beim Tippen). Zum Verbinden dient der Knopf **Mit ARDUINO verbinden** in der Kopfzeile (siehe 2.11). Beim Verlassen des Reiters oder der Seite wird ein laufender Impuls beendet. Die Tastenkürzel Strg-R und Alt-R der Tk-Seite gibt es nicht (Strg-R lädt im Browser die Seite neu).
- **DCC Keyboard** (`/DCCKeyboard`) schickt dem ARDUINO die Kommandos `@ AAA CC 01`, die sonst der DCC-ARDUINO an den LED-ARDUINO schickt, damit man die DCC-Schalter ausprobieren kann: 32 Adressen ab der **DCC-Basisadresse**, je ein roter (Kanal 0, mit der Adresse beschriftet) und ein grüner Knopf (Kanal 1), in vier Gruppen zu acht. Beim ersten Tastendruck verbindet die Seite selbst; das kann etwas dauern, und manchmal reagiert der ARDUINO erst auf den zweiten Tastendruck.
- Alle drei Seiten sind für das Smartphone gebaut (Beschreibung einzeilig und aufklappbar, große Tasten) und im Vollbild (2.11) gut zu bedienen.

2.14 Z21 Simulator

Der Reiter **Z21 Simulator** (`/Z21Simulator`) gibt sich gegenüber Modellbahn-Steuerprogrammen (Rocrail, iTrain, Traincontroller, WinDigiPet, Z21-Apps) als **Roco Z21** aus. Weichen- und Lokbefehle des Steuerprogramms schickt er als DCC-Kommando an den Arduino (`@ AAA CC AA`), so lassen sich die DCC-Schalter der MobaLedLib direkt aus dem Steuerprogramm bedienen. Die Z21 zählt Adressen ab 0, die MobaLedLib ab 1; der Simulator rechnet das um.

- **Einschalten:** Programm Einstellungen → **Z21 Simulator anzeigen**, dann Speichern: Der Reiter erscheint sofort (ohne Neustart).
- **Bedienung:** Zuerst mit dem Arduino verbinden (Knopf **Mit ARDUINO verbinden** in der Kopfzeile; der Start versucht es selbst, startet aber auch ohne Verbindung). Dann **Start Simulator**. Die Seite zeigt die **IP-Adresse und den Port 21105 (UDP)**, die im Steuerprogramm als Z21 einzutragen sind. Läuft das Steuerprogramm auf demselben Rechner, genügt `127.0.0.1`. Im Fenster darunter steht jedes Paket mit Uhrzeit, Absender und dem Befehl an den Arduino. **Stop Simulator** beendet ihn. **Zeige Taster-Adressen** zeigt die Eingangskanäle aus der zuletzt erzeugten Header-Datei.
- Der Simulator läuft weiter, wenn man den Reiter wechselt oder die Seite neu lädt. Er endet mit **Stop** oder mit dem Beenden des Programms.
- **Firewall und Netz:** Das Steuerprogramm muss den **UDP-Port 21105** erreichen. **Windows** fragt beim ersten Start nach der Freigabe für Python: **Private Netzwerke** erlauben. **Linux** mit `ufw`: `sudo ufw allow 21105/udp`. Auf dem **Raspberry Pi** ist der Port normalerweise frei, der Simulator läuft dann dauerhaft auf dem Pi. Ist der Port schon belegt (andere Z21-Software, zweiter Simulator), erscheint eine Meldung.
- **Rückmeldung (RMBUS):** Hat das Steuerprogramm die Rückmeldung angemeldet (Broadcast-Flag „R-Bus“, so tun es zum Beispiel Rocrail und iTrain für die Rückmelder), fragt der Simulator den Arduino alle 100 ms nach dem Stand der Taster und Eingänge und meldet **Änderungen** an das Steuerprogramm (je Gruppe von 10 Modulen). Im Fenster erscheint nur eine Zeile, wenn sich etwas ändert; die Statuszeile nennt, für wie viele Programme die Rückmeldung läuft. Das Intervall steht in der (versteckten) Einstellung `RMbusTimer` (× 100 ms; 0 oder leer = 100 ms).
- **Noch nicht umgesetzt:** Die **Port-Liste** (bis zu drei Arduinos mit eigenen DCC-Adressbereichen); es wird der eine

mit dem Programm verbundene Arduino benutzt.

2.15 Sprache der Oberfläche

In den **Programm Einstellungen** wählt man unter **Sprache**: Auto (die Sprache des Rechners, auf dem das Programm läuft), Deutsch, English, Nederlands, French, Italian, Spanish oder Denmark. Nach dem **Speichern** wirkt die Auswahl **sofort**:

Aufbau der Seite Programm Einstellungen (seit 2.94): Die Einstellungen stehen in **Gruppen mit Überschrift**: **Allgemeine Einstellungen** (Anfänger Modus, Sprache, Darstellung auf kleinen Bildschirmen, Schriftgröße, Schrift von ProgGen/PattGen, Icon Größe), **Programm Generator** (Standardansicht, versteckte Tabellen anzeigen, Bedienung der Tabelle), **Farb Test Einstellungen** (Farbauswahl, LED-Farbkorrektur), **Zugriff aus dem Netz** (Schalter und Passwort) und **Spezialeinstellungen** (alles andere: DM Servo Einstellungen anzeigen, Z21 Simulator anzeigen, IP Upload Timeout, RMBUS-Interval, Dateien für das Excel-Programm speichern). Die Option **Check for Updates** hatte keine Funktion und entfällt. Die **Beschreibungstexte** der Seiten (zum Beispiel unter der Überschrift von Programm Einstellungen, ARDUINO Einstellungen, Farb Test, DCC Keyboard, Sound Test, Servo Einstellungen 2 und Z21 Simulator) sowie die Beschriftungen der Seiten Programm Einstellungen, ARDUINO Einstellungen und Farb Test werden in der eingestellten Sprache angezeigt.

Anfänger Modus und Experten Modus (seit 2.96): Eine **neue Installation** startet im **Anfänger Modus**. Er blendet aus, was für den Einstieg nicht nötig ist: die Reiter **Patterngenerator**, **Z21 Simulator** und **ProgGenerator Einstellungen**, den Knopf **Zeilen Färben** und die Makros der Liste `csv/Beginner_Hidden_Macros.csv` in der **Makroauswahl**, in der Suche und in der Grafik. Umschalten: **Programm Einstellungen → Allgemeine Einstellungen → Haken „Anfänger Modus“** (Haken weg = **Experten Modus**, alles sichtbar), danach **Speichern**; die Reiter und der Knopf erscheinen und verschwinden sofort, der Makrobaum beim nächsten Öffnen. **Bestehende Installationen** bleiben nach einem Update im Experten Modus. Der Z21 Simulator und der Reiter DM Servo Einstellungen erscheinen im Experten Modus wie bisher nur mit ihrer Option. Ruft man die Adresse eines ausgeblendeten Reiters auf (zum Beispiel `/Patterngenerator`), öffnet sich der Programm Generator mit einem Hinweis. **Makros**, die schon in der Tabelle stehen, bleiben und lassen sich weiter öffnen; ausgeblendet wird nur die Auswahl neuer Makros. Die **Liste** der ausgeblendeten Makros ist eine Textdatei (Zeilen `Makro;<Name>` oder `Gruppe;<Gruppe>`, Beschreibung am Anfang der Datei); sie ist zunächst **leer** (welche Makros für Anfänger fehlen, wird noch besprochen) und braucht keinen Neustart. Die Seite wird in allen Browsern neu geladen, und alle Texte erscheinen in der neuen Sprache. Ein Neustart ist nicht nötig.

- **Was übersetzt wird:** alle Texte der Oberfläche: Menüs, Reiter, Knöpfe, Hinweise, die Texte und Beschreibungen in den Formularen (zum Beispiel „Verteiler und Stecker Nummer“, „Optionen und spezielle Funktionen“) und in den **Programm Einstellungen**, die Meldungen des Programm Generators und des Patterngenerators, die **Spaltenköpfe** mit ihren Hinweisen, die **Namensspalte** und die **Typen** in der Tabelle. Die Texte stehen an zwei Stellen: Die Texte, die es schon im Excel-Programm gab, kommen wie dort aus der Tabelle Languages der Excel-Datei; alles, was es nur im Web-Programm gibt, steht im Katalog `python/csv/Web_Languages.csv`. Fehlt ein Text in der Tabelle Languages, wird im Katalog nachgesehen. Stand: **English** ist vollständig (Entwurf), die anderen Sprachen sind nur dort übersetzt, wo die Tabelle Languages eine Übersetzung hat (sonst erscheint English).
- **Gespeichert wird immer deutsch:** In den Zellen, in den Dateien (`.MLL_pgf`) und im erzeugten Arduino-Programm stehen die deutschen Begriffe. Ein Sprachwechsel **ändert keine Daten**; Dateien lassen sich auch in einer anderen Sprache lesen. Eine Datei mit den Begriffen einer anderen Sprache (zum Beispiel aus dem Excel-Programm mit niederländischer Oberfläche) wird beim Laden in die deutschen Begriffe umgewandelt. In der Spalte Typ darf man die Begriffe oder ihren ersten Buchstaben in der eingestellten Sprache eintippen.
- **Spaltenköpfe und Namensspalte:** Die Köpfe der Datenblätter (Zeile 2, mit ihrem Hinweistext beim Daraufzeigen) und die **Namensspalte** (der Name des Makros, zum Beispiel „Belebtes Haus“ / „Busy house“ / „Druk huis“) erscheinen in der gewählten Sprache. Auch das nur in der Anzeige: Die Zellen bleiben unverändert. Ein Kopf, den man selbst geändert hat, bleibt, wie er ist. Der Name wird wie in Excel aus der Tabelle Lib_Macros bestimmt (Spalte Name der Sprache, sonst der Name ohne Sprache), auch auf Deutsch: Ein Name, der beim Laden einer Datei in einer anderen Sprache entstanden ist, wird dadurch ersetzt.
- **Meldungen mit eingesetzten Werten und zusammengesetzte Meldungen** (zum Beispiel „Fehler beim Speichern: …“, die Meldungen des Updates, der Firewall, des Z21 Simulators, „… Seite(n) gelöscht“) werden ebenfalls übersetzt; die eingesetzten Werte (Dateinamen, Zahlen, Fehlertexte des Systems) bleiben, wie sie sind.
- **Noch nicht übersetzt** (erscheinen auf Deutsch): die Beschreibungen in den Beispielblättern, die Anleitung und Texte, die nur im Excel-Programm vorkommen (Animationsformulare der Tk-Version).
- **Fehlende Übersetzungen:** Fehlt ein Text in der gewählten Sprache, erscheint der englische, fehlt auch der, der deutsche. Stand: English (Entwurf) für alle Texte der Oberfläche (766 Einträge im Katalog; English, Nederlands, Français, Italiano, Español und Dansk als Entwürfe); Nederlands, French, Italian, Spanish und Denmark folgen. Wie man mitübersetzt, steht in `install/UEBERSETZEN.md`.

2.16 Entwicklungsumgebung: ARDUINO IDE oder PlatformIO

Programme werden mit der **ARDUINO IDE** (Standard, Abschnitt 2.2) oder mit **PlatformIO** übersetzt und auf den Arduino, Pico oder ESP32 geladen. PlatformIO ist ein Ersatz für die ARDUINO IDE; man braucht dafür nur das Kommandozeilenprogramm `pio`, nicht die IDE.

- **Umschalten:** Menü **Optionen → Optionen und spezielle Funktionen**, Reiter **Entwicklungsumgebung**:

ARDUINO-IDE (Standard) oder *PlatformIO (Ersatz für die ARDUINO-IDE)*. Die Auswahl wird beim Schließen des Dialogs übernommen. Darunter steht, ob PlatformIO gefunden wurde; der Knopf **PlatformIO prüfen** zeigt die Version.

- **Installieren:** PlatformIO muss einmal installiert werden (Anleitung: https://platformio.org/install/cli).
  - **Windows:** zum Beispiel mit `pip install platformio` oder mit der PlatformIO-Erweiterung von Visual Studio Code.
  - **Linux, macOS und Raspberry Pi:** mit **pipx**: `pipx install platformio`, danach `pipx ensurepath` und neu anmelden (oder den Rechner neu starten). Debian, Ubuntu und Raspberry Pi OS erlauben `pip install` nicht außerhalb einer virtuellen Umgebung (Meldung „externally-managed-environment“); dort vorher `sudo apt install pipx`. Das Installationsskript von platformio.org (Ordner `~/.platformio`) geht ebenfalls.
  - **Mit dem Installationsskript:** `install_windows.bat /platformio`, `bash install_linux.sh --with-platformio`, `bash install_mac.command --with-platformio` oder `sudo bash install_pi.sh --with-platformio` (auch zusammen mit `--with-arduino-ide`). Das Skript legt eine **eigene Python-Umgebung** `~/.platformio/penv` an (Windows: `%USERPROFILE%\.platformio\penv`), genau wie der Installer von platformio.org, und installiert PlatformIO dort hinein. Das geht auch auf Debian, Ubuntu und Raspberry Pi OS (die Meldung „externally-managed-environment“ tritt nicht auf), `pipx` ist nicht nötig. Auf Linux, macOS und Raspberry Pi geht es auch einzeln und nachträglich: `bash install/install_platformio.sh` (ohne `sudo`). Auf dem Raspberry Pi gehört die Umgebung dem Benutzer des Dienstes. Das Skript installiert nur PlatformIO; danach in den Optionen **Entwicklungsumgebung** auf PlatformIO einstellen (**PlatformIO prüfen** zeigt den Stand). Das erste Hochladen lädt die Compiler und Bibliotheken nach (Internet): **etwa 5 Minuten beim Nano und etwa 10 Minuten beim Pico**, nur beim ersten Mal. Die Installation von PlatformIO selbst lädt ebenfalls eine größere Datenmenge.
  - Das Programm findet `pio` selbst: im Ordner von PlatformIO (**Windows:** `%USERPROFILE%\.platformio\penv\Scripts`; **Linux, macOS und Raspberry Pi:** `~/.platformio/penv/bin`), unter Linux und macOS auch in `~/.local/bin` (dort legt pipx es ab), und im Pfad (PATH). Fehlt es, erscheint beim Hochladen eine Meldung mit dem Installationshinweis; man kann dann in den Optionen wieder die ARDUINO IDE einstellen.
- **Ablauf:** Beim Hochladen schreibt das Programm die Datei `platformio.ini` (im Ordner `python`) mit den aktuellen Einstellungen (Boards `nano_new`, `nano_old`, `nano_full`, `pico`, `esp32`; die Bibliothek MobaLedLib kommt aus dem Ordner `libraries`) und startet `pio run -t upload`. Beim ersten Mal lädt PlatformIO die Werkzeuge für das Board aus dem Internet; das kann einige Minuten dauern.
- **Raspberry Pi Pico unter Windows:** Für das Hochladen mit PlatformIO braucht der Pico unter Windows **einmalig einen zusätzlichen USB-Treiber**. Den Pico mit gedrückter BOOTSEL-Taste anstecken (Bootloader-Modus), das Programm **Zadig** starten (https://zadig.akeo.ie), unter *Options → List All Devices* den Eintrag **„RP2 Boot (Interface 1)“** wählen, als Treiber **WinUSB** einstellen und *Install Driver* (oder *Replace Driver*) anklicken. Danach lässt sich der Pico hochladen. Fehlt der Treiber, steht in der Fehlermeldung von PlatformIO bereits ein Hinweis auf Zadig.
- **Anschluss scannen (Dialog „Überprüfung des USB Ports“):** Solange der Dialog offen ist, aktualisiert das Programm jede Sekunde die Liste der Anschlüsse; ein **neu angeschlossenes Gerät (zum Beispiel ein Pico) wird gleich gewählt**. Bei einem **Pico und ESP32 unter Linux und macOS** wird der Anschluss dabei **nicht** mehr jede Sekunde geöffnet und geprüft (das stört deren USB-Anschluss), nur bei einem Arduino. Für die Fehlersuche schreibt das Programm bei jeder Änderung der Liste eine Zeile `Anschluesse: [...] (Board ..., unbekannte Ports anzeigen: ..., gewaehlt: ...)` ins Protokoll. Zeigt die Liste den Pico nicht: das **Board** in den Optionen muss **Raspberry Pi Pico** sein (sonst erscheint er nur mit „Unbekannte Ports anzeigen“), und unter Linux muss der Benutzer in der Gruppe `dialout` sein oder die udev-Regel (Knopf in den Optionen) eingerichtet haben.
- **PlatformIO benutzt noch die alte Fassung der Bibliothek (2.66):** PlatformIO übernimmt die Bibliothek MobaLedLib als **Kopie** in den Ordner `.pio` des Projekts (`.pio/libdeps/<Board>/MobaLedLib`) und merkt sich übersetzte Dateien in `.pio/build`. Ändert sich eine Datei der Bibliothek (zum Beispiel eine `.h`-Datei), ohne dass sich die Versionsangabe in der Datei `platformio.ini` ändert (`MobaLedLib=file://libraries/MobaLedLib@3.5.0-L3`), nimmt PlatformIO weiter die **alte Kopie**. **Seit 3.16** schreibt das Programm `MobaLedLib=symlink://libraries/MobaLedLib@3.5.0-L3` (wie das Excel-Programm 3.5.0L3): PlatformIO **verknüpft** die Bibliothek dann, statt sie zu kopieren, und eine geänderte Datei gilt sofort. Das geht nur, wo der Rechner symbolische Verknüpfungen anlegen darf (Linux, macOS, Windows mit **Entwicklermodus** oder Administratorrechten); sonst nimmt das Programm selbst wie bisher `file://` (Kopie). Im Protokoll steht, was gilt: „platformio.ini: Bibliothek MobaLedLib wird mit symlink:// eingebunden“. Abhilfe: in den **Optionen** im Reiter **Entwicklungsumgebung** den Knopf **„PlatformIO-Zwischenspeicher löschen“** drücken. Das Programm zeigt, was gelöscht wird (den Ordner `.pio` der Projekte, die dieses Programm angelegt hat, und den Download-Zwischenspeicher `.cache` im Ordner von PlatformIO, mit Größe), fragt nach (**Ja** / **Nein**) und löscht erst nach **Ja**. **Das nächste Übersetzen baut alles neu auf** und dauert deshalb länger (die Bibliotheken werden neu geholt). **Erhalten bleiben** die Plattformen, Compiler und Werkzeuge von PlatformIO (`packages`, `platforms`), alle Quelldateien und die Datei `platformio.ini`; Projekte, die nicht von diesem Programm stammen, werden nicht angefasst. Läuft gerade ein Übersetzen oder ist eine Datei geöffnet, meldet das Programm, was nicht gelöscht werden konnte; danach noch einmal versuchen. Der Knopf ist auf **Linux, Windows und macOS** da. Von Hand geht es ebenso: den Ordner `.pio` im Projektordner löschen (neben `platformio.ini`).
- **Raspberry Pi Pico mit PlatformIO braucht den Git-Client (`git`):** PlatformIO holt die Plattform des Pico (`platform-raspberrypi`) aus GitHub. Ohne `git` endet das Übersetzen mit „UserSideException: Please install Git client“ (im Programm erscheint danach eine Meldung mit den Befehlen). **Linux (Mint, Ubuntu, Debian, Raspberry Pi OS, Fedora, Arch, openSUSE):** in den **Optionen** im Reiter **Entwicklungsumgebung** den Knopf

Git installieren drücken (erscheint nur, wenn `git` fehlt, **unter Linux, Windows und macOS**; es erscheint ein Hinweisfenster „Git wird installiert ... kann mehrere Minuten dauern“, danach meldet das Programm „git ist installiert“; das Skript unter Linux ist `install/install_git.sh`), oder im Terminal `sudo apt install git`; `install_linux.sh` (ab Version 1.06) installiert `git` gleich mit. **Windows:** Der Knopf installiert `git` mit **winget** (`winget install --id Git.Git -e`); gibt es winget nicht oder scheitert es, lädt das Programm das Installationsprogramm von **Git for Windows** (von GitHub, mit Prüfsumme) und startet es still. **Windows fragt dabei um Erlaubnis** (Benutzerkontensteuerung): mit **Ja** bestätigen. Das Programm trägt den Ordner von `git` danach selbst in `PATH` ein, ein Neustart ist nicht nötig; von Hand: Git von https://git-scm.com/download/win installieren und das Programm danach neu starten. **macOS:** Der Knopf installiert `git` mit **Homebrew** (`brew install git`), wenn es vorhanden ist; sonst öffnet er die **Kommandozeilen-Entwicklerwerkzeuge von Apple** (`xcode-select --install`, enthalten `git`): **macOS öffnet ein Fenster, dort auf „Installieren“ klicken**; die Installation läuft danach im Hintergrund weiter (mehrere Minuten) und das Programm wartet nicht darauf: erscheint die Meldung „macOS hat ein Fenster ... geöffnet“, die Installation abwarten und danach das Übersetzen wiederholen. Von Hand: im Terminal `xcode-select --install`. Für den Nano, Uno und ESP32 ist `git` nicht nötig.

- **Raspberry Pi Pico: Start-Firmware und fertige Firmware installieren (neu):**
  - **Hinweis „PICO im Bootmodus gefunden“:** Steckt man den Pico mit gedrückter Taste **BOOTSEL** ein (er erscheint als Laufwerk **RPI-RP2**), während der Dialog zur Wahl des USB-Ports offen ist, fragt das Programm, ob der Pico mit der **Start-Firmware** (`Boards/Heartbeat.uf2`) initialisiert werden soll. **Ja** kopiert die Datei auf den Pico; er startet danach neu. Das funktioniert unter Windows, macOS und **Linux** (Mint, Ubuntu, Raspberry Pi OS): Das Programm findet das Laufwerk, wo immer der Desktop es einhängt (`/media/<Benutzer>`, `/run/media/<Benutzer>`), und hängt ein noch nicht eingehängtes Laufwerk selbst ein (Befehl `udisksctl`, ohne Passwort; Voraussetzung ist der Dienst udisks2, der auf den Desktop-Systemen läuft).
  - **Knopf „Firmware installieren“** (Optionen, Reiter **LED Arduino**; **seit 3.14 für alle Boards sichtbar** (wie im Excel-Programm 3.5.0L3; vorher nur beim Raspberry Pi Pico und nur mit Firmware-Dateien im Ordner); der Ordner `MobaLedLib/firmware` wird bei Bedarf **angelegt**, die Schreibweise `Firmware` statt `firmware` wird unter Linux auch gefunden; liegt keine Datei `*.uf2` im Ordner, sagt das Programm es): Öffnet eine Auswahl der **fertigen Firmware-Dateien** (`*.uf2`) im Ordner **`MobaLedLib/firmware`** des Sketchbook-Ordners (mit PlatformIO ohne IDE ist das der Ordner `python`, also `python/MobaLedLib/firmware`). Links stehen die Firmware-Namen mit der höchsten Version, rechts die **Version** (ein Menü) und die **Beschreibung** (aus der gleichnamigen Datei `.desc`, JSON mit Abschnitten `DE`, `EN` usw.; ohne `.desc` ergeben sich Name und Version aus dem Dateinamen, zum Beispiel `Servo-v2.1.uf2`). **Installieren** schreibt die Datei auf den Pico: Ist er schon im Bootmodus, wird sie gleich kopiert; läuft er, versetzt ihn das Programm über den **seriellen Anschluss** (kurz mit 1200 Baud öffnen, wie `uf2conv.py` im Excel-Programm) in den Bootmodus und wartet bis zu 20 Sekunden auf das Laufwerk. Meldet das Betriebssystem beim Öffnen des Anschlusses einen Fehler, weil der Pico im selben Augenblick neu startet (Windows zum Beispiel „Ein nicht vorhandenes Gerät wurde angegeben“), wartet das Programm trotzdem auf das Laufwerk; erst wenn es nicht erscheint, steht die Meldung mit diesem Grund da. Fehlt das **Recht** auf den Anschluss (Linux), bricht es sofort ab. Ist kein Pico zu finden, sagt das Programm, was zu tun ist. **Linux:** Der Anschluss (`/dev/ttyACM0`) muss für den Benutzer zugänglich sein (Gruppe `dialout`); ist der Ordner leer oder fehlt er, nennt das Programm ihn.
- **Excel-Programm 3.5.0L3 (neu in 3.14):** Der **Nano Every** (ATmega4809) wird nicht mehr unterstützt (Konstante, Erkennung über die Gerätesignatur, Skripte für den schnellen Bootloader und der Sonderweg bei der Bootloader-Prüfung sind entfallen). **Servo Test:** Solange die Pfeiltasten **<< < > >>** aktiv sind (Minimal- und Maximalposition einstellen), bedient die **Tastatur** sie: **Pfeil hoch** = `>`, **Pfeil runter** = `<`, **Bild hoch** = `>>`, **Bild runter** = `<<`; die Taste gedrückt halten wie die Maustaste; mit Umschalt, Strg, Alt oder Meta tut sie nichts; die Seite scrollt dabei nicht. **Anschlussliste:** `EnumComPorts`, `Check_If_Port_is_Available` und `Check_If_Port_is_Available_And_Get_Name` haben den **Board-Typ als Parameter** (Vorgabe: der Board-Typ des Blatts). **Tabellen:** `csv/Languages.csv` (1175 Zeilen, vorher 1061), `csv/Lib_Macros.csv` (376 Zeilen; neue Makros zum Beispiel `SingleLedSignal`, `CopyNLEDs`, `RGB_Ring`, `Signal2`, `InCh_to_LocalVar`, `Name_LED_Number`; **ohne** die Zeilen der geladenen Erweiterungen `EX.*`, zum Beispiel die Test-Erweiterungen `HLMakro`, `HL_Test1` und die Schedule-Erweiterung), `csv/Par_Description.csv` (199 Zeilen) und `csv/Platform_Parameters.csv` (ESP32 und PICO: bis zu 16384 LEDs und 2000 Eingänge) stammen aus der Arbeitsmappe 3.5.0L3 (Hilfsprogramm `xlsm_nach_csv.py`). **Noch nicht übernommen:** die Bibliothek `MobaLedLib@3.5.0-L3` samt Test-Suffix `L3` und `symlink://` in der `platformio.ini`. **Firmware herunterladen:** siehe unten (seit 3.15).
- **Firmware herunterladen (neu in 3.15, wie im Excel-Programm 3.5.0L2/L3):** Im Dialog **Firmware installieren** (Optionen, Reiter **LED Arduino**, bei jedem Board) gibt es den Knopf **Firmware herunterladen**. Er holt die **Liste der verfügbaren Firmware** aus dem Internet (`https://raw.githubusercontent.com/Hardi-St/MobaLedLib_Docu/master/firmware/index.json`; Anzeige „Liste der verfügbaren Firmware wird geladen...“, Zeitgrenze 30 Sekunden) und zeigt einen **Baum Plattform → Firmware-Name → Version** (zum Beispiel Plattform `MLL512V1` mit `Blink`, `Pico2x3-Tiny`, `PicoServo-Tiny`, `PicoSound-Tiny`; ein Name zeigt die neueste Version, darunter stehen alle Versionen). Der **ausgewählte Eintrag** ist **gelb hinterlegt** (seit 3.18). Rechts stehen **Kurz- und Langtext** in der Sprache der Oberfläche (sonst Englisch, sonst die erste Sprache der Liste). **Herunterladen** lädt die ZIP-Datei (`zip_path` der Liste, relativ zum Ordner `firmware` des GitHub-Archivs) in eine temporäre Datei, kopiert sie in den Ordner **`MobaLedLib/firmware`**, **entpackt** sie dort (UF2- und `.desc`-Datei) und **löscht** die ZIP-Datei; Meldung „Firmware heruntergeladen und entpackt nach: ...“. Danach liest der Dialog den Ordner neu ein und **wählt die heruntergeladene Firmware vor**; **Installieren** schreibt sie wie bisher auf den Pico (BOOTSEL). Existiert die ZIP-Datei schon, fragt das Programm vor dem Überschreiben. Bei einem Fehler beim Entpacken **bleibt die ZIP-Datei** im Ordner (zum Entpacken von Hand). Fehler (kein Netz, GitHub antwortet mit einem Fehler, falsche Liste, leere Datei) zeigt das Programm mit den Texten des Excel-Programms an. **Sicherheit (zusätzlich zum Original):** nur

`https`; absolute Adressen in der Liste nur von `github.com` und `githubusercontent.com`; beim Entpacken werden Pfade außerhalb des Ordners (`..`, absolut, Laufwerk), Verknüpfungen, mehr als 500 Dateien und Archive über 128 MB abgelehnt (dann wird nichts entpackt); Download höchstens 64 MB, Liste höchstens 2 MB. Statt PowerShell (Excel) arbeitet das Programm mit `urllib` und `zipfile` und läuft damit auch unter Linux, macOS und auf dem Raspberry Pi. **Nicht geprüft:** gegen das echte GitHub im Browser (Checkliste 171).

- **Bibliothek 3.5.0-L3, platformio.ini und Pico (neu in 3.16, Excel-Programm 3.5.0L3):** Die mitgelieferte **Bibliothek** **`libraries/MobaLedLib`** hat die Version **3.5.0-L3** (Datei `library.properties`; der Quelltext in `src` ist mit dem von 3.5.0-L **identisch**, geändert hat sich nur die Versionsangabe; neu im Ordner: `LICENSE`, `README.md`, `keywords.txt`); das Test-Suffix ist `L3` (Anzeige „Ver. 3.5.0L3“). Die **`platformio.ini`**: Der Eintrag `MobaLedLib=…` steht jetzt mit **`symlink://`** (wo der Rechner Verknüpfungen erlaubt, sonst `file://`), und die **Erweiterungen** (Extensions) stehen **einmal im Abschnitt** **`[env]`** und nicht mehr in jedem Board-Abschnitt: Alle Boards erben sie, **auch der Raspberry Pi Pico** (im Excel-Programm hieß der Fehler „fehlender Erweiterungspfad in der platformio.ini bei der PICO Plattform“). **Korrektur:** Die Zeile `#define GEN_BUTTON_RELEASE_COM` (Taster Loslassen simulieren) brach bei den Werten 2 und 3 mit einem Fehler ab (Text plus Zahl). Diese Einstellung gibt es in der Web-Oberfläche noch nicht (nur in der Konfigurationstabelle des Excel-Programms, Zeile „Taster Loslassen simulieren“, mit dem Namen `GEN_BUTTON_RELEASE_COM`).
- **Raspberry Pi Pico unter Linux:** Linux erlaubt einem normalen Benutzer den Zugriff auf den Pico im Bootloader-Modus nur mit einer **udev-Regel**. Fehlt sie, meldet das Hochladen: „RP2040 device … appears to be in BOOTSEL mode, but picotool was unable to connect. Maybe try 'sudo' or check your permissions“. Das Programm weist vor dem Hochladen darauf hin. Einrichten (einmalig): **Optionen → Optionen und spezielle Funktionen → Entwicklungsumgebung → „USB-Zugriff für den Pico einrichten“** (fragt nach dem Passwort) oder im Terminal `sudo bash install/install_pico_udev.sh`. Danach den Pico neu anstecken (BOOTSEL-Taste gedrückt halten). Das Skript legt die Regel `/etc/udev/rules.d/99-mobaledlib-pico.rules` an und nimmt den Benutzer in die Gruppen `plugdev` und `dialout` auf (gilt nach dem nächsten Anmelden).
- **PlatformIO ganz ohne ARDUINO IDE:** Ist **PlatformIO** als Entwicklungsumgebung eingestellt (Optionen → Entwicklungsumgebung), braucht das Programm die ARDUINO IDE **nicht**. Dann gilt:
  - Beim Start gibt es **keine Suche nach der IDE** (keine Meldung „ARDUINO IDE nicht gefunden“) und **keine Prüfung oder Aktualisierung der ARDUINO Bibliotheken und Boards**.
  - Der **Sketchbook-Pfad ist der Python-Ordner**: Das Programm liest die Datei `preferences.txt` der IDE nicht mehr. Der Ordner `libraries` im Python-Ordner enthält die MobaLedLib (für PlatformIO) und die Erweiterungen (Extensions). Die Version der MobaLedLib steht in `libraries/MobaLedLib/library.properties`. FastLED, NmraDcc und weitere Bibliotheken lädt PlatformIO beim ersten Übersetzen selbst.
  - **Erweiterungen (Extensions) mit PlatformIO (2.70):** Eine Erweiterung (zum Beispiel `P_ScheduleMLX`, `P_ScheduleDuoMLX`) liegt als Ordner mit der Datei `MobaLedLib.properties` im Ordner `libraries`. Das Programm trägt in die Datei `platformio.ini` und in die Kopfdatei `LEDs_AutoProg.h` nur die Erweiterungen ein, die das Programm **wirklich benutzt** (Zeilen `EX.<Name>(…)` in der Tabelle). Vorher stand der Header **aller** installierten Erweiterungen in der Kopfdatei; PlatformIO kannte aber nur die benutzten und meldete beim Übersetzen „fatal error: P_ScheduleDuoMLX.h: No such file or directory“. Die ARDUINO IDE war nicht betroffen, weil sie alle Ordner in `libraries` sieht. Ab Version 2.71 stehen die benutzten Erweiterungen auch in der Umgebung **`pico`** (Raspberry Pi Pico) der Datei `platformio.ini`; ohne Erweiterung bleibt diese Umgebung unverändert.
  - Der Reiter **Update** (Bibliotheken und Boards der IDE aktualisieren) wird ausgeblendet; er erscheint wieder, wenn man die ARDUINO IDE einstellt. Der Reiter **Bootloader** bleibt (ab 2.74, siehe Abschnitt 2.9e). Das Update des Programms selbst bleibt im Menü **Hilfe → Update webMobaLedLib**.
  - **ESP32** und **Pico** sind als Board wählbar, auch wenn ihre Pakete in der IDE fehlen.
  - Schaltet man auf die ARDUINO IDE zurück, fragt das Programm, ob die Bibliotheken und Boards jetzt geprüft werden sollen (und meldet, wenn die IDE nicht gefunden wird).
  - Weiter an die IDE gebunden ist das Übersetzen des Simulators (nur Windows). **Der schnelle Bootloader** läuft in der Webversion **nur mit PlatformIO** (Abschnitt 2.9e); mit der ARDUINO IDE meldet der Knopf im Reiter **Bootloader** weiter einen Hinweis (dafür bitte das Excel-Programm benutzen).
- **Fehler „UnknownPackageError“ beim Hochladen:** Die Paketverwaltung von PlatformIO holt beim ersten Hochladen für ein Board (ESP32, Pico) Compiler und Werkzeuge aus dem Netz und meldet manchmal für ein Paket, das es gibt, „UnknownPackageError: Could not find the package …“ (zum Beispiel `tool-mkfatfs`). Das Programm **wiederholt den Aufruf dann einmal**: Schon geladene Pakete bleiben, es wird nur das Fehlende geholt. Hilft das nicht: Internetverbindung prüfen und später noch einmal hochladen; bleibt es dabei, in einem Terminal `pio --version` und `pio pkg show <Paket>` (das Paket steht in der Meldung) ausführen und die Ausgabe mit der Protokolldatei melden.
- **Boardliste und PlatformIO:** Stellt man im Dialog Optionen die Entwicklungsumgebung auf PlatformIO um, erscheint **ESP32** in der Boardliste **sofort** (vorher erst nach dem erneuten Öffnen). Mit der ARDUINO IDE steht ESP32 nur in der Liste, wenn die IDE das ESP32-Paket hat.
- **Ordner `.pio`:** PlatformIO legt im Ordner `python` den Ordner `.pio` an (Übersetzungsergebnisse und geladene Bibliotheken, **mehrere hundert MB**). Er gehört nicht ins Release (`zip_erstellen.bat` lässt ihn weg), Update und Reparatur fassen ihn nicht an. Man kann ihn jederzeit löschen, um Platz zu schaffen; beim nächsten Hochladen wird er neu aufgebaut (das dauert dann länger).
- **Hinweis:** PlatformIO ist neu im Programm. Rückmeldungen, vor allem von Linux, macOS und dem Raspberry Pi, sind willkommen.

2.17 Anzahl der Eingänge (Excel-Programm 3.5.0-K-long)

Das Programm beruht auf dem Excel-Programm **3.5.0-K-long** der MobaLedLib. Es erlaubt je nach Board unterschiedlich viele Eingänge (Variablen): **Arduino Nano und Uno bis 250**, **ESP32 und Pico bis 2048** (Tabelle Platform_Parameters, Zeile MaxInput). Sind es zu viele, meldet das Programm beim Erzeugen „Die Anzahl der verwendeten Eingangskanäle ist zu groß“ mit der erlaubten Zahl. Dazu gehört die Bibliothek MobaLedLib in der Version **3.5.0-K-long** (Optionen → Optionen und spezielle Funktionen → Update); ein beim Laden von GitHub entstandener Ordner mit dem Zusatz `-longInpAddresses` wird dabei automatisch umbenannt.

2.17 Grafische Ansicht im Programm Generator (neu)

Über der Tabelle des Programm Generators (Blätter DCC, Selectrix, CAN und die anderen Datenblätter) gibt es drei Knöpfe mit **Symbol und Beschriftung** (seit 3.20, im Stil der Werkzeugleiste): **Tabelle**, **Grafik** und **Beides**; der Knopf der **gewählten Ansicht ist gelb hinterlegt** (Beides: Grafik oben, Tabelle unten; die **Trennlinie** dazwischen lässt sich mit Maus oder Finger ziehen: mehr Platz für die Grafik oder die Tabelle, ein **Doppelklick** setzt sie in die Mitte; die Stellung merkt sich der Browser und gilt für alle Blätter). Die Tabelle bleibt die einzige Wahrheit: Die Grafik liest dieselben Zeilen und zeichnet sie nach jeder Änderung neu, beide stimmen immer überein.

- **Ketten:** Jede Zeile mit LEDs ist ein Kästchen mit dem Symbol des Makros, dem Namen und den LED-Nummern (zum Beispiel „LED 4 bis 7“). Makros, die nur **eine Farbe** einer LED einzeln ansteuern (in der Tabelle `C1-1`, `C2-1` und so weiter, zum Beispiel Sound-Module oder einzelne Lampen), hängen ebenfalls in der Kette; sie zeigen die LED-Nummer und die Farbe, zum Beispiel „LED 3, C1-1“. Benutzt die Tabelle mehrere LED Kanäle, steht in der Spalte LED Nr „Kanal-Nummer“ (zum Beispiel `1-5`); die Grafik liest beides. Oben am Kästchen ist der Eingang, unten der Ausgang; eine Linie verbindet den Ausgang mit dem Eingang des nächsten Makros. Die Linien zeigen **nur die Reihenfolge** (die LED-Nummern), keine Signalverbindung. Für jeden **LED Kanal** gibt es eine eigene Kette (Spalte mit der Überschrift „Kanal 0“, „Kanal 1“ …).
- **Ohne LEDs:** Makros ohne LEDs (zum Beispiel Sound) stehen in einer eigenen Spalte ohne Linien und gehören zu keiner Kette.
- **Anklicken** eines Kästchens markiert die **ganze Zeile** in der Tabelle (bei „Beides“ springt die Tabelle dorthin); umgekehrt markiert die Auswahl in der Tabelle das Kästchen. **Doppelklick** öffnet die Makroauswahl dieser Zeile, wie ein Klick auf das Symbol in der Tabelle. Änderungen stehen sofort in der Tabelle. Mit der Maus über einem Kästchen erscheint der Aufruf des Makros und die Beschreibung.
- **Standardansicht:** Programm Einstellungen → **Standardansicht des Programm Generators** (Tabelle, Grafik, Beides). Voreinstellung ist **Tabelle**. Die Wahl der Knöpfe gilt für das Blatt, bis das Programm beendet wird.
- **Alle Zeilen (seit 3.21):** In der Ansicht **Ketten** zeigt das Kontrollkästchen **Alle Zeilen** in der Kopfzeile der Grafik alle **aktiven** Zeilen der Tabelle **in der Reihenfolge der Tabelle** in den Ketten, auch die **ohne LEDs** (Sound, Logik, Eingänge; gestrichelter Rand, Text „Ohne LEDs“). Eine Zeile ohne LEDs steht bei **mehreren Kanälen** in der Kette der **nachfolgenden** Zeile mit LEDs, direkt vor ihr (solche Makros gehören meist zu ihr); Zeilen hinter der letzten Zeile mit LEDs stehen am **Ende der Kette dieser letzten Zeile**; gibt es keine Zeile mit LEDs, bleiben sie in der Liste „Ohne LEDs“. **Aktiv** heißt wie beim Erzeugen des Programms: Spalte **Aktiv** nicht leer und Zeile nicht ausgeblendet (Filter); **inaktive Zeilen fehlen** in diesem Modus, auch die mit LEDs. Der Schalter gilt **je Blatt** (wird mit der Arbeitsmappe und in der Datei `.MLL_view` gesichert), Voreinstellung **aus**, und nur für die Ansicht Ketten (die **Anlage** bleibt unverändert). **Verschieben** geht weiter nur innerhalb der Kette; ein Zug, bei dem eine **andere Zeile in eine andere Kette wechseln** würde (weil Zeilen ohne LEDs an der nachfolgenden Zeile hängen), wird mit einer Meldung **abgelehnt**. Ein **neues Makro ohne LEDs**, das du in die Grafik ziehst, erscheint in der Kette der nachfolgenden Zeile mit LEDs, nicht unbedingt dort, wo du es abgelegt hast.
- **Makrobaum links:** Links in der Grafik steht die Liste aller Makros (nach Gruppen, mit Symbol; das Feld darüber sucht in Name, Gruppe und Beschreibung). Jedes Makro hat zwei Wege: **Doppelklick** oder **+** hängt es **ans Ende der Kette** an und öffnet nur den **Dialog mit den Parametern** (das Makro ist schon gewählt); der Knopf **Dialog** am Makro hängt es ebenfalls an, durchläuft aber die **geführte Eingabe des Dialogknopfs** der Werkzeugleiste: nacheinander die DCC-Adresse, der Typ (Testtasten), der Startwert, die Beschreibung, der Verteiler und zuletzt die Makroauswahl mit dem Makro vorgewählt (man kann ein anderes wählen) und die Parameter. Das entspricht genau dem Weg von Hand: Makro in die Zeile eintragen, die Zelle in Spalte D aktivieren, den Dialogknopf drücken. Welche Kette, bestimmt das markierte Kästchen: Ist eines markiert, kommt das Makro ans Ende seiner Kette (gleicher LED Kanal), sonst ans Ende der ersten Kette. Der LED Kanal im Dialog ist der der Kette. **Abbrechen legt keine Zeile an** (bei beiden Wegen). In der Tabelle ist es eine neu eingefügte Zeile direkt hinter der letzten Zeile der Kette.
- **Ziehen und Ablegen:** Ein Makro aus dem Baum lässt sich mit der Maus **auf die Grafik ziehen**. Die Stelle bestimmt, wo die neue Zeile entsteht; eine blaue Linie oder Markierung zeigt sie schon beim Ziehen:
  - **vor oder hinter einem Kästchen** (obere oder untere Hälfte): Das Makro kommt in dieser Kette direkt davor oder dahinter, in der Tabelle als Zeile direkt über oder unter der Zeile des Kästchens, mit dem LED Kanal der Kette. Die LED-Nummern der Makros dahinter ändern sich entsprechend.
  - **am Ende einer Kette** (im freien Raum unter dem letzten Kästchen): ans Ende der Kette.
  - **in der Zone „Neue Kette“** (rechts neben den Ketten; sie steht auch in einer leeren Grafik): ein neuer LED Kanal mit dem Makro als erstem Kästchen.
  - **in der Spalte „Ohne LEDs“**: für Makros ohne LEDs (zum Beispiel Sound). Nach dem Ablegen läuft wie beim Knopf **Dialog** der **ganze Dialog** (die geführte Eingabe: Adresse, Typ und Testtasten, Beschreibung, Verteiler, Makroauswahl mit dem Makro vorgewählt, Parameter); **hält man beim Loslassen die Umschalt-Taste**, läuft nur der Dialog mit den Parametern wie beim +. **Abbrechen legt keine Zeile an.**
- **Anlagenansicht (Hintergrundbild):** Über der Grafik schaltet **Ketten | Anlage** die Darstellung um (je Blatt, wird

mit der Arbeitsmappe gesichert). In der **Anlage** liegen die Kästchen **frei auf einem Bild** (zum Beispiel ein Foto oder Gleisplan der Anlage) und lassen sich mit der Maus dorthin ziehen, wo das Gerät auf der Anlage sitzt. Gestrichelte Linien zeigen die Reihenfolge der Kette (je Kette eine Farbe; nur die Reihenfolge, keine Signalverbindung).
    - **Kästchen anklicken (Ketten und Anlage):** Ein **Klick** markiert die Zeile in der Tabelle (und umgekehrt: wer in der Tabelle eine Zeile anklickt, sieht das Kästchen markiert). Ein **Doppelklick** öffnet den Dialog (die Makroauswahl). Ein **Umschalt-Doppelklick** öffnet gleich den **Dialog mit den Parametern** des Makros der Zeile, ohne die Makroauswahl; steht in der Zeile kein bekanntes Makro, öffnet sich die Makroauswahl. Wird ein Kästchen gezogen, ist es danach markiert.
- **Bild laden ...:** PNG, JPG, GIF oder WebP. Das Bild wird **im Browser verkleinert** (höchstens 2048 Pixel an der längeren Kante, als JPEG, mit Transparenz als PNG; bei einer GIF-Datei das erste Bild) und erst dann zum Programm hochgeladen. Dafür braucht das Programm **kein Pillow** (es gibt es nicht für alle Systeme). Eine Datei über 40 MB lehnt der Browser ab. Ohne Bild liegt ein Raster als Fläche. **Bild entfernen** (Symbol daneben) lässt die Positionen stehen.
    - **Bedienung:** Mausrad zoomt um die Zeigerstelle, den Hintergrund ziehen verschiebt die Fläche, **Alles zeigen** (Symbol) und die Zoom-Symbole in der Kopfzeile; Zoom und Lage bleiben beim Neuzeichnen. Ein **Doppelklick** öffnet wie in der Kettenansicht den Dialog, die rechte Maustaste das Kontextmenü, **Entf** löscht (mit Rückfrage).
    - **Makro aus dem Baum auf die Fläche ziehen:** hängt es wie das **+** an die Kette des markierten Kästchens an (sonst die erste Kette) und setzt das neue Kästchen an die Ablagestelle; mit der Umschalt-Taste nur der Dialog mit den Parametern.
    - **Größe der Kästchen:** Der **Schieber „Größe“** in der Kopfzeile stellt die Breite der Kästchen ein (30 bis 300); Symbol, Schrift und Abstände wachsen mit. Er gilt in **beiden Ansichten, aber getrennt**: In der **Kette** (Standard 140 Pixel auf dem Bildschirm) und in der **Anlage** (Standard 90 Bildpixel) hat jede Ansicht ihre eigene Größe. Je Blatt, wird mit der Arbeitsmappe gesichert (auch in der Datei `.MLL_view`); **Rückgängig** gilt auch für die Größe der Kette.
    - **Raster:** Mit dem Kontrollkästchen **Raster** rasten die Kästchen beim Loslassen auf ein Raster von der halben Kästchengröße ein (zum Ausrichten von Hand).
    - **Mehrere Kästchen wählen** (in der **Ketten- und der Anlagenansicht**): **Strg-Klick** (am Mac Cmd-Klick) nimmt ein Kästchen in die Auswahl auf oder wieder heraus; in der Anlage zieht **Umschalt + Ziehen auf dem Hintergrund** einen **Auswahlrahmen** (mit Strg zusätzlich zur bisherigen Auswahl). Ein einfacher Klick oder die Auswahl einer Tabellenzeile wählt wieder nur ein Kästchen. Die Mehrfachauswahl gehört nur zur Grafik (die Tabelle markiert weiter eine Zeile). In der **Kette** lassen sich mehrere markierte Kästchen **gemeinsam ziehen** (siehe „Kästchen verschieben“).
    - **Testtasten:** Die **Testtasten der Tabelle** (die farbigen Tasten in der Zeile eines Schalters, zum Beispiel der Taster eines Ausgangs) gibt es auch **in der Kette und in der Anlage**: Sie stehen **unter dem Symbol** des Kästchens, mit **Farbe, Text und Namen** (Hinweis) wie in der Tabelle, wachsen mit der Größe der Kästchen und haben **dieselbe Wirkung** wie in der Tabelle (Taste drücken sendet den Befehl, Loslassen meldet das Ende; Umschalter ändern Farbe und Text in Tabelle und Grafik). Ein Druck auf eine Taste wählt kein Kästchen aus und zieht nichts. Bei sehr kleinen Kästchen (**unter 50 Pixel**) sind die Tasten ausgeblendet, weil sie dort nicht mehr bedienbar sind.
    - **Mehrere gemeinsam:** Ein gewähltes Kästchen ziehen verschiebt **alle gewählten** gemeinsam (in der Anlage; keines verlässt die Fläche). **Rechte Maustaste** auf die Auswahl: **Ausrichten** (links, rechts, oben, unten, senkrecht oder waagerecht mittig, waagerecht oder senkrecht gleichmäßig verteilen; Verteilen braucht mindestens drei) und **„n Kästchen löschen“**. **Entf** löscht ebenfalls alle gewählten; das Programm fragt **einmal** nach („n Zeilen löschen?“). In der Kettenansicht gibt es bei mehreren gewählten Kästchen nur das Löschen.
    - **Rückgängig:** Der Knopf ↶ in der Kopfzeile oder **Strg+Z** (Cmd+Z) macht die letzte Änderung der **Positionen, des Ausrichtens und der Größe** rückgängig (bis zu 30 Schritte je Blatt; der Knopf zeigt die Änderung). **Nicht** rückgängig gemacht werden Änderungen an der Tabelle (Einfügen, Verschieben und Löschen von Zeilen) und alles nach dem Beenden des Programms.
    - **Neue Kästchen** (aus der Tabelle oder dem Dialog) bekommen automatisch einen Platz links oben (je Kette eine Spalte) und lassen sich danach verschieben. Die Positionen hängen an der **Zeile** (Makroaufruf): Ändert man den Makroaufruf einer Zeile, bekommt sie einen neuen automatischen Platz; Zeilen, die man löscht, verlieren ihre Position.
    - **Speichern:** Bild und Positionen stehen in der **Arbeitsmappe** (auch in der automatischen Sicherung nach einem Absturz). Zum Weitergeben oder Sichern gibt es im Menü **Datei** „ProgrammGenerator: Anlagenansicht speichern ..." (lädt eine Datei `<Blatt>.MLL_view` herunter, Bild eingebettet) und „... laden ..." (ersetzt Bild und Positionen des aktuellen Blatts). Die PGF-Datei bleibt unverändert; Bild und Positionen sind **nicht** in der PGF-Datei.
- **Symbole in hoher Auflösung:** Die Kästchen der Grafik und der Anlage zeigen **scharfe Symbole**, wo es sie schon in hoher Auflösung gibt (Ordner `icons_gross`, 200 × 200 Pixel; der Browser nimmt sie, wenn die Anzeige auf diesem Bildschirm mehr als 64 Gerätepixel braucht, also auf hochauflösenden Bildschirmen und bei großen Kästchen). Makrobaum, Tabelle und Kopfzeile nehmen die kleinen Symbole (`icons`, 64 × 64 Pixel). Wo es noch kein neues Symbol gibt, bleibt das **alte** (16 × 16). Neue Symbole in hoher Auflösung (PNG mit Alphakanal, zum Beispiel 400 × 400, gleicher Name wie das alte Symbol) erzeugt der Entwickler mit `python install/make_web_icons.py <Ordner mit den Vorlagen>` (braucht Pillow nur dafür); die fertigen Dateien liegen im Release bei.

- **Kästchen verschieben:** Ein Kästchen einer Kette lässt sich **mit der Maus ziehen**, aber **nur innerhalb seiner eigenen Kette**: vor oder hinter ein anderes Kästchen derselben Kette oder ans Ende der Kette (blaue Markierung wie beim Ablegen). In eine **andere Kette** (anderer LED Kanal) lässt es sich **nicht** ziehen, der Zeiger zeigt „verboten“. So kann man nichts zerstören; den LED Kanal ändert man bei Bedarf im Dialog der Zeile. Die Zeile wandert in der Tabelle mit (Symbole und Testtasten folgen), die LED-Nummern werden neu gerechnet. Kästchen in der Spalte „Ohne LEDs“ sind nicht ziehbar. **Mehrere markierte Kästchen** (Strg-Klick) ziehen Sie **gemeinsam**: Zieht man eines der markierten, gehen alle mit; sie behalten ihre Reihenfolge in der Kette und stehen danach zusammen an der Stelle, an der man loslässt (oder am Ende der Kette); auf dem **Papierkorb** löscht das Ablegen alle markierten (mit einer Rückfrage). Liegen die markierten Kästchen in **verschiedenen Ketten** (oder eines in „Ohne LEDs“), lässt sich nichts ziehen, und eine Meldung sagt es.
- **Löschen:** Das Kästchen auf den **Papierkorb** ziehen (er erscheint rechts, solange man ein Kästchen zieht) oder bei markiertem Kästchen die Taste **Entf** drücken. Das Programm **fragt vorher nach** („Zeile n löschen?“). Es löscht wie der Knopf „Zeilen löschen“; die LED-Nummern werden neu gerechnet.
- **Kontextmenü** (rechte Maustaste auf einem Kästchen): **Dialog öffnen** (wie der Doppelklick), **Nach oben** und **Nach unten** (ein Platz in der Kette, ohne Ziehen, auch für Tablet), **Löschen**. In der Spalte „Ohne LEDs“ nur Dialog öffnen und Löschen.
- **Stand:** In dieser Fassung kann man die Grafik ansehen, anklicken, per Doppelklick bearbeiten, Makros aus dem Baum anhängen oder ziehen, Kästchen innerhalb der Kette verschieben, Zeilen löschen und die Kästchen frei auf einem Hintergrundbild der Anlage verteilen. Das **Rückgängig** gilt für Positionen, Ausrichten und Größe, **nicht** für Änderungen an der Tabelle (Konzept: `KONZEPT_Grafische_Ansicht.md`).

3. Windows

Voraussetzungen: Windows 10 oder 11, Python 3.10 oder neuer (das Skript bietet die Installation an), Internet für die Installation der Python-Pakete.

Installation

1. Im Ordner `install` **`install_windows.bat` doppelklicken**. Für alles in einem Zug (IDE und Bibliotheken) in einer Eingabeaufforderung im Ordner `install` `install_windows.bat /ide` eingeben. Meldet Windows „Der Computer wurde durch Windows geschützt“: *Weitere Informationen → Trotzdem ausführen*.
2. Das Skript sucht Python 3.10 oder neuer. Fehlt es, bietet es die Installation von Python 3.12 mit `winget` an und sucht danach in den Standardordnern weiter, die Installation läuft dann ohne Pause weiter. Findet es Python danach nicht, meldet es das: **Dann den PC neu starten** (oder ab- und wieder anmelden) und das Skript noch einmal starten. Erst nach dem Neustart kennt Windows den neuen Suchpfad von Python.
3. Es legt die Python-Umgebung `venv_web` im Projektordner an, installiert die Pakete aus `requirements-web.txt`, erzeugt das Programmsymbol und legt auf dem Desktop die Verknüpfung **„webMobaLedLib“** an.
4. Mit `/ide` lädt es die IDE 1.8.19 (ca. 130 MB) von arduino.cc und führt den Installer still aus. Windows fragt nach der Erlaubnis (Benutzerkontensteuerung); den Treiber für den USB-Anschluss bitte zulassen. Danach installiert es FastLED und MobaLedLib. (Nur die Bibliotheken, wenn die IDE schon da ist: `/libs`.)
5. Am Ende läuft ein Test der Installation (`pi_check.py`, etwa eine Minute). Ohne ihn: `/nocheck`.

Option	Wirkung
`/ide`	ARDUINO IDE 1.8.19 laden und installieren, dazu `/libs`
`/libs`	Bibliotheken FastLED (3.9.12) und MobaLedLib mit `arduino_debug.exe` installieren
`/platformio`	PlatformIO in eine eigene Python-Umgebung `%USERPROFILE%\.platformio\penv` installieren (siehe 2.16); **seit 2.92 der Standard**, die Option bleibt gültig
`/nopio`	PlatformIO **nicht** installieren (wer nur die ARDUINO IDE benutzt)
`/nocheck`	ohne den Test der Installation am Ende

Starten: Verknüpfung auf dem Desktop doppelklicken (oder `install\start_windows.bat`). Es öffnet sich ein (minimiertes) Konsolenfenster und nach 20 bis 60 Sekunden der Browser. Das Konsolenfenster nicht schließen. **Beenden:** Knopf „Beenden“ in der Oberfläche.

Besonderheiten: Der Pfad des Projektordners sollte kein `'` und kein `&` enthalten (sonst kann die Verknüpfung nicht angelegt werden; dann `install\start_windows.bat` per Rechtsklick → *Senden an → Desktop*). Die Windows-Firewall fragt nicht, weil die Oberfläche nur auf dem eigenen Rechner lauscht (für den Zugriff aus dem Netz siehe Kapitel 8).

4. macOS

Voraussetzungen: macOS 10.13 oder neuer, Python 3.10 oder neuer (das von macOS mitgelieferte Python ist zu alt), Internet. **macOS 10.11 und 10.12 gehen nicht:** Für diese Systeme gibt es keine fertigen Pakete (zum Beispiel `orjson`), die Installation der Python-Pakete scheitert.

Getestet: MacBook Air (2024) mit Apple M3, 8 GB RAM und macOS 27.0 („Golden Gate“) mit Version 1.38 des Programms: Der Tester meldet, dass es funktioniert.

Installation

1. Im Ordner `install` **`install_mac.command` doppelklicken**. Ist es nicht ausführbar, im Terminal `chmod +x install/install_mac.command` und dann `bash install/install_mac.command`. Für alles in einem Zug: `bash install_mac.command --with-arduino-ide`.
2. Das Skript sucht Python 3.10 oder neuer und bietet bei fehlendem Python die Installation mit Homebrew an. Sonst: Python 3.12 von python.org installieren und das Skript noch einmal starten.
3. Es legt `venv_web` im Projektordner an, installiert die Pakete, erzeugt das Symbol und legt auf dem Schreibtisch das Programm **`webMobaLedLib.app`** an.
4. Mit `--with-arduino-ide` lädt es `arduino-1.8.19-macosx.zip` (ca. 200 MB) und kopiert `Arduino.app` nach *Programme*, danach installiert es FastLED und MobaLedLib. Auf Macs mit Apple-Chip fragt macOS beim ersten Start der IDE nach Rosetta: bitte zustimmen. (Nur die Bibliotheken: `--with-arduino-libs`.)
5. Am Ende läuft ein Test der Installation (ohne ihn: `--nocheck`).

Option	Wirkung
`--with-arduino-ide`	ARDUINO IDE 1.8.19 laden und installieren, dazu `--with-arduino-libs`
`--with-arduino-libs`	Bibliotheken FastLED (3.9.12) und MobaLedLib mit der IDE installieren
`--with-platformio`	PlatformIO in eine eigene Python-Umgebung `~/.platformio/penv` installieren (siehe 2.16); **seit 2.92 der Standard**, die Option bleibt gültig
`--no-platformio`	PlatformIO **nicht** installieren (wer nur die ARDUINO IDE benutzt)
`--nocheck`	ohne den Test der Installation am Ende

Starten: `webMobaLedLib` auf dem Schreibtisch doppelklicken. Beim ersten Mal fragt macOS eventuell: Rechtsklick → *Öffnen* → *Öffnen*. Die Oberfläche startet ohne Terminalfenster und öffnet den Browser. Meldungen: `webmobaledlib-console.log` und `webmobaledlib.log` im Datenordner (2.6). Zum Beobachten im Terminal: `install/start_mac.command`.
Beenden: Knopf „Beenden“ in der Oberfläche.

5. Linux-Desktop (Debian, Linux Mint, Ubuntu, Fedora, Arch …)

Das Programm läuft im Benutzerkonto, ohne Systemdienst. Du startest es mit einem Symbol, die Oberfläche öffnet sich im Browser.

Voraussetzungen: Debian 12, Linux Mint 21 / 22, Ubuntu 22.04 oder neuer (Fedora, Arch, openSUSE gehen auch, die Paketnamen setzt das Skript). **Python 3.10 oder neuer** (Mint 20 und Debian 11 sind zu alt). Ein Benutzer mit `sudo` (nur für die Systempakete und die Gruppe `dialout`). Internet.

Installation (als normaler Benutzer, **ohne** sudo):

```
cd ~/mobaledlib/python/install
bash install_linux.sh --with-arduino-ide
```

Das Skript erledigt der Reihe nach:

1. Systempakete (`python3`, `python3-venv`, `python3-pip`, `curl`, `xz-utils`), nur was fehlt, mit einer Passwortabfrage von `sudo`.
2. Prüfung auf Python 3.10 oder neuer.
3. Die Python-Umgebung `venv_web` im Projektordner mit den Paketen aus `requirements-web.txt`.
4. Der Benutzer kommt in die Gruppe **`dialout`** (Zugriff auf `/dev/ttyUSB0`, `/dev/ttyACM0`). **Danach einmal ab- und wieder anmelden.**
5. Mit `--with-arduino-ide`: die IDE 1.8.19 nach `~/arduino-1.8.19` (etwa 150 MB) und danach FastLED und MobaLedLib.
6. Ein Eintrag im Anwendungsmenü (Entwicklung / Elektronik) und ein Symbol auf dem Schreibtisch.
7. Ein Test der Installation (`pi_check.py`, etwa eine Minute).

Option	Wirkung
`--with-arduino-ide`	IDE 1.8.19 laden, dazu `--with-arduino-libs`
`--with-arduino-libs`	nur die Bibliotheken (die IDE ist schon da)
`--with-platformio`	PlatformIO in eine eigene Python-Umgebung `~/.platformio/penv` installieren (siehe 2.16); **seit 2.92 der Standard**, die Option bleibt gültig

`--no-platformio`	PlatformIO **nicht** installieren (wer nur die ARDUINO IDE benutzt)
`--no-desktop-icon`	kein Symbol auf dem Schreibtisch
`--no-dialout`	die Gruppe `dialout` nicht eintragen
`--no-check`	ohne den Test der Installation

Das Skript kann **mehrfach** laufen (zum Beispiel nach einem Update).

Starten: Symbol „webMobaLedLib“ doppelklicken (Schreibtisch oder Anwendungsmenü), oder `install/start_linux.sh`. Beim ersten Doppelklick fragt der Dateimanager (Nemo, Nautilus, Dolphin) eventuell **„Ausführen“** bzw. **„Starter vertrauen“**; sonst Rechtsklick → *Ausführen erlauben* (Mint: in den Eigenschaften *Als Programm ausführen*). **Beenden:** Knopf „Beenden“ in der Oberfläche. Zum Beobachten der Meldungen `install/start_linux.sh` im Terminal starten. Auf dem Desktop läuft die IDE mit deinem Bildschirm, es wird kein virtuelles Display gebraucht.

6. Raspberry Pi (Dauerdienst)

Der Programmgenerator läuft als Dienst auf dem Pi. Du startest ihn mit einem **Desktop-Symbol** oder öffnest ihn mit dem Browser, auf dem Pi selbst oder von jedem Rechner im Netz. Der Arduino/Pico steckt per USB am Pi. Es wird kein Tk gebraucht.

Voraussetzungen: Raspberry Pi 4 oder 5 (ein Pi 3 läuft, Kompilieren dauert dort deutlich länger); Raspberry Pi OS **64 Bit**, **Trixie** (Debian 13, Python 3.13) oder Bookworm, mit Desktop für das Symbol (Lite geht auch, dann ohne Symbol); 2 GB RAM sollten reichen, 4 GB sind sicherer; ein Benutzer mit `sudo`-Recht (im Folgenden `pi`, bei dir kann er anders heißen); Internet für die Installation.

6.1 Dateien auf den Pi bringen

Der **gesamte Projektordner** `python` kommt auf den Pi, so wie er auf dem PC läuft. Auf dem Pi liegt er **in** `~/mobaledlib`:

```
/home/pi/mobaledlib/python/                 <- Projektordner (hier liegt webmobaledlib.py)
/home/pi/mobaledlib/python/pi/              <- Installationsskripte (install_pi.sh ...)
```

So entsteht es mit einer ZIP-Datei:

1. Auf dem PC den **Ordner** `python` **selbst** als ZIP packen (nicht nur seinen Inhalt), ohne `__pycache__`.
2. Die ZIP-Datei auf den Pi in den Ordner `~/mobaledlib` kopieren (`mkdir -p ~/mobaledlib`, falls es ihn nicht gibt).
3. Dort **„Hier entpacken“**: Es entsteht automatisch der Ordner `python` mit allen Dateien darunter. (Enthält die ZIP nur den Inhalt ohne Ordner, würden die Dateien lose in `~/mobaledlib` landen: dann mit „Entpacken nach …“ in einen neuen Ordner `python` entpacken.)

Alternativ im Terminal:

```
# vom Windows-PC (PowerShell); zuvor auf dem Pi:  mkdir -p ~/mobaledlib
scp -r D:\webMobaLedLib\python pi@raspberrypi.local:/home/pi/mobaledlib/
# oder auf dem Pi:  git clone <dein Repository> ~/mobaledlib       (das Repository enthält den Ordner python)
```

6.2 Installation (ein Befehl)

```
cd /home/pi/mobaledlib/python/pi
sudo bash install_pi.sh --with-arduino-ide
```

Das Skript erledigt der Reihe nach:

1. Systempakete (Python, venv, `xvfb`, ein paar Bibliotheken für die Java-IDE),
2. eine eigene Python-Umgebung `venv` mit den Paketen aus `requirements-pi.txt`,
3. Zugriff auf die USB-Anschlüsse (Gruppe `dialout`),
4. `/etc/mobaledlib-web.env` mit Port und Passwort (Voreinstellung: Benutzer `mobaledlib`, Passwort `mobaledlib`, im Programm änderbar, siehe Kapitel 8),
5. den Starter für das Desktop-Symbol und eine sudo-Regel, mit der du **nur** den Dienst `mobaledlib-web` ohne Passwort starten, anhalten und neu starten sowie den Pi herunterfahren und neu starten darfst. Dazu kommt ein kleines Hilfsprogramm (`/usr/local/sbin/mobaledlib-set-hostname`, Besitzer root), mit dem die Oberfläche den Rechnernamen ändern darf (6.7). Das Hilfsprogramm prüft den Namen streng,
6. das **Desktop-Symbol** (6.3),
7. die ARDUINO IDE 1.8.19 (nur mit `--with-arduino-ide`, etwa 150 MB Download) und die Bibliothek MobaLedLib mit ihren Abhängigkeiten,
8. den Dienst `mobaledlib-web` (startet beim Hochfahren) und einen Test der Installation (`pi_check.py`).

Option	Wirkung
`--with-arduino-ide`	IDE 1.8.19 laden, dazu `--with-arduino-libs`
`--with-arduino-libs`	nur die Bibliotheken
`--with-platformio`	PlatformIO für den Benutzer des Dienstes in `~/.platformio/penv` installieren (siehe 2.16)
`--port 9000`	anderer Port (Voreinstellung 8080)
`--hostname webMLLraspi`	Rechnernamen des Pi ändern, danach erreichbar als `http://webMLLraspi.local:8080` (6.7)
`--password GEHEIM`	anderes Anfangspasswort festlegen (Voreinstellung: `mobaledlib`)
`--local-only`	nur auf dem Pi erreichbar, kein Passwort
`--desktop-icon` / `--no-desktop-icon`	Symbol auch ohne erkannten Desktop / kein Symbol
`--no-check`	ohne den Test der Installation

Das Skript kann **mehrfach** laufen (zum Beispiel nach einem Update). Vorhandene Einstellungen bleiben erhalten. Windows-Zeilenenden (CRLF) in den Skripten entfernt das Skript selbst. Falls schon der Start mit `$'\r': command not found` abbricht: `sed -i 's/\r$//' install_pi.sh` und noch einmal starten.

6.3 Das Desktop-Symbol

Nach der Installation liegt auf dem Desktop das Symbol **„webMobaLedLib“** (und ein Eintrag im Startmenü unter Entwicklung / Elektronik). Ein Doppelklick:

1. startet den Dienst, wenn er nicht läuft (nach einem „Beenden“ im Programm zum Beispiel),
2. wartet, bis die Oberfläche bereit ist (20 bis 60 Sekunden, dabei erscheint die Meldung „webMobaLedLib wird gestartet“),
3. öffnet die Oberfläche in einem **eigenen Chromium-Fenster** ohne Adressleiste. Fehlt Chromium, nimmt er Firefox, sonst den Standardbrowser. Mit `MLL_BROWSER=<Programm>` wählst du einen anderen.

Auf dem Pi selbst brauchst du **kein Passwort** (Zugriffe von `localhost`). Aus dem Netz gilt es weiter. Fragt der Desktop beim ersten Doppelklick, ob die Datei ausgeführt werden soll, wähle „Ausführen“. Kommt nichts, starte den Starter im Terminal, dann siehst du die Meldung: `/home/pi/mobaledlib/python/pi/start_mobaledlib.sh`.

6.4 ARDUINO IDE und Bibliotheken auf dem Pi

Die allgemeinen Hinweise stehen in 2.2 und 2.3. Besonderheiten des Pi:

- Mit `--with-arduino-ide` lädt das Skript die IDE 1.8.19 für 64-Bit-ARM (`linuxaarch64`) und packt sie nach `/opt/arduino-1.8.19`. Von Hand: die Datei `arduino-1.8.19-linuxaarch64.tar.xz` laden, nach `/opt` auspacken.
- Die Bibliothek MobaLedLib installiert das Skript **ohne sudo** als der Benutzer, unter dem der Dienst läuft (sie liegt in `~/Arduino/libraries/MobaLedLib`):

  ```
  bash ~/mobaledlib/python/pi/install_arduino_libs.sh
  sudo systemctl restart mobaledlib-web
  ```

- Boards, gleich mit Version:

  ```
  bash ~/mobaledlib/python/pi/install_arduino_libs.sh --pico           # Raspberry Pi Pico (RP2040)
  bash ~/mobaledlib/python/pi/install_arduino_libs.sh --pico=5.4.2 --esp32
  ```

  (Das ESP32-Paket lässt sich auf dem Pi nicht installieren, siehe Kapitel 10; das Programm wählt es dort seit 1.90 nicht mehr aus.)

6.5 Alltag auf dem Pi

Was	Befehl
Starten	Desktop-Symbol (oder `sudo systemctl start mobaledlib-web`)
Läuft der Dienst?	`systemctl status mobaledlib-web`
Meldungen ansehen	`journalctl -u mobaledlib-web -f` (und die Datei `webmobaledlib.log` im Datenordner, 2.6)
Neu starten / anhalten	`sudo systemctl restart mobaledlib-web` / `sudo systemctl stop mobaledlib-web`
Nicht mehr beim Hochfahren starten	`sudo systemctl disable mobaledlib-web`
Installation prüfen	`cd ~/mobaledlib/python && MLL_TK_ROOT=0 venv/bin/python`

	`pi_check.py`
Herunterfahren / Neustart	Menü **Datei → Raspberry Pi herunterfahren ...** (bzw. **neu starten ...**), siehe 6.6
Alles wieder entfernen	`sudo bash ~/mobaledlib/python/pi/uninstall_pi.sh` (Projektordner, venv und IDE bleiben)

Der Knopf **„Beenden“** in der Oberfläche beendet das Programm. Der Dienst startet danach von selbst neu; mit dem Symbol öffnest du die Oberfläche dann wieder. Soll er aus bleiben: `sudo systemctl stop mobaledlib-web`.

6.6 Raspberry Pi herunterfahren

Ein Pi sollte nicht einfach vom Strom getrennt werden: Dabei kann die SD-Karte beschädigt werden. Die Oberfläche hat deshalb im Menü **Datei** die Einträge **Raspberry Pi herunterfahren ...** und **Raspberry Pi neu starten ...** (nur auf dem Pi sichtbar).

1. Eine Rückfrage fragt: **Speichern und herunterfahren**, **Ohne Speichern** oder Abbrechen.
2. Das Programm wird wie beim Knopf „Beenden“ ordentlich abgeschlossen (Tabellen und Konfiguration sichern, Arduino trennen).
3. Es erscheint die Seite „Der Raspberry Pi wird heruntergefahren“. Danach läuft `sudo systemctl poweroff`.
4. **Warte, bis die grüne LED am Pi nicht mehr blinkt** (etwa 10 Sekunden), dann kannst du den Strom trennen.

Dafür muss der Dienstbenutzer `systemctl poweroff` und `systemctl reboot` ohne Passwort ausführen dürfen. Das legt `install_pi.sh` an (sudo-Regel) und trägt `MLL_ALLOW_SHUTDOWN=1` in `/etc/mobaledlib-web.env` ein. Bei einer älteren Installation einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` ausführen. Fehlt die Regel, sagt die Oberfläche das und beendet das Programm nicht. Im Terminal geht es auch mit `sudo shutdown -h now`.

Mit einem Taster am Pi (optional): Ein Taster zwischen GPIO 3 (Pin 5) und GND (Pin 6) kann den Pi herunterfahren und wieder starten. In `/boot/firmware/config.txt` die Zeile `dtoverlay=gpio-shutdown` ergänzen und neu starten. Das Programm wird dabei nicht gefragt: Der Dienst wird von systemd beendet, nicht gespeichert. Vorher in der Oberfläche speichern.

6.7 Den Raspberry Pi im Netz finden

Von einem anderen Rechner brauchst du die Adresse des Pi. Die einfachsten Wege, in dieser Reihenfolge:

1. **Name im Netz (mDNS): `http://raspberrypi.local:8080`.** Der Pi meldet sich mit dem Dienst `avahi-daemon` unter `<Rechnername>.local` im Netz (auf Raspberry Pi OS vorhanden, das Installationsskript installiert ihn, falls er fehlt). Das funktioniert unter Windows 10 (neuere Versionen) und 11, macOS, iPhone und iPad und unter Linux mit `avahi` bzw. `libnss-mdns` (bei Linux Mint und Ubuntu Standard). Auf Android klappt es oft nicht. Der Rechnername ist der, den du im Raspberry Pi Imager gewählt hast (sonst `raspberrypi`). Test: `ping raspberrypi.local`.
2. **Eigener, kurzer Name** (zum Beispiel `webMLLraspi`; Groß- und Kleinschreibung spielt bei der Suche keine Rolle, erlaubt sind Buchstaben, Ziffern und `-`, höchstens 31 Zeichen). Es gibt drei Wege:
   - **Im Programm: Programm Einstellungen** → Block **„Name des Raspberry Pi im Netz“** → neuen Namen eingeben → **„Namen ändern“** und die Rückfrage bestätigen. Der Name gilt sofort, ohne Neustart. Die Oberfläche zeigt danach die neue Adresse an. Die alte Adresse gilt dann nicht mehr: Lesezeichen und das Desktop-Symbol (es benutzt `localhost` und ist nicht betroffen) entsprechend anpassen. Den Block gibt es nur auf dem Pi und nur, wenn die Installation mit `install_pi.sh` ab Version 1.06 gelaufen ist (sie legt das Hilfsprogramm und die sudo-Regel an).
   - **Bei der Installation:** `sudo bash install_pi.sh --no-check --hostname webMLLraspi`.
   - **Von Hand:** `sudo raspi-config nonint do_hostname webMLLraspi` und neu starten.
3. **Der Router:** Dort steht der Pi in der Liste der Geräte, bei der FRITZ!Box unter *Heimnetz → Netzwerk → Netzwerkverbindungen* mit seiner IP-Adresse. Die FRITZ!Box löst auch Namen auf: `http://raspberrypi.fritz.box:8080`. Empfehlung: dem Pi im Router eine **feste IP-Adresse** geben (FRITZ!Box: Gerät bearbeiten → „Diesem Netzwerkgerät immer die gleiche IPv4-Adresse zuweisen“), dann ändert sie sich nicht.
4. **Suchprogramm auf dem PC:** `install/find_mobaledlib.py` probiert zuerst `raspberrypi.local` und `mobaledlib.local` und fragt dann alle Adressen des eigenen Netzes (/24) auf dem Port 8080 ab. Die Oberfläche erkennt es an der Anmeldung „webMobaLedLib“ bzw. dem Seitentitel und zeigt Adresse und Namen. Aufruf: `python install/find_mobaledlib.py` (Windows nach der Installation: `venv_web\Scripts\python install\find_mobaledlib.py`). Andere Netze oder Ports: `--subnet 192.168.178.0/24`, `--port 9000`, `--name mein-pi`.
5. **Am Pi selbst:** `hostname -I` zeigt die IP-Adresse, `hostname` den Namen. Beide stehen auch am Ende der Installation.

7. Aktualisieren und Deinstallieren

System	Aktualisieren	Deinstallieren
alle	Menü **Hilfe → Update webMobaLedLib ...** (empfohlen, siehe 7.1), oder neue Programmdateien in den Projektordner kopieren oder den Ordner `python`	–

	ersetzen (die Version steht im Dateikopf und unter dem Titel der Oberfläche), Oberfläche neu starten. **Das laufende Programm vorher beenden** (Knopf BEENDEN oben rechts): Läuft die alte Instanz noch, öffnet der Starter nur den Browser auf ihr, und es erscheint die alte Version, obwohl die Dateien neu sind. Deine Daten liegen im Datenordner und bleiben erhalten (2.6)	
Windows	Neue Pakete: `venv_web\Scripts\pip install -U -r install\requirements-web.txt`, oder das Skript noch einmal ausführen	Verknüpfung und Ordner `venv_web` löschen
macOS	`venv_web/bin/pip install -U -r install/requirements-web.txt`, oder das Skript noch einmal ausführen	`webMobaLedLib.app` und Ordner `venv_web` löschen
Linux-Desktop	`venv_web/bin/pip install -U -r install/requirements-web.txt`, oder `install_linux.sh` noch einmal	`bash install/uninstall_linux.sh` (mit `--purge` auch `venv_web`)
Raspberry Pi	`sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` (aktualisiert auch die Python-Pakete und die Dienstdatei)	`sudo bash ~/mobaledlib/python/pi/uninstall_pi.sh`

Update von einer älteren Version (Windows und macOS): Das Update führt der **Update-Code der installierten Version** aus. Versionen **bis 2.97** löschen am Ende alle Dateien, die das neue Release nicht kennt, auch wenn dieselbe Datei dort nur in **anderer Schreibweise** steht (Symbole wie `InfoB.png` und `infoB.png`; unter Windows und macOS ist das **dieselbe Datei**). Dann verschwindet die gerade geschriebene Datei, das Update meldet **„nach dem Schreiben nicht lesbar“** und stellt den **alten Stand wieder her** (es ist nichts verändert; die Meldung nennt die Datei, zum Beispiel `icons_gross/Ampel.png`). **Ab Version 3.00** enthält das Release die alten Schreibweisen zusätzlich (Liste `install/legacy_names.txt`), ein Update aus älteren Versionen klappt dann ohne Meldung; Version 2.98 und neuer löschen solche Dateien nie. **Wenn die Meldung trotzdem erscheint** (Release ohne diese Liste): Programm beenden und das ZIP **von Hand** in den Projektordner entpacken (Ordner `python` ersetzen, Anleitung 7.2), danach ist das Update-Programm neu genug.

Beim Deinstallieren bleiben Projektordner, ARDUINO IDE und Bibliotheken erhalten.

Sichern: den Datenordner (Konfiguration, Sicherungen der Tabellen, siehe 2.6) und deine Anlagendateien. Die Daten der Tabelle speicherst du mit dem Menü „Datei“ als Download im Browser (die Datei landet auf dem Rechner, an dem du sitzt). Auf dem Pi geht auch ein fertiges Abbild der SD-Karte (Raspberry Pi Imager oder `dd`).

7.1 Update über das Programm (Hilfe → Update webMobaLedLib …)

Das Programm lädt das neueste Release (`webmobaledlib.zip` aus https://github.com/haroldlinke/WebMLL/releases), vergleicht **jede Datei** mit der installierten und zeigt, was sich ändert: neue, geänderte und zu löschende Dateien, mit Versionsnummer. Erst nach **Aktualisieren** passiert etwas.

- **Nur Geändertes wird geschrieben.** Gleiche Dateien werden nicht angefasst. Das sind meist nur wenige Dateien statt des ganzen Ordners, und Virenscanner schlagen deshalb nicht an. Jede Datei wird zuerst in eine Hilfsdatei geschrieben und dann ersetzt.
- **Alles oder nichts:** Vor dem Schreiben werden die ersetzten Dateien gesichert und ein Journal angelegt (`.update_journal.json` im Ordner `python`). Zuerst kommen alle Dateien, dann werden sie noch einmal geprüft, und **erst zuletzt** `recover.py`, `webmobaledlib.py` und die Prüfliste. Schlägt ein Schritt fehl (zum Beispiel weil ein Sicherungsprogramm eine Datei sperrt), stellt das Programm sofort den alten Stand wieder her und meldet es. Bricht das Programm mitten im Update ab (Stromausfall, Prozess beendet), macht es **beim nächsten Start selbst** das Update rückgängig, bevor irgendetwas anderes geladen wird, und zeigt oben einen Hinweis (siehe 9.6).
- **Gelöscht werden nur Dateien, die ein früheres Update installiert hat** und die im neuen Release fehlen (die Liste dazu steht in `update_manifest.json` im Datenordner). Beim **ersten Update** einer Installation gibt es diese Liste noch nicht: Dann wird nichts gelöscht. Eigene Dateien im Projektordner bleiben immer erhalten.
- **Nie angefasst** werden die Python-Umgebung (`venv`, `venv_web`), `.git`, `__pycache__`, Protokolle, Konfiguration, Passwort-Datei und die Sicherungen der Tabellen, auch wenn ein ZIP solche Dateien enthalten sollte. Eine Ausnahme sind die Dateien, die das Programm **später selbst ändert**: `LEDs_AutoProg.h` und die `.cmd`-Dateien im Ordner `LEDs_AutoProg` (`Start_Arduino.cmd`, `Start_ESP32_Sub.cmd`, `Start_PICO_Sub.cmd`, `fastBuild.cmd`, `fastBuild2.cmd`) und die Datei `platformio.ini` im Ordner `python` (sie wird beim Hochladen mit PlatformIO geschrieben, Abschnitt 2.16). Sie **gehören zum Release**: Fehlt eine, installiert das Update sie. Ist sie da, ersetzt das Update sie **nicht**, auch wenn sie vom Release abweicht, und es meldet die Abweichung nicht als Unterschied oder Fehler; ebenso die Prüfung beim Start und die Reparatur (die nur fehlende Dateien wieder herstellt). Gelöscht werden sie nie. (Andere `.cmd`-Dateien, zum Beispiel `Restart_ProgGen.cmd` und die in `hex-files`, werden wie alle Dateien behandelt.)
- **Sicherung:** Ersetzte und gelöschte Dateien liegen vorher im Datenordner unter `update_backup/<Datum-Uhrzeit>/`

(die letzten 5). Zum Zurückholen die Dateien in den Projektordner zurückkopieren. Die Tabellen werden vor dem Ersetzen gesichert. Das installierte Release (ZIP) wird im Datenordner unter `release_cache` aufgehoben (die letzten zwei), damit eine Reparatur auch ohne Internet geht.

- **Python-Pakete:** Haben sich die Anforderungen geändert (`install/requirements-web.txt`, auf dem Pi `pi/requirements-pi.txt`), aktualisiert das Programm sie.
- **Name der Verknüpfungen:** Das Programm heißt seit Version 1.31 **webMobaLedLib** (vorher pyMobaLedLib). Nach einem Update über das Menü heißen Symbol und Starter noch wie bisher. Das Installationsskript (`install_windows.bat`, `install_mac.command`, `install_linux.sh`, auf dem Pi `install_pi.sh --no-check`) benennt sie um und entfernt die alten.
- **Neustart:** Nach dem Update läuft noch die alte Version. **Neu starten** sichert die Tabellen und startet das Programm neu, der offene Reiter lädt sich selbst neu. Auf dem Raspberry Pi startet der Dienst von selbst neu. Haben sich Dateien im Ordner `pi` geändert, muss dort einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` laufen (die installierte Dienstdatei ist eine Kopie).
- **Entwicklungsordner:** In einer Git-Arbeitskopie ist das Update gesperrt, damit es die Dateien nicht mit dem Release überschreibt (`MLL_ALLOW_UPDATE=1` hebt die Sperre auf). `MLL_UPDATE_URL` richtet das Update auf eine andere ZIP-Adresse.
- **Sicherheit:** Es wird nur über HTTPS geladen. Das ZIP muss den Ordner `python` mit `webmobaledlib.py` und `_web` enthalten, Pfade nach außen und Verknüpfungen werden abgelehnt, und aus dem ZIP wird nichts ausgeführt.
- **Schutz vor einem falschen oder unvollständigen ZIP:** Ein vollständiges Release enthält die **Prüfliste** `release_manifest.json` (Version, Art „vollständig“, Größe und SHA-256 jeder Datei). Das Update installiert nur, wenn die Prüfliste da ist und genau zu den Dateien im ZIP passt. Es **bricht ab, ohne etwas zu verändern**, wenn die Prüfliste fehlt (zum Beispiel bei einer Sammlung geänderter Dateien), wenn das ZIP deutlich weniger Dateien enthält als die installierte Version (weniger als 80 Prozent) oder wenn eine Datei nicht zur Prüfliste passt. Jede geschriebene Datei wird mit ihrer Prüfsumme verglichen; weicht sie ab (zum Beispiel durch ein Sicherungs- oder Virenprogramm), wird sie nicht ersetzt und als Fehler gemeldet.
- **Viele Dateien löschen (2.65):** Steht in der neuen Version vieles **nicht mehr**, was installiert ist (zum Beispiel **100 nicht mehr benutzte Symbole**), bricht das Update **nicht** mehr ab. Werden **mehr als 20 Dateien** gelöscht, zeigt der Dialog in Rot **„Achtung: Das Update löscht … Dateien (mehr als 20) …“** mit den ersten Dateinamen (die ganze Liste steht im Fenster unter „Wird gelöscht“). Der Standardknopf ist **Abbrechen**; erst der Knopf **„Ja, … Dateien löschen und aktualisieren“** führt das Update aus. Die gelöschten Dateien werden vorher wie alles andere im Datenordner gesichert (`update_backup`). Ein **unvollständiges ZIP** (deutlich weniger Dateien als installiert, zum Beispiel eine Sammlung geänderter Dateien) wird **weiter abgewiesen**, ohne zu fragen. Bei einer **älteren Version** mit vielen zu löschenden Dateien gilt der Knopf „Trotzdem … zurückgehen“ für beides.
- **Ältere Version:** Hat das ZIP eine **ältere** Version als die installierte, zeigt der Dialog in Rot **„ACHTUNG: Dieses ZIP hat eine ÄLTERE Version“**. Der Standardknopf ist dann **Abbrechen**; erst der Knopf **„Trotzdem auf Version … zurückgehen“** installiert sie. Das ist für einen gewollten Rücksprung gedacht; wurde nur versehentlich ein altes ZIP bereitgestellt, bitte abbrechen.
- **Meldung „konnte nicht geladen werden“:** Internetverbindung, Firewall oder Proxy prüfen, und ob das Release die Datei `webmobaledlib.zip` enthält. „Nicht ersetzt“ bei einer Datei heißt meist, dass ein anderes Programm sie gesperrt hat: Programm beenden und das Update wiederholen.

7.1a Testversion (Hilfe → Update Testversion …)

Für Tester und für den Entwickler lädt **Hilfe → Update Testversion …** eine Version **von GitHub, die man selbst aus einer Liste aussucht** (**Pre-Release oder fertiges Release**; **vorgewählt ist die neueste**), damit man immer die neueste Fassung hat oder gezielt eine bestimmte ausprobieren kann (seit 3.04). Das normale **Update webMobaLedLib …** lädt weiter nur das letzte **fertige** Release (GitHub liefert Pre-Releases dafür nie aus).

- **Rückfrage:** Zuerst fragt das Programm, ob man wirklich eine Testversion installieren möchte (nicht fertig geprüft, kann Fehler enthalten). Erst nach **Testversion suchen** wird GitHub gefragt. Die Rückfrage bietet **Abbrechen** als Standard an.
- **Anzeige:** Nach **Version laden** zeigt das Fenster, welche Version gefunden wurde (**Testversion: Name (Pre-Release oder Release, veröffentlicht am Datum)**), die neue Versionsnummer und die Liste der Dateien. Erst nach **Aktualisieren** passiert etwas; danach läuft alles **wie beim normalen Update** (Sicherung der Tabellen, Prüfliste, alles oder nichts, automatisches Zurückgehen bei einem Abbruch, Neustart).
- **Auswahl der Version (seit 3.04):** Nach **Testversion suchen** liest das Programm die Liste der Releases (`https://api.github.com/repos/haroldlinke/WebMLL/releases`, ohne Konto, 60 Abfragen je Stunde) und zeigt **die letzten 15** in einer Auswahlliste, **das neueste zuerst und vorgewählt**: Name, **Pre-Release** oder **Release** und Datum, dazu **neueste Version**, **installiert** (die laufende Version) und **ältere Version**. **Version laden** lädt die gewählte (ein Klick, wenn man die neueste will); **Abbrechen** schließt das Fenster, ohne etwas zu ändern. Aufgeführt werden nur Releases mit dem Anhang `webmobaledlib.zip`; Entwürfe (Draft) gelten nicht. **Seit 3.12** stehen Releases, deren Name oder Tag mit **test** beginnt (zum Beispiel `test1`: Tests des Entwicklers), **am Ende der Liste** (höchstens 5), werden **nie vorgewählt** und tragen den Zusatz **Entwicklertest**; so installiert kein Tester sie aus Versehen.
- **Ältere Version wählen:** Wählt man eine **ältere** als die installierte Version, zeigt das Fenster nach dem Laden in **roter Schrift** die Warnung „**ACHTUNG: Dieses ZIP hat eine ÄLTERE Version als die installierte!**“ und bietet **Trotzdem auf Version … zurückgehen** an (Abbrechen ist die Vorgabe). Einstellungen und Dateien können sich

beim Zurückgehen ändern; die Tabellen werden vorher gesichert.
- **Nach einer Testversion:** Das normale Update lädt später das letzte fertige Release. Ist dessen Version **älter** als die installierte Testversion, fragt das Programm ausdrücklich zurück, ob man zurückgehen möchte (wie sonst bei einer älteren Version).
- **Fehler:** Ohne Netz, hinter einem Proxy oder bei überschrittener Abfragegrenze (GitHub: 60 je Stunde) steht eine Meldung im Fenster; es wird nichts verändert.
- **Für den Entwickler:** Das Release auf GitHub als **Pre-Release** kennzeichnen (Haken „Set as a pre-release“) und die Datei `webmobaledlib.zip` (von `zip_erstellen.bat`) anhängen. Zum Prüfen mit einer eigenen Liste kann die Umgebungsvariable `MLL_UPDATE_API_URL` die Adresse der Release-Liste ersetzen.

8. Zugriff aus dem Netz und Passwort

Standardmäßig ist die Oberfläche **nur auf dem eigenen Rechner** erreichbar (`http://localhost:8080`). Auf dem **Raspberry Pi** ist sie im Netz erreichbar und mit einem Passwort geschützt. Beim Aufruf von einem anderen Rechner fragt der Browser nach Benutzername und Passwort.

Voreinstellung auf dem Pi: Benutzername **`mobaledlib`**, Passwort **`mobaledlib`**. Auf dem Pi selbst (Desktop-Symbol) ist kein Passwort nötig.

Passwort ändern (empfohlen, sobald der Pi im Netz erreichbar ist): In der Oberfläche **Programm Einstellungen** öffnen, im Block **„Zugriff über das Netz (Web-Oberfläche)“** das neue Passwort zweimal eingeben (mindestens 4 Zeichen) und **„Passwort ändern“** drücken. Es gilt sofort, ohne Neustart. Von einem anderen Rechner fragt der Browser beim nächsten Aufruf nach dem neuen Passwort. Solange noch das Standardpasswort gilt, zeigt der Block eine Warnung.

iPad und iPhone (seit 3.05): Alle Browser auf dem iPad und iPhone (Safari, Chrome, Firefox) benutzen dieselbe Browser-Engine von Apple (WebKit) und senden die **Anmeldung** (HTTP Basic) **nicht beim Aufbau der Dauerverbindung** (WebSocket) mit. Das Programm wies diese Verbindung deshalb ab; die Seite lief zunächst über eine Ersatzverbindung und brach bei größeren Aktionen (zum Beispiel beim **Wechsel des Tabellenblatts**) mit „Connection lost“ ab. Seit **3.05** setzt das Programm nach der Anmeldung ein **Sitzungs-Cookie** (`mllsession`, HttpOnly, 24 Stunden); Cookies werden beim Aufbau der Verbindung mitgesendet. Das Cookie gilt nur im **laufenden Programm**: Nach einem **Neustart** oder einer **Passwortänderung** meldet man sich neu an. **Cookies müssen erlaubt sein** (nicht „Alle Cookies blockieren“). Eine abgewiesene Verbindung steht im Protokoll einmal je Adresse („WebSocket-Verbindung von … abgewiesen: weder Anmeldung noch Sitzungs-Cookie“).

Firefox oder Chrome auf dem iPad/iPhone: gar keine Verbindung und keine Anmeldung? iOS verlangt für Apps die Berechtigung **Lokales Netzwerk**: **Einstellungen → Datenschutz & Sicherheit → Lokales Netzwerk** und den Browser einschalten (oder Safari benutzen). Auf einem echten iPad nicht geprüft.

- Das neue Passwort steht nicht im Klartext auf dem Pi, sondern nur als Hash in der Datei `web_password.json` im Datenordner (2.6). Sie geht dem Passwort aus `/etc/mobaledlib-web.env` vor.
- **Passwort vergessen:** Entweder auf dem Pi in der Oberfläche **„Eigenes Passwort entfernen“** drücken (auf dem Pi selbst braucht man kein Passwort), oder die Datei `web_password.json` im Datenordner löschen. Dann gilt wieder das Passwort aus `/etc/mobaledlib-web.env` (Voreinstellung `mobaledlib`).
- Das Anfangspasswort lässt sich bei der Installation mit `--password` festlegen, oder später in `/etc/mobaledlib-web.env` (`MLL_PASSWORD=...`).

Auf anderen Systemen (Windows, macOS, Linux-Desktop) ist die Oberfläche standardmäßig nur auf dem eigenen Rechner erreichbar und ohne Passwort. **Zugriff aus dem Netz einschalten (ab Version 1.77):** Reiter **Programm Einstellungen**, Kasten **Zugriff aus dem Netz erlauben**, Schalter einschalten. Es muss **nichts in einer Startdatei bearbeitet** werden.

- Ist noch kein Passwort eingerichtet, fragt ein Dialog danach (mindestens 4 Zeichen; leer lassen: ohne Passwort; *Abbrechen* schaltet den Schalter zurück). Der Benutzername ist `mobaledlib`. Ohne Passwort kann jeder im Heimnetz die Oberfläche benutzen und Programme auf den angeschlossenen Arduino laden. Das Passwort lässt sich im Kasten darunter jederzeit ändern.
- Danach **Jetzt neu starten**: Das Programm sichert, startet neu, und der Browserreiter lädt sich selbst neu. Die Einstellung steht in der Datei `network.json` im Datenordner (2.6).
- **Windows:** Das Einschalten legt gleich die Firewall-Regel „webMobaLedLib 8080“ an (nur Privatnetzwerke). Windows fragt dabei nach Administratorrechten: mit *Ja* bestätigen, mehr ist nicht zu bestätigen. Lässt die Firewall den Port schon zu, fragt es nicht. Gilt das Netzwerk als „Öffentlich“, weist das Programm darauf hin, es auf „Privat“ zu stellen. Das **Ausschalten** entfernt die Regel nicht (Knopf *Freigabe entfernen* im Bereich Firewall). Die Regel steht in der Windows-Firewall unter *Eingehende Regeln* (nicht unter „Zugelassene Apps“).
- Auf dem Rechner, auf dem das Programm läuft, ist kein Passwort nötig. Von einem anderen Rechner im Browser `http://<Adresse des PCs>:8080` öffnen, Benutzername und Passwort eingeben.
- Ist die Adresse von außen vorgegeben (Startparameter `--host` oder Umgebungsvariable `MLL_HOST`), ist der Schalter gesperrt, und die Seite sagt es.

Alternativ von Hand (für Fortgeschrittene; die Vorgabe geht dem Schalter vor): vor dem Start die Umgebungsvariablen `MLL_HOST=0.0.0.0` und ein Passwort `MLL_PASSWORD=...` setzen (Benutzername `MLL_USER`, Voreinstellung `mobaledlib`). Wo:

System	Wo
Windows	`set MLL_HOST=0.0.0.0` und `set MLL_PASSWORD=...` in `install\start_windows.bat`
macOS	`export ...` in der Datei `webMobaLedLib.app/Contents/MacOS/webMobaLedLib`
Linux-Desktop	`export ...` in `install/start_linux.sh`
Raspberry Pi	in `/etc/mobaledlib-web.env` (`sudo nano /etc/mobaledlib-web.env`), danach `sudo systemctl restart mobaledlib-web`

Windows im Detail: So geht der Zugriff von einem anderen Rechner im Heimnetz. Mit dem Schalter (oben) erledigen die Schritte 1 und 2 das Programm; die Beschreibung hilft, wenn etwas nicht klappt.

1. Schalter **Zugriff aus dem Netz erlauben** einschalten und neu starten. **Von Hand statt des Schalters:** In `install\start_windows.bat` nach der Zeile `set "MLL_HEADLESS=1"` die Zeilen `set "MLL_HOST=0.0.0.0"` und `set "MLL_PASSWORD=..."` einfügen. **Nicht die IP-Adresse des PCs** (zum Beispiel `192.168.178.79`) als `MLL_HOST` eintragen: Dann lauscht der Server nur auf dieser einen Adresse, und `http://localhost:8080` auf dem PC selbst geht nicht mehr. Ab Version 1.66 öffnet das Programm den Browser mit der Adresse, auf der es lauscht, in älteren Versionen kam „Seite nicht gefunden“.
2. **Windows-Firewall:** Das Einschalten des Schalters legt die Regel gleich an. Das Folgende gilt, wenn es nicht geklappt hat. Beim ersten Start fragt Windows, ob Python Verbindungen aus dem Netz zulassen darf. **Privatnetzwerke** anhaken und *Zugriff zulassen*. Wurde das weggeklickt oder gilt das Netzwerk als **Öffentlich**, antwortet der Server anderen Rechnern nicht, und deren Browser meldet einen Ladefehler. Abhilfe:

   - **Einfachster Weg (ab Version 1.67):** Reiter **Programm Einstellungen**, Bereich **Firewall**, Knopf **Zugriff aus dem Netz freigeben**. Windows fragt nach Administratorrechten (*Ja*), nach einigen Sekunden steht dort „Der Port 8080 ist freigegeben, Regel „webMobaLedLib 8080“ (Privatnetzwerke)“. Der Bereich prüft nach **Wirkung**: Er erkennt jede Regel, die eingehende Verbindungen auf den Port zulässt, auch eine von Hand angelegte oder die von Windows für Python angelegte. Regeln, deren Name mit `webMobaLedLib` beginnt (auch eine von Hand angelegte, zum Beispiel „webMobaLedLib 8080“), ersetzt oder entfernt das Programm; andere Regeln lässt es unverändert und zeigt sie an. Er warnt außerdem vor einer **Blockierregel** (gewinnt gegen jede Zulassung), wenn der Server nur auf diesem Rechner lauscht (`MLL_HOST`, Schritt 1) und wenn das Netzwerk als „Öffentlich“ eingestuft ist. **Freigabe entfernen** löscht die Regeln wieder. Die folgenden Schritte sind dasselbe von Hand. Das Programm liest die Firewall ohne Administratorrechte auf zwei Wegen (PowerShell-Befehle und die COM-Schnittstelle der Firewall). Steht dort „Der Zustand der Firewall konnte nicht gelesen werden (…)“, hat keiner der Wege geklappt; der Grund steht in Klammern und im Protokoll (**Hilfe → Logfile anzeigen**, Zeile „Firewall-Status“). Ob der Port freigegeben ist, zeigt dann nur ein Versuch vom anderen Rechner (`curl`, siehe unten).

   - Einstellungen → Netzwerk und Internet → (WLAN oder Ethernet) → Eigenschaften → Netzwerkprofil **Privat**.

   - Regel anlegen: Eingabeaufforderung **als Administrator** öffnen und eingeben (bei einem anderen Port die Nummer ersetzen; das Programm gibt den Befehl beim Start aus):

     ```
     netsh advfirewall firewall add rule name="webMobaLedLib 8080" dir=in action=allow protocol=TCP
     localport=8080 profile=private
     ```

     Entfernen: `netsh advfirewall firewall delete rule name="webMobaLedLib 8080"`. Die Regel steht in der Firewall-Oberfläche unter *Eingehende Regeln* mit der Spalte *Lokaler Port* 8080.

   - **Blockierregeln für Python:** Wurde die Rückfrage der Firewall beim ersten Start mit *Abbrechen* oder *Zugriff verweigern* beantwortet, legt Windows für `python.exe` eine Regel **Blockieren** an. Eine Blockierregel gewinnt gegen jede Zulassungsregel, auch gegen die Regel für den Port. Anzeigen (PowerShell **als Administrator**):

     ```
     Get-NetConnectionProfile
     Get-NetFirewallApplicationFilter | Where-Object { $_.Program -like "*python*" } | Get-NetFirewallRule |
     Format-Table DisplayName, Direction, Action, Enabled, Profile -AutoSize
     ```

     Die erste Zeile zeigt die Einstufung des Netzes (`NetworkCategory` muss `Private` sein, sonst `Set-NetConnectionProfile -NetworkCategory Private`). Steht in der zweiten Liste eine Zeile mit `Inbound`, `Block` und `Enabled True`, diese Regeln löschen (die Rückfrage erscheint beim nächsten Start wieder, dann *Privatnetzwerke* zulassen):

     ```
     Get-NetFirewallApplicationFilter | Where-Object { $_.Program -like "*python*" } | Get-NetFirewallRule |
     Where-Object { $_.Action -eq "Block" -and $_.Direction -eq "Inbound" } | Remove-NetFirewallRule
     ```

     Ohne PowerShell: Windows-Taste → *Windows Defender Firewall mit erweiterter Sicherheit* → *Eingehende Regeln*, nach *Name* sortieren, Einträge `python` mit Aktion *Blockieren* löschen.

   - **Andere Sicherheitssoftware** (Virenscanner mit eigener Firewall) hat eigene Regeln und muss den Port

ebenfalls zulassen.

- **Zum Eingrenzen** die Windows-Firewall für kurze Zeit ausschalten (`netsh advfirewall set allprofiles state off` als Administrator), den Test wiederholen und **sofort wieder einschalten** (`netsh advfirewall set allprofiles state on`). Geht es ohne Firewall, ist sie die Ursache; dann die Regeln oben prüfen.

3. **Prüfen**, in dieser Reihenfolge:
   - Auf dem PC: `netstat -ano | findstr :8080` zeigt eine Zeile mit `LISTENING` (bei `MLL_HOST=0.0.0.0` mit `0.0.0.0:8080`). Fehlt sie, läuft der Server nicht oder auf einem anderen Port.
   - Auf dem PC im Browser `http://192.168.178.79:8080` (die eigene Adresse, `ipconfig`). Geht das, läuft der Server richtig.
   - Auf dem anderen Rechner die Verbindung zum Port testen:
     - Windows, **PowerShell** (nicht die Eingabeaufforderung): `Test-NetConnection 192.168.178.79 -Port 8080`. `TcpTestSucceeded : False` heißt: keine Verbindung.
     - Windows 10 und 11, jede Eingabeaufforderung: `curl -v --max-time 5 http://192.168.178.79:8080`.
     - Linux und macOS: `curl -v --max-time 5 http://192.168.178.79:8080` oder `nc -vz 192.168.178.79 8080`.

     Antwortet der Server (`HTTP/1.1 401 Unauthorized` bei gesetztem Passwort, sonst Seitentext, oder `succeeded`), ist die Verbindung in Ordnung. Bei **Zeitüberschreitung** (`timed out`, `TcpTestSucceeded : False`), obwohl es auf dem PC selbst geht, werden die Pakete verworfen: Firewall (Schritt 2) oder die Geräte sehen sich nicht (Gastnetz der FRITZ!Box, „Geräte dürfen untereinander kommunizieren“ ausgeschaltet). Bei **Connection refused** lauscht nichts auf dieser Adresse und diesem Port (anderer Port, oder `MLL_HOST` stimmt nicht). `ping` taugt nicht zum Prüfen: Windows beantwortet es standardmäßig nicht.
   - Dann im Browser des anderen Rechners `http://192.168.178.79:8080`, Benutzername `mobaledlib`, Passwort wie `MLL_PASSWORD`.
4. Die Adresse des PCs ändert sich, wenn der Router sie neu vergibt. Im Router dem PC eine **feste IP-Adresse** geben (FRITZ!Box: Heimnetz → Netzwerk → Gerät bearbeiten).

Die Datei auf dem Pi sieht so aus:

```
MLL_HOST=0.0.0.0        # 127.0.0.1 = nur auf dem Pi selbst
MLL_PORT=8080
MLL_USER=mobaledlib
MLL_PASSWORD=mobaledlib # Anfangspasswort; ohne diese Zeile ist die Oberfläche im Netz für jeden offen
# MLL_LOCAL_NO_PASSWORD=0   # mit 0 gilt das Passwort auch auf dem Pi selbst
```

Nach einer Änderung des Ports einmal `sudo bash ~/mobaledlib/python/pi/install_pi.sh --no-check` aufrufen, damit das Symbol den neuen Port kennt.

Sicherheit: Wer die Oberfläche erreicht, kann Programme auf den angeschlossenen Arduino laden (und auf dem Pi den Rechner herunterfahren). Das Passwort wird ohne Verschlüsselung (kein https) übertragen, und das Standardpasswort ist allgemein bekannt. Nutze den Zugriff nur im eigenen Heimnetz, ändere das Passwort und mach den Rechner nicht aus dem Internet erreichbar. Sicherer ist `MLL_HOST=127.0.0.1` und der Zugriff über einen SSH-Tunnel: `ssh -L 8080:localhost:8080 pi@raspberrypi.local`, danach im Browser `http://localhost:8080`.

9. Fehlersuche

9.1 Alle Systeme

Beobachtung	Ursache und Abhilfe
„Port 8080 ist belegt“	Läuft die Oberfläche schon? Dann öffnet das Symbol nur den Browser. Sonst anderer Port (`--port 8081` beim Start, auf dem Pi in der env-Datei)
Browser öffnet sich nicht	Das Programm öffnet den Standardbrowser und probiert weitere Browser (2.4). Im Protokoll steht, welcher Weg geklappt hat (`Browser geöffnet (…)`) oder dass keiner ging. Einen bestimmten Browser erzwingt `MLL_BROWSER=brave-browser` (oder `google-chrome`); Standardbrowser unter Linux prüfen mit `xdg-settings get default-web-browser`. Sonst Adresse von Hand eingeben: `http://localhost:8080`. Der Start dauert auf langsamen Rechnern bis zu einer Minute, bevor der Browser sich öffnet: nicht noch einmal auf das Symbol klicken. Der Starter erkennt, dass die erste Instanz noch startet (Sperre im Datenordner), wartet bis zu 4 Minuten auf sie und öffnet dann den Browser
Nach dem Aktualisieren zeigt das Programm noch die alte Version (Titel der Oberfläche), auch nach einer Neuinstallation	Die alte Instanz läuft noch auf Port 8080, und der Starter öffnet nur den Browser auf ihr. Ab Version 1.25 sagt der Starter das (Terminal, Meldung auf dem Bildschirm und Protokoll: „ACHTUNG: Auf Port 8080 läuft noch eine Instanz …“). Beenden: Knopf BEENDEN, sonst Linux/macOS `ps aux | grep webmobaledlib` und `kill <Nummer>` (oder `pkill -f webmobaledlib.py`), Windows `netstat -ano | findstr :8080` und `taskkill /PID <Nummer> /F`. Danach neu starten. Welche Version gerade läuft, zeigt `http://localhost:8080/mll-version`
Meldung „Connection lost“, die Seite baut sich immer wieder	Meist ein **langsamer Rechner**: Der Aufbau der Seite dauert Sekunden, und der Browser braucht danach noch einmal 10 bis 30 s, bis er sich mit dem Server verbindet. Der Server

neu auf	wartet nur die Zeit `MLL_RECONNECT_TIMEOUT` (Standard 60 s) auf den Browser; ist sie zu kurz, wird die Verbindung verworfen und die Seite neu geladen. Abhilfe: einen größeren Wert setzen (zum Beispiel `MLL_RECONNECT_TIMEOUT=180 install/start_linux.sh`), einen leichteren Browser benutzen, andere Programme schließen. Die Seiten Einstellungen und Farb Test werden erst beim ersten Öffnen aufgebaut (`MLL_LAZY_TABS=0` baut alles sofort auf), das verkürzt den Start auf langsamen Rechnern deutlich. Zur Diagnose im Protokoll (2.7) die Zeilen `Seitenaufbau: …` (wo die Zeit vergeht), `Browser verbunden` und `Browser getrennt` ansehen. Läuft der Server nicht mehr, im Terminal starten (Linux/Mac `install/start_linux.sh`, Windows `install\start_windows.bat`); `webmobaledlib.log` und `webmobaledlib-crash.log` (Datenordner, 2.6) schicken. Zum Vergleich mit echtem Tk (Linux, Mac, Windows mit Bildschirm): `MLL_TK_ROOT=1 install/start_linux.sh`
Nach jeder Änderung bleibt die Tabelle einen Moment **weiß**, bevor sie wieder erscheint (langsamer Rechner)	Seit Version 1.49 bekommt der Browser bei einer Änderung nur noch die geänderten Zeilen, bei älteren Versionen die ganze Tabelle: **aktualisieren** (Hilfe → Update). Außerdem im Browser die **Hardwarebeschleunigung** prüfen (Linux: 9.4), Fenster und Browser-Zoom verkleinern, das kompakte Layout wählen (`MLL_LAYOUT=compact`). Mit `MLL_DEBUG_TIMING=1` steht im Protokoll, wie lange Aufbau und Senden dauern (`TIMING:`), das Protokoll bitte mitschicken
„Die Arduino Entwicklungsumgebung ist nicht … installiert“	IDE **1.8.19** installieren (2.2), ggf. den Pfad in „ARDUINO Einstellungen“ eintragen
Kompilieren: „MobaLedLib.h: No such file“ oder Bibliothek/Board fehlt	Bibliotheken installieren (2.3), in der Tabelle „Libraries“ prüfen
„Headerfileerstellung abgebrochen – Die Headerdatei wurde nicht geschrieben“ (Statuszeile, kein Fehlerfenster) direkt nach **Setup übertragen**; im Protokoll: `tkinter.messagebox.askyesno ohne Display: ARDUINOTest - Erzeugen der Headerdatei wird gestartet` (Version vor 3.22)	Im Reiter **ARDUINO Einstellungen** (neben „Programm Einstellungen“) ist der **oberste Haken „Test ARDUINO Programm Creation“** angehakt (Testmodus für Entwickler: nach jedem Schritt der Erzeugung kommt eine Frage). Bis 3.21 kam die Frage als Tk-Fenster, das es im Browser nicht gibt: Die Antwort war immer „Nein“ und die Erzeugung brach ab. **Seit 3.22** erscheint die Frage als normales Fenster (Ja/Nein, beim Kompilieren Ja/Nein/Abbrechen); **seit 3.23 wirkt der Haken sofort nach dem Speichern** (in 3.22 wurde er erst nach einem Neustart des Programms beachtet). **Sofort-Hilfe bei einer älteren Version:** den Haken **wegnehmen**, **Speichern** drücken und die Seite neu laden.
Hochladen mit der ARDUINO IDE endet mit `avrdude: stk500_paged_load(): (a) protocol error, expect=0x10, resp=0x4c, programmer is out of sync, avr_read(): error reading address 0x0000` und `failed to read all of flash memory` (seit 3.09 zeigt das Programm die Meldung **„Hochladen: Prüfung fehlgeschlagen“**)	Das Programm wurde **geschrieben**; gescheitert ist nur die **Prüfung** (avrdude liest den Speicher zurück und vergleicht). Es gibt **keinen** Fehler beim Schreiben (`stk500_paged_write`) oder beim Verbinden (`stk500_getsync`). Zuerst prüfen, ob die **Heartbeat LED** blinkt (die Lichtmaschine läuft dann mit dem neuen Programm). Häufigste Ursachen: der **Bootloader** passt nicht zur Einstellung (**beide** Einstellungen **„Nano Normal (old Bootloader)“** und **„Nano (neue Version)“** in **Optionen Werkzeuge → LED ARDUINO** ausprobieren); ein schlechtes USB-Kabel oder ein wackelnder Anschluss; zu schwache Stromversorgung (die LEDs nicht über den USB-Anschluss des Computers speisen, auf dem Raspberry Pi besonders wichtig); ein Nachbau-USB-Chip. Das Protokoll enthält die Zeile „Hochladen: Das Programm wurde geschrieben, nur die Pruefung … ist gescheitert“
Windows: Beim Auswählen des Anschlusses (Optionen Werkzeuge → LED ARDUINO → **USB Port erkennen**) erscheint „**Der COM Port COMx wird bereits von einem anderen Programm benutzt**“, obwohl kein anderes Programm läuft; **Nein** und danach Setup übertragen geht (Protokoll: `could not open port 'COM9': PermissionError(13, 'Zugriff verweigert')`)	Das Programm hatte den Anschluss kurz zuvor **selbst** benutzt (Blink-Zeitgeber des Auswahlfensters, Versand an die LED) und **nicht geschlossen**; Windows (CH340-Treiber) gibt ihn erst nach einem Moment frei. **Seit 3.10** schließt das Programm den Anschluss nach jeder Prüfung, sichert die Zugriffe gegeneinander und **wiederholt** das Öffnen bis zu dreimal. Kommt die Meldung weiter, ist der Anschluss wirklich belegt (zum Beispiel serieller Monitor, Farb Test, ARDUINO IDE).
Linux/Raspberry Pi: Der USB-Anschluss erscheint erst mit **„Unbekannte Ports anzeigen“**, danach „**Es ist kein LED Arduino an COM Port /dev/ttyUSB0 angeschlossen**“ (Protokoll: `EnumComPorts:'USB Serial' '/dev/ttyUSB0'`)	Die Liste erkannte Nanos nur an der **Beschreibung** (zum Beispiel „CH340“, „USB2.0-Ser“), der Nano meldet sich aber als „USB Serial“. **Seit 3.10** erkennt sie CH340/CH341, FTDI und Arduino zusätzlich an der **Hersteller-Kennung** (VID), und die Prüfung akzeptiert einen gewählten Anschluss bei einem Nano auch mit unbekannter Beschreibung. Welche Kennung der Anschluss hat: `python3 -m serial.tools.list_ports -v`.
„**Der … am COM Port … lässt sich nicht ansprechen (Fehlercode -2 oder -3)**“ (seit 3.10)	Der Anschluss ließ sich nicht zuweisen (-2) oder der Arduino nicht zurücksetzen (-3), zum Beispiel weil ein anderer Teil des Programms die gemeinsame Verbindung gleichzeitig änderte (Protokoll: `AttributeError: 'NoneType' object has no attribute 'dtr'`). Das Programm hat es **dreimal** versucht. USB-Kabel abziehen, wieder anstecken, noch einmal versuchen. Vor 3.10 hieß der Text fälschlich „von einem anderen Programm benutzt“.

Hochladen mit der ARDUINO IDE: „**Hochladen: Keine Antwort**“ (Protokoll: `stk500_getsync() attempt … not in sync: resp=0x00`, `programmer is not responding`; seit 3.10)	Der Nano antwortet dem Bootloader-Programm nicht: meist passt die Einstellung **„Nano Normal (old Bootloader)“ / „Nano (neue Version)“** nicht zum Bootloader des Nano (alt: 57600 Baud, neu: 115200 Baud). **Beide Einstellungen ausprobieren.** Sonst USB-Kabel (muss Daten übertragen), Anschluss, und dass kein anderes Programm den Anschluss benutzt. Ohne Antwort wiederholt avrdude zehnmal (je 5 Sekunden), das Hochladen dauert dann fast eine Minute.
Im Protokoll (und im Fenster Kompilieren und Hochladen) stehen **tausende Zeilen** `javax.jmdns … Bad domain name`, `dns[response …` und Hexdumps (Raspberry Pi)	Meldungen der **ARDUINO IDE**: Ihre Java-Bibliothek sucht im Netz nach Boards (JmDNS) und stolpert über den Rechnernamen des Pi (zum Beispiel `RaspiMLL-fritz-box-fritz-box-…`, durch Namenskonflikte mit der FRITZ!Box verlängert). **Seit 3.10** blendet das Programm diese Zeilen aus. Ursache beheben: einen **kurzen, festen Rechnernamen** setzen (`sudo raspi-config` → System Options → Hostname).
Programm Einstellungen → Spezialeinstellungen → „Bootloader des Nano beim Prüfen erkennen“ (seit 3.10, **seit 3.13 Vorgabe an**)	**An (Vorgabe):** Das Programm spricht nach **USB Port erkennen** den Bootloader an (STK500-Sync bei 57600 und 115200 Baud), erkennt **alt und neu** und stellt das Board passend ein, **wenn der Haken „Automatisch erkennen“ gesetzt ist** (es zeigt dazu einen Hinweis). Das Zurücksetzen und die Anfragen sind seit 3.12 auf den alten Bootloader abgestimmt (150 ms Wartezeit, Puffer leeren, GET_SYNC bis zu zehnmal im Abstand von etwa 0,3 Sekunden; Ergebnis von `bootloader_test.py` auf dem Raspberry Pi). **Aus (nur auf ausdrücklichen Wunsch):** Die Prüfung öffnet nur den Anschluss und lässt die gewählte Einstellung „old Bootloader“ / „neue Version“ **unverändert** (so verhielt sich das Programm vor 3.10, nur stellte die Automatik damals jeden Nano auf „neue Version“ um, weil die Prüfung den Bootloader nicht ansprach). **ESP32** bleibt wie bisher. Der **Nano Every** wird seit 3.14 nicht mehr unterstützt (wie im Excel-Programm 3.5.0L3). In 3.10 bis 3.12 war der Haken ausgeschaltet; wer ihn damals beim Speichern der Seite mitgespeichert hat, ist nicht betroffen (der Haken hat seit 3.13 einen neuen Namen). Das Protokoll nennt die gesendeten und empfangenen Bytes (`M07.transact: gesendet 30 20, Antwort 14 10`). Zur Fehlersuche gibt es das Hilfsprogramm `bootloader_test.py`.
„Fehler beim Lesen der Datei …/preferences.txt“ (zum Beispiel bei „Check actual versions“)	Die Einstellungsdatei der IDE fehlt auf einem frischen System (Linux, macOS, Pi). Das Programm legt sie beim Start an. Von Hand (Linux/Pi): `mkdir -p ~/.arduino15 ~/Arduino/libraries; printf 'sketchbook.path=%s/Arduino\ntarget_package=arduino\ntarget_platform=avr\nboard=uno\n' "$HOME" > ~/.arduino15/preferences.txt`
Board installieren: `NullPointerException at … BaseNoGui.onBoardOrPortChange`	Die IDE hat ein nicht installiertes Board ausgewählt. Das Programm und die Skripte setzen die Auswahl auf `arduino:avr:uno` zurück. Von Hand (Linux/Pi): `sed -i -E '/^(target_package|target_platform|board|custom_[A-Za-z0-9_.]+)=/d' ~/.arduino15/preferences.txt; printf 'target_package=arduino\ntarget_platform=avr\nboard=uno\n' >> ~/.arduino15/preferences.txt`, dann wiederholen
Board `esp32:esp32` installieren: `Tool esptool_py is not available for your operating system`	Für diese Version des ESP32-Pakets fehlen Werkzeuge für die Prozessorart (64-Bit-ARM), siehe Kapitel 10
Makroauswahl: Ein Makro wird beim Doppelklick nicht vorausgewählt, in der Liste stehen Makros doppelt, oder PatternConfig zeigt die Beschreibung des falschen Makros	Im Ordner `libraries` darf **nur** stehen, was dort hingehört (`MobaLedLib` und die **Erweiterungen**). **Jeder Ordner mit einer Datei `MobaLedLib.properties` wird als Erweiterung eingelesen** und fügt die dort beschriebenen Makros der Liste hinzu; heißen sie wie eingebaute Makros (zum Beispiel `ServoAnim`, `Pattern`), stehen diese doppelt in der Liste, und im Baum bleibt nur einer (so entstanden Einträge, die sich nicht vorauswählen ließen). Fremde oder alte Ordner (zum Beispiel `ServoAnim`, `LEDColorAnim`, `MultiLEDColorAnim`, `Pattern`, `ServoAnim2B`) aus `libraries` entfernen. Seit 2.87 wählt die Auswahl bei doppelten Einträgen den, der im Baum steht. **Seit 2.89 weist das Programm Makros einer Erweiterung ab, deren Name (oder Suchname) schon vorhanden ist, und meldet das beim Start:** „Das Makro ‚…‘ der Erweiterung ‚…‘ ist schon eingebaut (oder in der Erweiterung ‚…‘ definiert) und wird ignoriert“; dann den Ordner der Erweiterung umbenennen oder entfernen. Die Meldung kommt einmal je Programmlauf und steht im Protokoll.
Pakete lassen sich nicht installieren	Internetverbindung, Firmenproxy; im Fenster steht die Fehlermeldung von `pip`
macOS: „Die Liste der Releases konnte nicht gelesen werden: `<urlopen error [SSL: CERTIFICATE_VERIFY_FAILED] certificate verify failed: unable to get local issuer certificate>`“ (Hilfe → Update, Update Testversion, Git installieren)	**Python von python.org** bringt auf dem Mac keine Zertifikate mit. **Ab Version 2.79** erkennt das Programm das beim Start und benutzt die Zertifikate von `certifi` oder des Systems (`/etc/ssl/cert.pem`); Zertifikate, die Python selbst findet (Homebrew, Windows, Linux), bleiben unverändert. **Bei einer älteren Version** (zum Beispiel 2.59) einmal im Terminal `open "/Applications/Python 3.12/Install Certificates.command"` ausführen (den Ordner der installierten Python-Version nehmen) oder das Programm mit `SSL_CERT_FILE=$(python3 -c "import certifi; print(certifi.where())")` starten; danach klappt das Update, und ab 2.79 ist es nicht mehr nötig. Prüfen mit `python3 -c "import urllib.request as u; print(u.urlopen('https://api.github.com').status)"` (Antwort 200).
Beim Start: „Fehler beim Aktualisieren aufgetreten“, Bibliotheken oder Boards fehlen	Die Frage kommt im Browser (2.3). Die Wiederholung lädt nur, was fehlt, und endet ohne Fortschritt mit einer Meldung (ab 1.88). Die Ausgabe der IDE steht im Protokollfenster des Befehls und im Protokoll (Hilfe → Logfile anzeigen). Platinen sind groß: Internet und Platz auf dem Datenträger prüfen
ESP32 prüfen oder hochladen:	Die ESP32-Platine 1.0.4 braucht den Befehl `python`, neuere macOS-Versionen und Debian

`exec: "python": executable file not found in $PATH`	haben nur `python3`. Seit Version 1.89 legt das Programm ihn beim Start selbst an (`bin/python` im Datenordner, 2.3). Das Programm neu starten; fehlt die Datei dort, steht der Grund im Protokoll
ESP32 hochladen bleibt bei `Connecting........_____` stehen und endet mit `A fatal error occurred: Failed to connect to ESP32: Timed out waiting for packet header`	Der Chip kommt nicht in den Bootloader: auf dem ESP32-Board die Taste **BOOT** (IO0) **gedrückt halten**, sobald im Fenster „Connecting…“ erscheint, bis „Writing at …“ kommt. Viele DevKit-Nachbauten haben keinen zuverlässigen Auto-Reset (ein Kondensator von 10 µF zwischen EN und GND behebt das dauerhaft). Hilft es nicht: ein Datenkabel (kein reines Ladekabel) direkt am Rechner ohne Hub, einen anderen USB-Port, bei zwei Anschlüssen desselben Boards (`/dev/cu.usbserial-…` und `/dev/cu.SLAB_USBtoUART`) den anderen wählen. Das Programm weist vor dem Hochladen darauf hin
Beim Installieren der Python-Pakete: `Failed to build <Paket>` oder `Failed to build installable wheels`	Für dieses System gibt es keine fertige Datei (Wheel) des Pakets, und pip versucht, es zu übersetzen, was meist scheitert. Häufigste Ursachen: 32-Bit-System, sehr altes oder sehr neues Python, kein Internet. Das Linux-Skript zeigt eine Diagnose (Prozessor, Python, pip, glibc): bitte mitschicken. Pillow wird nicht mehr gebraucht (`requirements-web.txt` ab Version 1.01); bei einer älteren Fassung die Zeile `Pillow` aus `install/requirements-web.txt` löschen und das Skript noch einmal ausführen
„ARDUINO IDE nicht gefunden“ (Protokoll, Meldung beim Start, kein Kompilieren möglich)	Siehe 2.2. Die IDE 1.8.19 nach `~/arduino-1.8.19` legen, oder in „ARDUINO Einstellungen“ **IDE suchen** benutzen bzw. den vollständigen Pfad der Startdatei `arduino` eintragen. Es muss die IDE **1.8.x** sein, nicht 2.x. Die Orte, an denen gesucht wurde, stehen im Protokoll
`ModuleNotFoundError: No module named 'xyz'`	`venv_web/bin/pip install xyz` (Windows `venv_web\Scripts\pip`, Pi `venv/bin/pip`)
Konfiguration wird nicht gespeichert	Ist der Datenordner (2.6) beschreibbar? Ist er es nicht, nimmt das Programm einen Ersatzordner im temporären Verzeichnis und warnt im Protokoll. Anderen Ort einstellen: `MLL_DATA_DIR`

9.2 Windows

Beobachtung	Ursache und Abhilfe
„Python wurde nicht gefunden“ oder „Python wurde installiert, ist aber in diesem Fenster noch nicht bekannt“	**PC neu starten** (oder ab- und wieder anmelden) und das Skript noch einmal starten. Python von Hand: python.org, dabei „Add python.exe to PATH“ ankreuzen
„Der Computer wurde durch Windows geschützt“	*Weitere Informationen → Trotzdem ausführen*
Beim Start „Seite nicht gefunden“ unter `localhost:8080`, obwohl `MLL_HOST` gesetzt ist	`MLL_HOST` enthält eine IP-Adresse. Dann lauscht der Server nur dort. Besser den Schalter **Zugriff aus dem Netz erlauben** in den Programm Einstellungen benutzen und `MLL_HOST` nicht setzen, oder `MLL_HOST=0.0.0.0` verwenden, siehe Kapitel 8
Rote Meldung oben auf der Seite „Nicht alle Dateien passen zur Version …“	Es wurden nicht alle Dateien des Stands eingespielt. Jede Zeile nennt Datei, gefundene und verlangte Version. Die genannten Dateien in den **richtigen Ordner** kopieren (der Pfad steht davor, zum Beispiel `ExcelAPI/headless_sheet.py`); die Version steht in der ersten Zeile jeder Datei. Dieselbe Meldung steht beim Start im Konsolenfenster und im Protokoll
Ein anderer Rechner im Netz erreicht die Oberfläche nicht (Ladefehler, keine Antwort)	Meist die Windows-Firewall oder das Netzwerkprofil „Öffentlich“: Programm Einstellungen → Firewall → **Zugriff aus dem Netz freigeben** (ab 1.67), sonst Kapitel 8, Abschnitt „Windows im Detail“ (Regel von Hand, mit `netstat` und `curl` prüfen)
Fenster geht sofort wieder zu	`install\start_windows.bat` in einer Eingabeaufforderung starten und die Meldung lesen; außerdem `webmobaledlib.log` im Datenordner (2.6)
Verknüpfung fehlt auf dem Desktop	Pfad des Projektordners ohne `'` und `&`? Sonst `install\start_windows.bat` per Rechtsklick → *Senden an → Desktop*
IDE-Installation (`/ide`) bricht ab	Download von arduino.cc von Hand (2.2), danach `install_windows.bat /libs`

9.3 macOS

Beobachtung	Ursache und Abhilfe
„… kann nicht geöffnet werden, da es von einem nicht verifizierten Entwickler stammt“	Rechtsklick auf `webMobaLedLib` → *Öffnen* → *Öffnen* (nur beim ersten Mal)
`install_mac.command` lässt sich nicht öffnen	`chmod +x install/install_mac.command`, oder im Terminal `bash install/install_mac.command`
IDE startet nicht (Apple-Chip)	Rosetta installieren: `softwareupdate --install-rosetta`
Programm startet nichts	`install/start_mac.command` im Terminal ausführen und die Meldung lesen; `webmobaledlib-console.log` im Datenordner (2.6)

9.4 Linux-Desktop

Beobachtung	Ursache und Abhilfe
„Python 3.10 oder neuer wurde nicht gefunden“	System zu alt (Mint 20, Debian 11): neueres Python über pyenv oder deadsnakes installieren, das Skript noch einmal starten
`venv fehlgeschlagen` / `ensurepip is not available`	`sudo apt install python3-venv python3-pip`
Symbol startet nichts	`install/start_linux.sh` im Terminal ausführen und die Meldung lesen; Symbol ausführbar? (`ls -l ~/Desktop/webMobaLedLib.desktop`)
Kein serieller Anschluss / „permission denied“ auf `/dev/ttyACM0`	Gruppe `dialout`: `groups` muss `dialout` enthalten, sonst ab- und wieder anmelden (oder `sudo usermod -aG dialout $USER`)
Tabelle zeichnet sich langsam (weiße Fläche, Ruckeln)	Meist zeichnet der Browser ohne Grafikbeschleunigung auf der CPU. Chromium/Chrome: `chrome://gpu` muss bei „Canvas“ und „Rasterization“ „Hardware accelerated“ zeigen; Firefox: `about:support`, Abschnitt „Grafik“. In den Einstellungen die Hardwarebeschleunigung einschalten, bei Bedarf den Grafiktreiber installieren (`sudo ubuntu-drivers autoinstall`). Auf sehr alten Rechnern hilft ein kleineres Fenster

9.5 Raspberry Pi

Beobachtung	Ursache und Abhilfe
Symbol: „Der Dienst konnte nicht gestartet werden“	sudo-Regel fehlt: `sudo bash install_pi.sh --no-check` noch einmal, oder `sudo systemctl start mobaledlib-web`
Symbol: keine Reaktion	im Terminal `~/mobaledlib/python/pi/start_mobaledlib.sh` ausführen und die Meldung lesen; Symbol ist ausführbar? (`ls -l ~/Desktop`)
Seite lädt nicht	`systemctl status mobaledlib-web`, Adresse und Port prüfen (Port `:8080` an die Adresse anhängen), beim ersten Start 1 Minute warten
`raspberrypi.local` wird nicht gefunden	mDNS geht nicht überall: Gastnetz oder WLAN mit Geräteisolation, Firmennetz, Android. Die IP-Adresse nehmen (Router, `find_mobaledlib.py`, 6.7); auf dem Pi `systemctl status avahi-daemon`
Anmeldefenster, Passwort wird nicht akzeptiert	Benutzer `mobaledlib`; Passwort: das Standardpasswort `mobaledlib` oder das im Programm gesetzte. Vergessen: Datei `web_password.json` im Datenordner löschen (Kapitel 8)
Kein Port in der Auswahl / „permission denied“ auf `/dev/ttyACM0`	Gruppe `dialout` (der Dienst hat sie), bei Tests von Hand einmal ab- und anmelden
Kompilieren: „no X11 DISPLAY“ oder Java-Fehler	Läuft der Dienst über `xvfb-run`? (`/etc/systemd/system/mobaledlib-web.service`, `sudo apt install xvfb`)
Der Dienst startet nicht	`journalctl -u mobaledlib-web -e`; `pi_check.py` zeigt fehlende Pakete
Anmeldung am Pi per Netz geht nicht	`MLL_HOST=0.0.0.0` und Passwort in `/etc/mobaledlib-web.env` (Kapitel 8)

Der Raspberry Pi hängt sich auf, oder das Programm verschwindet ohne Meldung. Das Programm selbst bricht in solchen Fällen nicht ab: Im Protokoll (`~/.local/share/webMobaLedLib/webmobaledlib.log`) endet dann eine Zeile mitten im Satz, es folgen **Nullbytes** (in Editoren als Punkte oder Kästchen), und die Uhrzeit des nächsten Starts ist **früher** als die letzte Zeile (der Raspberry Pi hat keine Uhr mit Batterie und startet mit einer alten Zeit). Das Betriebssystem hat den Pi dann hart angehalten, meist wegen zu wenig Strom. Typische Ursachen: - **Netzteil zu schwach:** beim Pi 400 das offizielle Netzteil (5 V, 3 A) benutzen. Eine **USB-SSD oder -Festplatte** und ein **Arduino mit vielen LEDs** am USB-Anschluss belasten die Stromversorgung zusätzlich; solche Geräte besser getrennt versorgen (Arduino und LEDs mit eigenem Netzteil, SSD an einem Hub mit eigener Stromversorgung). - **Hitze:** der Pi 400 hat keinen Lüfter; bei längerer Last (Übersetzen mit der ARDUINO IDE) sollte das Gehäuse frei stehen. - **Speicher:** die ARDUINO IDE (Java) und das Programm brauchen zusammen gut 1,5 GB; weitere Programme (Browser mit vielen Tabs) schließen.

Das Programm beobachtet das ab 2.22 selbst: Im Protokoll steht beim Start eine Zeile „Raspberry Pi: Modell, get_throttled=…, Temperatur, Speicher“, danach etwa einmal pro Minute eine weitere. Meldet der Pi **Unterspannung** oder **Drosselung**, steht eine Warnung im Protokoll und beim nächsten Öffnen der Seite eine gelbe Meldung. Von Hand zeigt `vcgencmd get_throttled` die Bits (0x0 ist in Ordnung; 0x50005 heißt Unterspannung, aktuell und seit dem Start), und `dmesg | grep -i -E "volt|oom|killed"` zeigt Meldungen des Systems. Bei einem Absturz bitte das Protokoll aufheben und mitschicken.

Dateien mit Umlauten fehlen nach dem Entpacken (zum Beispiel `Pattern_Config_Examples/vorsignal_grün.png`, beim Start „9 Dateien fehlen“): Ältere ZIP-Dateien (bis 2.18) enthielten die Namen mit Umlauten ohne UTF-8-Kennzeichen;

Linux („Hier entpacken“) legte sie dann mit falschem Namen an. Abhilfe: **Hilfe → Update webMobaLedLib …** (stellt die Dateien wieder her). Ab 2.22 baut `zip_erstellen.bat` das ZIP mit Python und setzt das Kennzeichen; die Prüfung des ZIP lehnt ein ZIP ohne es ab.

9.6 Wenn nichts mehr geht (Installation beschädigt oder Update unterbrochen)

Ein Update kann abbrechen, wenn der Strom ausfällt, das Programm beendet wird oder ein Sicherungs- oder Virenprogramm Dateien sperrt. Das Update ist deshalb „alles oder nichts“ (siehe 7.1), und es gibt drei Hilfen:

1. **Hinweis „Das Update … ist abgebrochen oder fehlgeschlagen. Das Programm ist automatisch zur vorherigen Version … zurückgegangen“** (orange, oben auf der Seite): Das Programm hat selbst den alten Stand wiederhergestellt, Ihre Tabellen und Einstellungen sind nicht betroffen. Mit **OK** schließen. Das Update später noch einmal versuchen. Scheitert es wieder an derselben Datei, **Programm beenden, Sicherungs- oder Virenprogramm kurz anhalten** (oder den Ordner `python` und den Datenordner ausnehmen) und das Update wiederholen.

2. **Rote Meldung „… Dateien fehlen oder haben eine andere Größe als in der Prüfliste des Releases“:** Menü **Hilfe → Update webMobaLedLib …** ausführen. Auch bei gleicher Version vergleicht das Update jede Datei mit dem Release und **stellt fehlende und beschädigte Dateien wieder her**. Danach das Programm neu starten. (Dateien, die das Programm selbst erzeugt, zum Beispiel `LEDs_AutoProg.h`, werden beim Start nur auf Vorhandensein geprüft.)

3. **Das Programm startet gar nicht, oder rot „Das automatische Zurückgehen war nicht vollständig“: Reparaturskript.** Es braucht das Programm nicht und läuft auch, wenn `webmobaledlib.py` beschädigt ist. Das Programm vorher beenden, dann:

System	Aufruf
Windows	im Ordner `python\install` die Datei **`repair_windows.bat`** doppelklicken
Linux	`bash install/repair_linux.sh` im Ordner `python`
Raspberry Pi	`bash install/repair_linux.sh`, danach `sudo systemctl restart mobaledlib-web`
macOS	im Ordner `python/install` die Datei **`repair_mac.command`** doppelklicken

   Es macht ein abgebrochenes Update rückgängig, vergleicht die Dateien mit der Prüfliste (`release_manifest.json`), sucht ein vollständiges Release (zuerst die zwei zwischengespeicherten im Datenordner unter `release_cache`, sonst der Download von GitHub), prüft es und schreibt nur die fehlenden oder abweichenden Dateien neu. Die ersetzten Dateien werden vorher im Datenordner unter `repair_backup` gesichert. Es fragt vor dem Schreiben (**j**). **Ohne Internet:** `webmobaledlib.zip` von https://github.com/haroldlinke/WebMLL/releases laden und das Skript mit `--zip <Pfad zu webmobaledlib.zip>` aufrufen (unter Windows an `repair_windows.bat` anhängen, zum Beispiel in einer Eingabeaufforderung). Weitere Optionen: `--check-only` (nur prüfen), `--offline`, `--allow-older` (auch ein älteres Release), `--all` (auch erzeugte Dateien vergleichen).

4. **Das Update bricht mit „keine Prüfliste“, „unvollständig“ oder „würde … Dateien löschen“ ab:** Das bereitgestellte ZIP ist nicht das vollständige Release. Es wurde nichts verändert. Bitte dem Entwickler melden, bis das richtige ZIP da ist.

5. **Auch das Reparaturskript fehlt oder geht nicht** (von Hand):

   1. Das Programm beenden.
   2. Das aktuelle `webmobaledlib.zip` von https://github.com/haroldlinke/WebMLL/releases laden.
   3. Den Ordner `python` **umbenennen** (zum Beispiel in `python_defekt`), das ZIP entpacken (es enthält den Ordner `python`) und wie gewohnt starten (Windows `install\start_windows.bat`, macOS `start_mac.command`, Linux `start_linux.sh`; auf dem Raspberry Pi den Dienst neu starten: `sudo systemctl restart mobaledlib-web`).
   4. Ihre Tabellen, Einstellungen und das Passwort liegen im **Datenordner** (Abschnitt 2.6) und bleiben dabei erhalten. Die Python-Umgebung (`venv`, `venv_web`) gehört nicht zum ZIP; liegt sie im alten Ordner `python`, den Ordner `venv_web` (bzw. `venv`) in den neuen Ordner verschieben.

6. **Dateien zurückholen:** Ersetzte und gelöschte Dateien liegen vor jedem Update im Datenordner unter `update_backup/<Datum-Uhrzeit>/` (die letzten 5), die von der Reparatur ersetzten unter `repair_backup/<Datum-Uhrzeit>/`. Einzelne Dateien lassen sich von dort in den Ordner `python` zurückkopieren.

10. Was nicht oder noch nicht geht

- **ESP32 auf 64-Bit-ARM (Raspberry Pi, vermutlich auch Macs mit Apple-Chip):** Das Programm verlangt das ESP32-Boardpaket in der Version 1.0.4. Für diese Version bietet Espressif keine Werkzeuge für diese Prozessorart an, die Installation über die Boardverwaltung der IDE 1.8.19 scheitert mit „Tool esptool_py is not available for your operating system“. ESP32-Programme werden deshalb auf einem Windows- oder Intel-Rechner erzeugt und

hochgeladen. Pico und Arduino laufen auch auf dem Pi. `python3 pi/esp32_index_check.py <Version>` zeigt, für welche Versionen alle Werkzeuge für den Rechner vorhanden sind. Eine neuere Version müsste das Projekt festlegen und testen. Seit Version 1.90 wählt das Programm das ESP32-Paket auf Linux mit 64-Bit-ARM nicht mehr automatisch aus (sonst scheiterte die Installation bei jedem Start).

- **Noch nicht umgesetzte Formulare** (es gab sie in der Tk-Version): LEDAnim und der Mustereditor. Sie zeigen nichts an. **ServoAnim** gibt es seit Version 2.78 (Abschnitt 2.9f), **LEDColorAnim** seit 2.88 (Abschnitt 2.9g). **MultiLEDColorAnim** wird neu entworfen und steht **nicht mehr in der Makroliste**.
- **Weitere Reiter:** Die Ausgabe beim Kompilieren und Hochladen (der ARDUINO Monitor der Tk-Fassung) erscheint im Protokollfenster. Von den DM Servo Einstellungen fehlen das Programmieren des ATtiny (Hex-Datei über den Bus) und das Formular für den Programmieradapter (2.13).
- **Patterngenerator:** Das Senden an ein Modul (zurückgestellt, bis ein Tester es braucht), die Wirkung der Spezial Modes auf dem Modul und der Multiplexer fehlen noch (2.10). Test Pattern gibt es seit 2.90.
- **Stand der Prüfung:** Ausprobiert sind
  - **Raspberry Pi** (Raspberry Pi OS Trixie, Python 3.13): Neuinstallation mit `install_pi.sh`, Dienst, Zugriff aus dem Netz, Desktop-Symbol, IDE, Bibliotheken, Boards (ohne ESP32) und das Herunterfahren.
  - **Linux Mint** (32-Bit-Debian 12, Python 3.11, sehr langsamer Laptop) und **Ubuntu 25.10** (Python 3.13): Installation mit `install_linux.sh`, Betrieb, Update über das Menü und das kompakte Layout.
  - **Windows 10** (`install_windows.bat /ide` nach dem Neustart des PCs, Python 3.12) und **Windows 11** (Python 3.14): Installation und Betrieb.
  - **macOS:** Ein Tester setzt das Programm auf einem neueren macOS ein. Sein Fehler beim Prüfen für den ESP32 (`python` fehlt) ist seit Version 1.89 behoben, die Rückmeldung dazu steht noch aus.
  - **Nicht ausprobiert:** der Rest von **macOS** (auf macOS 10.11.6 scheitert die Installation der Python-Pakete, `orjson`; neuere Versionen sind nicht getestet) und der Download und die stille Installation der IDE unter Windows und macOS. Die Skripte für macOS liefen nur unter Linux mit einer lokalen ZIP-Datei als Ersatz.
  - **Ohne Tk** (Standard ab Version 1.33): Auf dem Pi ist das seit Langem so. Auf Windows, dem Linux-Desktop und macOS wurde bis Version 1.32 ein unsichtbares echtes Tk-Fenster angelegt. Der Ersatz ist dort ab Version 1.33 der Normalfall: **Auf Windows (Entwicklungsrechner) läuft er**, auf dem Linux-Desktop und macOS ist er noch wenig ausprobiert. Rückmeldungen sind willkommen (Protokollzeile `Tk: …`).
  - **32-Bit-Systeme** funktionieren nur, wenn für alle Python-Pakete fertige Dateien vorliegen (Pillow wird nicht mehr gebraucht). Auf dem Mint-Laptop war das der Fall.
- **Update über das Programm** gibt es ab Version 1.27. Beim ersten Update einer Installation werden keine veralteten Dateien gelöscht (7.1).
- **Kompaktes Layout:** Die Zeilenhöhe der Tabelle bleibt unverändert (22 Pixel), Dialoge sind nur über Schrift und Abstände angepasst.
- **Mehrsprachigkeit:** Die Texte, die das Web-Programm selbst ergänzt (Menüs, Dialoge, Einstellungen, Meldungen), sind nur deutsch. Die Mehrsprachigkeit folgt später.

Bei einer Fehlermeldung bitte den Text aus dem Installationsfenster und die Dateien `webmobaledlib.log` und `webmobaledlib-crash.log` aus dem Datenordner (2.6) schicken.